Dynamic Partial Reconfiguration of a RISC-V Custom Function Unit on 7-Series FPGAs

Capstone Project Report

Group 18

April 17, 2026

Contents

1 Preamble **5**
1.1 Introduction . . . 5
1.2 Problem Statement and Objectives . . . 7

2 Literature Review **10**
2.1 Foundational Survey: DPR Architectures, Methods, and Applications . . . 10
2.1.1 FPGA Architecture Evolution and the Foundations of Partial Reconfiguration 10
2.1.2 The Frame-Based Configuration Model and Partial Bitstream Structure . . . 11
2.1.3 Design Flows, Academic Toolchains, and Floorplanning . . . 12
2.1.4 Runtime Management of Reconfigurable Tasks . . . 12
2.1.5 Application Domains . . . 13
2.2 The Compile-Time Problem and the Motivation for Parallel Compilation . . . 14
2.2.1 The Monolithic Compilation Problem . . . 14
2.2.2 RapidWright: The Infrastructure Layer . . . 14
2.2.3 Latency-Insensitive Design and Pipeline Islands . . . 15
2.2.4 The Case for Incremental and Parallel Compilation . . . 15
2.3 RapidStream: Parallel Dataflow FPGA Compilation . . . 16
2.3.1 Motivation and Context . . . 16
2.3.2 RapidStream 1.0: Architecture and Pipeline . . . 16
2.3.3 RapidStream 2.0: HBM-Based Boards and SLR-Crossing Routing . . . 17
2.3.4 Relevance and Limitations with Respect to the Present Work . . . 18
2.4 Hierarchical Partial Reconfiguration (HPR) . . . 18
2.4.1 Motivation: Fixed Pages and Their Limitations . . . 18
2.4.2 Core Concept: Recombinable PR Pages . . . 19
2.4.3 The Hoplite-BF Network-on-Chip . . . 19
2.4.4 Compilation Workflow . . . 20
2.4.5 Runtime Reconfiguration with the Hoplite-BF NoC . . . 21
2.4.6 Comparison with Fixed-Page PR and RapidStream . . . 21
2.4.7 The PLD Predecessor: Incremental Refinement Compilation . . . 22
2.4.8 Relevance to the Present Work and the Decision to Limit Coverage . . . 22
2.5 ZyPR: End-to-End Build Tool and Runtime Manager for PR at the Edge . . . 23
2.5.1 Motivation and Positioning . . . 23
2.5.2 ZyCAP: The High-Throughput PCAP/ICAP Controller . . . 24
2.5.3 ZyPR Toolflow: Optimisation Levels and Build Pipeline . . . 25
2.5.4 Performance Benchmarks and Comparison . . . 26
2.5.5 Relevance to the Present Work and Implementation Note . . . 26
2.6 Other Related Works . . . 27

2.6.1 DML: Dynamic Partial Reconfiguration with Scalable Task Scheduling for Multi-Application FPGAs . . . 27
2.6.2 AMPER-X: Adaptive Mixed-Precision RISC-V Core for Embedded Applications 28
2.6.3 RV-CAP: Enabling Dynamic Partial Reconfiguration for FPGA-Based RISC-V SoC . . . 28
2.6.4 DORA: A Low-Latency Partial Reconfiguration Controller . . . 29
2.6.5 ACNNE: Adaptive Convolution Engine for CNN Acceleration with DPR . . . 30
2.7 Summary and Synthesis . . . 30

3 Background and Case Study 32
3.1 FPGA Partial Reconfiguration Fundamentals . . . 32
3.1.1 Configuration Frames and the Frame Address Register . . . 32
3.1.2 The DFX Decoupler IP . . . 33
3.1.3 Static and Reconfigurable Partitions on the Arty A7-35T . . . 33
3.2 CFU Playground and VexRiscv . . . 34
3.2.1 The CFU Playground Framework . . . 34
3.2.2 VexRiscv: Architecture and CFU Port . . . 34
3.2.3 The CFU–RP Interface in the DFX Context . . . 35
3.3 The ICAP and STARTUPE2 Primitives . . . 36
3.3.1 ICAP on 7-Series FPGAs . . . 36
3.3.2 The STARTUPE2 Primitive and the EOS Signal . . . 37
3.3.3 Partial Bitstream Structure . . . 38
3.4 System Architecture Overview . . . 39
3.4.1 Top-Level Module Decomposition . . . 39
3.4.2 Signal Flow from Processor to Reconfigurable Partition . . . 40
3.4.3 Reconfiguration Controller Hierarchy . . . 42

4 Design and Implementation 44
4.1 Design Goals . . . 44
4.2 Static Partition Design . . . 45
4.2.1 The `system_wrapper` Reconfiguration Controller . . . 45
4.2.2 Clock and Reset Distribution . . . 45
4.2.3 Board I/O Allocation . . . 46
4.3 Reconfigurable Partition Design . . . 46
4.3.1 The `cfu.v` Wrapper and DFX Decoupler Placement . . . 46
4.3.2 Reconfigurable Module Implementations . . . 47
4.3.3 Pblock Constraints and Floorplanning . . . 47
4.4 Reconfiguration Control Flow . . . 48
4.4.1 Step-by-Step Reconfiguration Sequence . . . 48
4.4.2 Current Implementation State . . . 49
4.5 Hardware Implementation Phases . . . 49
4.5.1 Phase 1 — Arty A7-35T (Standalone DPR Validation) . . . 49
4.5.2 Phase 2 — PYNQ-Z2 (Zynq-7020, Standalone Validation) . . . 55
4.5.3 Phase 3 — Arty A7 + PYNQ-Z2 Inter-Board Bridge . . . 60
4.6 DFX Tutorial Baseline Validation . . . 66
4.6.1 Tutorial Design Summary . . . 66
4.6.2 Measured Baseline Results . . . 67
4.6.3 Partial Bitstream Validation . . . 67
4.7 Partial Bitstream Storage — Current State and Roadmap . . . 67

4.7.1 Current State: In-Fabric ROM (Synthesis-Time Embedding) . . . 67
4.7.2 Near-Term Plan: QSPI Flash Fetch . . . 68
4.7.3 Longer-Term Plan: DDR3 SDRAM Storage . . . 68
4.7.4 Report Scope Clarification . . . 69

5 Software, Firmware, and RTL Source Organisation 70
5.1 Overview of the Software and HDL Stack . . . 70
5.2 CFU Playground — Top-Level Integration . . . 70
5.2.1 `top.v` — System-Level Composition . . . 71
5.3 CFU Playground — Decoupler Bridge (`cfu.v`) . . . 73
5.3.1 Signal Isolation Through the DFX Decoupler . . . 73
5.4 CFU Playground — Reconfiguration Timing Module (`recon_counter.v`) . . . 75
5.4.1 Complete Source and Annotation . . . 75
5.4.2 Counter State Machine . . . 75
5.5 CFU Playground — Reconfigurable Modules . . . 77
5.5.1 `example.v` — Multi-Function Combinational Accelerator . . . 77
5.5.2 `donut.v` — Signed Integer Multiply with Shift . . . 79
5.5.3 `dse_template.v` — SIMD Multiply-Accumulate . . . 80
5.5.4 `kws/` — Keyword Spotting Accelerator (SystemVerilog) . . . 81
5.6 CFU Playground — Firmware Sources . . . 84
5.6.1 `cfu.h` — Instruction Encoding and SW/HW Selection . . . 84
5.6.2 `software_cfu.c` — C-Language Reference Model . . . 85
5.6.3 `proj_menu.cc` — Interactive Test Menu . . . 86
5.6.4 `donut.c` — 3D Donut Rendering Benchmark . . . 87
5.7 AMD DFX Tutorial Baseline Implementation . . . 87
5.7.1 Tutorial `top.v` — Static Partition . . . 88
5.7.2 `shift_left.v` — BRAM-Based Reconfigurable Module . . . 88
5.7.3 `clocks.v` — MMCM Configuration . . . 89
5.8 Constraint Files and Physical Floorplanning . . . 90
5.8.1 Arty A7-35T Master XDC . . . 90
5.9 Build Flow Summary . . . 90
5.10 Repository Organisation . . . 91

6 Results and Analysis 92
6.1 Overview . . . 92
6.2 DFX Tutorial Baseline Results . . . 92
6.2.1 Resource Utilisation . . . 92
6.2.2 Timing Analysis . . . 93
6.3 CFU Playground System — Resource Utilisation . . . 94
6.3.1 Slice Logic and Memory . . . 94
6.3.2 Power Consumption . . . 95
6.4 CFU Playground System — Timing Analysis . . . 96
6.4.1 Clock Constraints and Timing Violation Root Cause . . . 96
6.4.2 Hold Timing . . . 97
6.5 Reconfiguration Latency Results . . . 97
6.5.1 Partial Bitstream Sizes . . . 97
6.5.2 JTAG Reconfiguration (Arty A7-35T) . . . 97
6.5.3 JTAG Reconfiguration (PYNQ-Z2) . . . 98
6.5.4 PCAP Reconfiguration (PYNQ-Z2) . . . 98

6.6 Functional Verification of the Reconfigurable CFU . . . 98
6.7 Comparative Discussion . . . 99
6.7.1 DFX Tutorial vs. Full System . . . 99
6.7.2 ICAP vs. PCAP vs. JTAG . . . 99
6.7.3 Device Utilisation Headroom . . . 100
6.8 Limitations and Caveats . . . 100

7 Conclusion and Future Work 101
7.1 Conclusion . . . 101
7.1.1 Literature and Toolflow Analysis . . . 101
7.1.2 DFX Baseline Validation . . . 102
7.1.3 Reconfigurable RISC-V CFU System . . . 102
7.2 Future Work . . . 103
7.2.1 Autonomous Partial Bitstream Delivery via SPI Flash . . . 103
7.2.2 DDR-Backed Runtime Bitstream Buffering . . . 103
7.2.3 ICAP Throughput Characterisation and Comparison with PCAP . . . 103
7.2.4 Multiple Simultaneous Reconfigurable Partition Slots . . . 104
7.2.5 Graph-Based CFU Selection and Scheduling . . . 104
7.2.6 Extension to RV32IMC and More Complex Accelerators . . . 104
7.2.7 Zynq ICAP Integration . . . 104
7.3 Summary . . . 105

Chapter 1

Preamble

Foreword

The hardware platform employed throughout this project evolved in response to practical engineering constraints encountered at each stage of the work.

Initial exploration of the Dynamic Function eXchange (DFX) workflow — including Xilinx-provided tutorials, basic partial reconfiguration demonstrations, and familiarisation with the DFX Decoupler and ICAP primitives — was conducted on the **Digilent Arty A7-35T** (Xilinx Artix-7, XC7A35T). This board served as the primary vehicle for understanding the toolflow mechanics, constraint authoring, and bitstream generation procedures for 7-series partial reconfiguration.

Once the DFX fundamentals were established, development shifted to the **Digilent Arty A7-100T** (Xilinx Artix-7, XC7A100T) for the initial CFU Playground implementations and experimental runs. The larger logic capacity of the XC7A100T provided the additional resources required to accommodate the full LiteX-generated VexRiscv SoC alongside multiple reconfigurable CFU variants and the associated DFX infrastructure — a combination that proved too resource-intensive for the XC7A35T fabric.

For the final implementation and validation phase, the project transitioned to the **Digilent PYNQ-Z2** (Xilinx Zynq-7020). This decision was motivated by two considerations: first, the Zynq-7020 offered substantially greater programmable logic resources together with an integrated ARM Processing System, relaxing the area constraints that had limited design complexity on the Artix-7 platforms; second, the PYNQ framework's Python-based overlay and driver infrastructure enabled far more effective runtime monitoring, instrumentation, and interactive control of the reconfiguration process than was achievable through the bare-metal firmware and JTAG-only interfaces available on the Arty boards.

1.1 Introduction

The proliferation of data-intensive workloads — spanning Deep Neural Networks (DNNs), real-time signal processing, and autonomous edge systems — has imposed increasingly stringent demands on computational hardware. General-purpose processors (GPPs), while versatile, frequently fail to deliver the throughput and energy efficiency required by these workloads within the area and power budgets characteristic of edge deployments. Application-Specific Integrated Circuits (ASICs), conversely, offer superior performance-per-watt but are inherently inflexible: once fabricated, their

logic cannot be altered to accommodate evolving algorithmic requirements, changing operational contexts, or post-deployment functional corrections.

Field-Programmable Gate Arrays (FPGAs) occupy a well-established middle ground in this design-space trade-off, coupling the high-performance acceleration of custom digital logic with post-fabrication reconfigurability. Unlike ASICs, FPGA-based designs can be re-synthesised, re-placed, and re-routed to realise entirely new hardware functions on an existing silicon substrate. This intrinsic reconfigurability has positioned FPGAs as a critical compute platform across diverse application domains, including cloud acceleration, network function virtualisation, robotics, and embedded inference at the edge.

A defining capability of modern FPGA architectures, supported from the early Xilinx XC6200 family through the contemporary AMD Virtex UltraScale+ and 7-series devices, is *Dynamic Function eXchange* (DFX), historically referred to as *Dynamic Partial Reconfiguration* (DPR). DFX permits the selective modification of a designated region of the programmable logic fabric — termed a *Reconfigurable Partition* (RP) — while the remainder of the design, the *static partition*, continues to operate without interruption. At the hardware level, this is achieved by writing a *partial bitstream* into the device's configuration memory through an internal primitive known as the *Internal Configuration Access Port* (ICAP). The partial bitstream, comprising a synchronisation word, configuration frame addresses, and the corresponding frame data, reprograms only those configuration frames associated with the target RP, leaving all other frames — and hence all static logic — undisturbed.

The practical significance of DFX is considerable. In resource-constrained environments, it enables hardware time-multiplexing: a set of functionally distinct hardware accelerators can share a single physical region of the FPGA fabric, with only the currently required accelerator being instantiated at any given time. This approach reduces the total area footprint relative to a monolithic design in which all accelerators are implemented simultaneously. Furthermore, DFX decouples the compilation of individual reconfigurable modules from the compilation of the static infrastructure, thereby creating opportunities for parallel and incremental synthesis workflows that can substantially reduce overall build times — a significant consideration given that monolithic FPGA compilation routinely requires hours to complete.

The interest in DFX has intensified across multiple domains in recent years. In cloud and data-centre contexts, partially reconfigurable FPGA overlays enable multi-tenant virtualisation, wherein distinct users share a single physical device by occupying different reconfigurable regions. In edge and embedded systems, DFX facilitates adaptive compute: a single low-cost FPGA can dynamically reconfigure its accelerator logic in response to changing sensor inputs, mission phases, or operating-environment parameters. In the domain of hardware security, DFX has been explored both as a defence mechanism — through runtime diversification of cryptographic cores — and as a subject of vulnerability analysis.

In parallel with these advances in FPGA reconfiguration technology, the RISC-V Instruction Set Architecture (ISA) has emerged as a transformative paradigm for customisable processor design. As a free, open-standard ISA governed by a modular specification, RISC-V permits implementers to extend the base integer instruction set (RV32I) with application-specific custom instructions without incurring proprietary licensing costs. When realised as a soft-core processor on an FPGA — such as the VexRiscv core, an RV32I implementation written in SpinalHDL and deployed within the LiteX System-on-Chip (SoC) framework — RISC-V provides a flexible software execution environment directly on the reconfigurable fabric.

A primary mechanism for accelerating domain-specific workloads on such a soft-core processor is through the integration of *Custom Function Units* (CFUs). The CFU Playground, an open-source

framework developed by Google, facilitates the design, integration, and benchmarking of CFUs that attach to the VexRiscv core via a well-defined handshake interface comprising `cmd_valid`, `cmd_ready`, `rsp_valid`, and `rsp_ready` control signals, a 10-bit `function_id` selector, and dual 32-bit operand inputs alongside a 32-bit result output. Each custom instruction encoded in the RISC-V CUSTOM0 opcode space is dispatched by the processor pipeline to the CFU, which executes the operation in hardware and returns the result within a deterministic number of clock cycles.

The convergence of these two paradigms — DFX-based partial reconfiguration and RISC-V custom instruction acceleration — presents a compelling architectural opportunity. Rather than statically binding a single CFU implementation to the processor at synthesis time, the CFU can be designated as a Reconfigurable Partition, allowing its hardware logic to be swapped at runtime via DFX. This approach enables a single physical RISC-V SoC to support a library of functionally diverse CFUs — for instance, transitioning from a byte-manipulation accelerator to a fixed-point multiply-accumulate unit — without re-synthesising or re-loading the entire bitstream. The processor, the memory subsystem, and the communication infrastructure remain operational throughout the reconfiguration process.

This project investigates the design, implementation, and validation of such a runtime-reconfigurable RISC-V co-processor architecture. The system is built upon the CFU Playground framework, targeting the Digilent Arty A7-35T development board (Xilinx Artix-7, XC7A35T) as the primary implementation platform, with baseline toolflow validation performed on the Arty A7-100T (XC7A100T) and supplementary cross-platform validation on the PYNQ-Z2 (Xilinx Zynq-7020). The AMD DFX IP suite, specifically the DFX Decoupler, is employed to isolate the reconfigurable CFU partition during bitstream loading, while the ICAP primitive and the STARTUPE2 End-of-Startup (EOS) signal are utilised for reconfiguration control and latency measurement, respectively.

1.2 Problem Statement and Objectives

Despite its demonstrated potential, the practical adoption of Dynamic Partial Reconfiguration in FPGA-based systems remains constrained by several interrelated challenges.

Toolflow complexity and accessibility. State-of-the-art DPR toolflows — including Hierarchical Partial Reconfiguration (HPR), ZyPR, and RapidStream — have each advanced the field along specific dimensions, yet each presents significant barriers to practical adoption. HPR introduces a hierarchical page-based compilation strategy with recombinable Single, Double, and Quad PR pages interconnected by a Hoplite-BF Network-on-Chip (NoC), enabling parallel per-operator synthesis and adaptive resource allocation; however, reproduction of its toolflow was found to be hindered by strict Vivado version dependencies, Linux kernel compatibility requirements, and compute-resource constraints during place-and-route. RapidStream demonstrated order-of-magnitude reductions in compile time through latency-insensitive partitioning and parallel island-based compilation using RapidWright as the underlying infrastructure; however, the transition of RapidStream to a commercial product has resulted in the revocation of all public API access, rendering independent evaluation infeasible. ZyPR provides an abstracted, end-to-end build and runtime management flow for Zynq-based SoCs, incorporating the high-throughput ZyCAP PR controller with AXI-Lite control and AXI4 DMA-based bitstream transfer to the ICAP at up to 400 MB/s; however, dependent tooling requires an educational licence whose acquisition was unsuccessful, precluding hands-on evaluation. These experiences underscore the challenge of reproducing and extending existing DPR research due to proprietary, version-sensitive, and resource-intensive toolchain dependencies.

Hardware controller requirements for non-Zynq platforms. The majority of existing DPR

controller architectures — including ZyCAP and the standard Linux FPGA Manager interface (which relies on the Processor Configuration Access Port, PCAP, on Zynq devices) — assume the availability of an integrated ARM Processing System (PS) for orchestrating bitstream transfers. On pure-FPGA platforms such as the Artix-7 family, no such PS exists. Reconfiguration must therefore be managed entirely from within the programmable logic (PL) fabric, using only the ICAP primitive and locally available memory resources. The design of a PL-autonomous reconfiguration controller that can fetch, validate, and write partial bitstreams to the ICAP without host intervention remains an open engineering challenge for 7-series devices.

Integration with soft-core processor ecosystems. While RISC-V-based DPR approaches have been explored in the literature — notably RV-CAP, which demonstrated ICAP-based reconfiguration orchestrated by a VexRiscv soft core, and AMPER-X, which employed DFX to dynamically alter the precision of arithmetic units attached to a RISC-V core — the specific integration of DFX with the CFU Playground framework has not been previously reported. The CFU Playground's standardised CFU interface and its tightly coupled relationship with the VexRiscv pipeline present both an opportunity for clean architectural partitioning and a set of design constraints around handshake signal isolation, clock domain management, and decoupler placement that must be addressed for correct DFX operation.

In light of these challenges, the objectives of the present work are defined as follows:

1. **Literature survey of state-of-the-art DPR toolflows and architectures.** A comprehensive review of existing approaches to DPR — encompassing HPR, ZyPR/ZyCAP, RapidStream, DORA, RV-CAP, AMPER-X, DML, and ACNNE — is conducted to identify the architectural principles, compilation strategies, and controller designs that inform the proposed system. Where hands-on evaluation was not feasible due to licensing, version, or resource constraints, the analysis is restricted to a literature-based assessment of the published results.

2. **Understanding of hardware controller requirements.** The ICAP primitive, the DFX Decoupler IP, and the STARTUPE2 configuration primitive on 7-series FPGAs are studied in detail to establish the hardware infrastructure necessary for controlled, non-disruptive partial reconfiguration on the Arty A7-35T.

3. **Design and implementation of a runtime-reconfigurable RISC-V co-processor.** A system architecture is proposed and implemented in which the `cfu_compute` module — the functional core of the CFU — is designated as a Reconfigurable Partition within a DFX-enabled Vivado project. The static partition comprises the VexRiscv SoC (generated by LiteX and wrapped in the `digilent_arty` module), the DFX Decoupler (instantiated within `cfu.v`), the ICAP controller infrastructure (within `system_wrapper`), and the reconfiguration latency measurement logic (`recon_counter.v` utilising the STARTUPE2 EOS signal). Multiple CFU implementations — including `example.v` (byte sum, byte swap, and bit reverse operations), `donut.v` (signed fixed-point multiply and multiply-shift-right), and `dse_template.v` — are synthesised as distinct Reconfigurable Modules (RMs), each producing an independent partial bitstream.

4. **Validation across multiple hardware platforms and reconfiguration methods.** The reconfiguration flow is validated using JTAG-based bitstream delivery on both the Arty A7-35T and the PYNQ-Z2, and additionally via PCAP-based reconfiguration through the Zynq PS on the PYNQ-Z2. Reconfiguration latency is measured using an Integrated Logic Analyser (ILA) core connected to the `recon_counter` output, which increments while the STARTUPE2 EOS signal is de-asserted during the reconfiguration window.

5. **Identification of a roadmap toward fully autonomous reconfiguration.** The current implementation employs host-driven, JTAG-based partial bitstream loading. The intended progression toward autonomous reconfiguration — first via SPI/QSPI flash fetch using the DFX Controller IP, and subsequently via DDR-backed runtime storage with DMA-based bitstream transfer to the ICAP — is documented as a defined engineering roadmap for future work.

Chapter 2

Literature Review

2.1 Foundational Survey: DPR Architectures, Methods, and Applications

The study of Dynamic Partial Reconfiguration (DPR) on Field-Programmable Gate Arrays has accumulated a substantial body of literature spanning device architecture, design methodology, runtime management, and domain-specific applications. Comprehensive surveys by Vipin and Fahmy [1][2] consolidate this body of work and provide a structured taxonomy that remains relevant to contemporary research. The discussion in this section draws principally on those surveys, supplemented by primary literature where appropriate.

2.1.1 FPGA Architecture Evolution and the Foundations of Partial Reconfiguration

A modern FPGA can be conceptually decomposed into two co-dependent layers: a hardware layer comprising Configurable Logic Blocks (CLBs), Block RAMs (BRAMs), Digital Signal Processing (DSP) slices, I/O blocks, and high-speed transceivers; and a configuration layer comprising the Static Random-Access Memory (SRAM) cells that encode the logic functions, routing multiplexer states, and initial memory contents of those resources. Because the configuration layer is implemented in volatile SRAM, the device can be reprogrammed after fabrication by overwriting the contents of those memory cells through a dedicated configuration interface.

Xilinx introduced support for Partial Reconfiguration (PR) as early as 1997 with the XC6200 series, which exposed a register-mapped interface through which an external host could read or write individual functional cells [3]. The architecture's regular tiled structure facilitated module relocation, making it the canonical early example of fine-grained, cell-level reconfigurability. However, the XC6200 family was ultimately discontinued, and mainstream PR support on Xilinx devices did not re-emerge at scale until the Virtex-II generation (2002–2003).

The Virtex-II architecture organised its programmable resources in a columnar fashion, with each column containing a single type of primitive — CLBs, BRAMs, or distributed multipliers. Configuration data for these resources is conveyed using a *frame* abstraction: each frame is a narrow column of bits extending the full height of the device, and the complete configuration of the device is expressed as an ordered sequence of frames addressed by a three-field tag comprising the block type, the major column address, and the minor address within that column. This frame-addressed architecture has

been preserved, with refinements, in all subsequent 7-series, UltraScale, and UltraScale+ devices.

The Virtex-II generation simultaneously introduced the Internal Configuration Access Port (ICAP), a synchronous parallel interface that allows configuration frames to be written to the device's configuration memory from logic executing within the programmable fabric itself [4]. Prior to ICAP, all reconfiguration had to be driven from an external host via SelectMAP or JTAG. ICAP — accessed through the ICAPB primitive (8-bit interface) on Virtex-II and the ICAP2 primitive (32-bit, 400 MB/s) from Virtex-4 onward — unlocked the possibility of self-reconfiguring systems in which a soft-core processor or dedicated state machine could manage the reconfiguration process without host intervention.

Virtex-4 (2004) introduced several refinements that directly shape the design of DPR systems. The tri-state buffer (TBUF) primitive — previously used as an anchor for bus macros, the interface signals between static and reconfigurable partitions — was replaced by LUT-based implementations, improving placement flexibility. The frame size was standardised to one bit wide and 16 CLBs high with 41 32-bit configuration words, relaxing the full-height column constraint to multiples of 16 CLBs. The ICAP interface width was increased from 8 to 32 bits, quadrupling throughput. These changes collectively improved the practicality of defining Reconfigurable Partitions (RPs) of varying heights and widths, while reducing the area granularity at which partial reconfiguration could be applied.

The Virtex-5 family and its successors (Virtex-6, 7-series, and their UltraScale derivatives) maintained and extended the columnar arrangement, adding rows of heterogeneous resources (DSPs, BRAMs, I/O columns) distinct from the CLB fabric. This heterogeneous layout introduces an important constraint for RP floorplanning: a partition must generally enclose complete columns of each resource type that it uses, since frames are column-aligned. On the 7-series (which encompasses the Artix-7 used in this project), a minimal RP must occupy at least one complete column of CLBs and any associated resource columns within its bounding box.

The Zynq-7000 SoC (2013) introduced a different reconfiguration model by integrating a dual-core ARM Cortex-A9 Processing System (PS) on the same die as the reconfigurable Programmable Logic (PL) fabric. On Zynq, the PS can drive either the Processor Configuration Access Port (PCAP), which offers a DMA-based path to the PL configuration memory without occupying ICAP, or delegate to an ICAP controller embedded in the PL. The distinction between PCAP and ICAP paths on Zynq is central to the comparison between the ZyCAP architecture (§2.5) and standalone ICAP-based controllers on pure FPGA families such as the Artix-7.

The UltraScale and UltraScale+ families extended DPR to additional resource types including PLLs, I/O buffers, and SERDES transceivers, and introduced the MCAP (Media Configuration Access Port) tethered to the PCIe hard macro. These improvements came at the cost of a two-phase reconfiguration protocol: before any new module bitstream is loaded, a *clearing bitstream* must first be written to reset the RP's configuration state, adding an additional latency step.

2.1.2 The Frame-Based Configuration Model and Partial Bitstream Structure

A partial bitstream targets a contiguous subset of the device's frame address space. Its structure follows the same synchronisation and instruction sequence as a full bitstream, beginning with a synchronisation word (0xAA995566 for 7-series devices), followed by a series of configuration register writes and FDRI (Frame Data Register In) write commands that deliver the new frame data. The CRC register (RCRC instruction) and the DGHIGH/GCAPTURE sequences bracket each configu-

ration block to maintain internal consistency [5].

For a Reconfigurable Partition defined over a given rectangular bounding box, the partial bitstream contains exactly the frames whose major column addresses fall within that box, and whose minor addresses cover the full height of each column. The ICAP controller writes these frames sequentially through its 32-bit data port, asserting chip-select and write-enable signals and deasserts them at the conclusion of the sequence. The static partition — comprising all frames outside the RP's address range — is never touched during this write sequence, which is the fundamental hardware guarantee that makes non-disruptive partial reconfiguration possible.

2.1.3 Design Flows, Academic Toolchains, and Floorplanning

Vendor DPR design flows have evolved from PlanAhead (the original Xilinx PR flow) to the modern Vivado Dynamic Function eXchange (DFX) implementation flow. In the current flow, the designer specifies an RP by drawing a Pblock constraint within the Vivado floorplanner, designating it as the container for a set of Reconfigurable Modules (RMs), and then performing a configuration run for each RM. Vivado enforces the requirement that the RP extends to complete column heights for each resource type it contains, and that the static partition is fully placed and routed before any RM is compiled.

Several academic toolchains have addressed limitations of vendor flows, particularly in the areas of module relocation, runtime placement, and bitstream manipulation:

- **GoAhead** [6] extends the Xilinx PR flow to support the connection and disconnection of reconfigurable modules to pre-existing static logic, addressing the bus macro routing constraint through explicit repositioning.
- **PARBIT** [7] operates on configuration bitstreams directly, extracting only the configuration frames associated with a user-defined region of interest. This enables an offline workflow in which subsets of a full bitstream are extracted without re-running the full implementation.
- **REPLICA** and **ReplicaPro** [8][9] provide online bitstream manipulation capabilities, enabling runtime relocation by manipulating the frame address fields within a stored bitstream, with ReplicaPro extending support to BRAMs and multiplier blocks.
- **OpenPR** [10] and **CoPR** [11] provide higher-level PR design flows, with CoPR automating low-level bitstream management through a configuration manager, abstracting reconfiguration from the application programmer in a manner analogous to OpenCL memory management.

Floorplanning remains a critical and often manually intensive step in DPR flows, since vendor tools do not automate RP placement. The academic literature has proposed multiple automated approaches, ranging from resource-aware fixed-outline algorithms to mixed-integer linear programming formulations. However, most early works assume a homogeneous resource layout, which is not representative of modern FPGAs. Post-2010 contributions have addressed heterogeneous resource distributions, though practical automation in the context of Vivado's DFX flow remains an active research challenge.

2.1.4 Runtime Management of Reconfigurable Tasks

Efficient management of multiple reconfigurable tasks throughout system operation introduces scheduling and memory management challenges. The early literature on task scheduling for DPR systems assumed homogeneous FPGA resources and modelled the problem as a two-dimensional bin-packing

variant, in which hardware tasks must be placed into available RP slots in space and time. Partitioning strategies have evolved from simple simulated annealing approaches to connectivity-graph-based methods that account for inter-module communication patterns and heterogeneous resource needs.

Software-based management frameworks represent one category of solution: the CoPR and ReconOS [12] models expose a standardised interface to application software, hiding reconfiguration behind API calls. Hardware-based management, by contrast, offers higher throughput by leveraging DMA-capable reconfiguration controllers — at the cost of controller design complexity and reduced scheduling flexibility. Hybrid approaches, in which a software scheduler issues DMA commands to a hardware controller, are considered the most practical for production systems, and are reflected in the ZyCAP architecture discussed in §2.5.

2.1.5 Application Domains

The literature surveys a broad set of application domains in which DPR has been explored to practical benefit:

- **Software-Defined Radio (SDR) and cognitive radio systems**: The ability to swap modulation and baseband processing modules at runtime — switching between OFDM, CDMA, or bespoke waveforms — maps directly to PR without requiring separate FPGA devices for each standard [13].
- **Automotive and safety-critical systems**: PR has been investigated for driver-assistance systems requiring runtime upgrades, and for fault mitigation through Selective Readback and triple-module redundancy combined with partial scrubbing to recover Single Event Upsets (SEUs) induced by radiation [14].
- **High-performance and cloud computing**: Multi-tenant FPGA virtualisation, in which distinct users share a single device by occupying different reconfigurable partitions, has been demonstrated in data-centre contexts. The Coyote v2 framework [15] provides hierarchical reconfiguration of services and user logic with claimed order-of-magnitude improvements in reconfiguration time over systems that require the device to be taken offline.
- **Packet processing and network function virtualisation**: Dynamically replacing network processing modules at runtime enables a single-device implementation of a virtualised network function platform.
- **Embedded inference and edge computing**: PR facilitates adaptive compute at the edge by allowing a single low-cost FPGA to instantiate different inference accelerators depending on the current sensor context, reducing area relative to a monolithic multi-accelerator design.

Despite this breadth of demonstrated applications, a recurring observation in the survey literature is that most systems have been designed in an *ad hoc* manner, without benefit of a standardised high-level flow. The practical barriers — principally toolchain complexity, floorplanning effort, and the difficulty of verifying correctness across all RM combinations — have prevented DPR from achieving the adoption that its theoretical advantages might suggest. The three toolflow papers reviewed in §§2.3–2.5 each represent a distinct attempt to address these barriers.

2.2 The Compile-Time Problem and the Motivation for Parallel Compilation

A fundamental impediment to the practical deployment of FPGA-based accelerators — both in static and partially reconfigurable forms — is the duration of the implementation toolflow. Unlike software compilation, which produces executable binaries within seconds to minutes even for large codebases, FPGA implementation encompasses logic synthesis, technology mapping, placement, and routing: collectively a combinatorial optimisation problem that scales super-linearly with design complexity. For large designs targeting 7-series or UltraScale devices, end-to-end compilation routinely requires multiple hours, and in complex DPR configurations — where each Reconfigurable Module must be independently placed and routed within the RP's floorplan — the aggregate compile time for a full library of modules can extend further still.

2.2.1 The Monolithic Compilation Problem

Vendor toolflows — Xilinx Vivado and Intel Quartus Prime — operate in a fundamentally monolithic mode: the entire netlist is treated as a single optimisation problem, with global placement and routing algorithms attempting to minimise wire length and critical-path delay simultaneously across the entire device. While this all-at-once view produces high-quality results in terms of timing closure, it makes no use of the parallel computation resources available on a modern multi-core or multi-machine build environment. A Vivado implementation run that would benefit from twelve simultaneous routing threads typically occupies only a fraction of available cores during its most time-critical phases.

The compile-time problem is exacerbated in DPR contexts. The standard Vivado PR flow requires that the static partition be fully implemented first, and then each RM must be separately implemented with the static partition's placement and routing locked. If there are N RMs for a single RP, the tool must run N separate implementation runs in sequence (or with manual parallelism), each of which reindexes the already-placed static logic to confirm that the RM's interface signals align correctly. Any change to the static partition invalidates all RM implementations and requires a complete rebuild — a particularly costly event during iterative design.

2.2.2 RapidWright: The Infrastructure Layer

A key enabling technology for research into alternative compilation strategies is **RapidWright** [16], an open-source Java-based framework developed by Xilinx (now AMD) Research. RapidWright provides programmatic access to Vivado's internal design checkpoint (.dcp) format, allowing third-party tools to read, manipulate, and write the placed-and-routed netlist without invoking Vivado's GUI or batch flows. Specifically, RapidWright exposes:

- A **device model** that enumerates all physical resources (tiles, sites, BELs, and routing nodes) for a target device family;
- A **design model** that maps logical net endpoints to physical BEL placements and switchbox routing paths;
- A **bitstream API** that allows the configuration frames associated with a placed design to be read and written directly.

This infrastructure makes it possible to implement *stitching*: combining separately compiled sub-designs into a single routed netlist by resolving their inter-partition nets at the physical routing

level, without re-running global placement. The RWRoute [17] programmable routing engine within RapidWright further enables inter-partition route completion within the assembled design context, respecting the already-committed intra-partition routes. These capabilities are exploited both by RapidStream (§2.3) and, in principle, by the HPR toolflow's floorplan-aware compilation pipeline (§2.4).

2.2.3 Latency-Insensitive Design and Pipeline Islands

An important conceptual prerequisite for parallel compilation is *latency insensitivity* (LI). A design is said to be latency insensitive if the correctness of its computation is preserved regardless of the number of pipeline register stages inserted on any inter-module data path, provided that a suitable handshake protocol (commonly a credit-based or ready/valid token system) is employed to maintain flow control across each stage. The LI property, originally formalised in the context of synchronous circuit design methodology [18], is directly applicable to the parallelisation of FPGA compilation because it permits partitioning boundaries to be placed anywhere in the design's dataflow graph without affecting functional correctness: additional pipeline registers can be inserted at each cross-partition boundary to absorb routing latency, and the handshake protocol ensures that data tokens are neither lost nor duplicated.

In the RapidStream framework (§2.3), this property is exploited by floorplanning the design into a grid of rectangular *islands*, synthesising and placing each island independently in parallel, and then resolving the inter-island routes using RapidWright's routing engine — inserting relay pipeline stages where necessary to satisfy timing constraints across boundaries. The HPR toolflow (§2.4) applies a related principle but in the context of runtime reconfiguration rather than compile-time speedup: by constructing a Hoplite Butterfly (Hoplite-BF) Network-on-Chip that itself provides the timing-insensitive inter-node communication fabric, the HPR framework decouples the compilation of individual operators from the compilation of the static NoC infrastructure.

2.2.4 The Case for Incremental and Parallel Compilation

Beyond latency insensitivity, the case for incremental compilation is grounded in the observation that, in the typical DPR development workflow, only a small fraction of the design changes between successive iterations. The static partition — comprising the processor subsystem, memory controllers, interconnect, and reconfiguration infrastructure — is compiled once and then frozen. Only the evolving RMs need to be re-synthesised and re-placed. This structural separation maps naturally to an incremental compilation model: if the static partition's place-and-route solution can be preserved across RM iterations, and if individual RM compilations can be executed concurrently on a build cluster, the effective end-to-end compile time for a library of N RMs degrades only linearly in the number of *changed* RMs, rather than requiring a full re-run of the complete design.

The three toolflows surveyed in this chapter — RapidStream, HPR, and ZyPR — each address this compile-time challenge from a different angle:

This distinction is significant: RapidStream's contribution is exclusively on the compilation side and does not directly address runtime management, while HPR and ZyPR are concerned with both the compilation workflow and the runtime infrastructure for performing partial reconfiguration on a live system. The following three sections review each toolflow in detail.

Toolflow	Primary Focus	Parallelism Model	Runtime Reconfiguration
RapidStream [ASPLOS 2022]	Compile-time reduction	Island-based parallel PnR via RapidWright	No (static deployment)
HPR [FCCM 2022]	Adaptive PR for dataflow	Per-operator HLS + parallel PnR per PR page	Yes (runtime NoC reconfiguration)
ZyPR [IEEE ESL 2023]	End-to-end build & runtime manager	Incremental per-RP module builds	Yes (ZyCAP DMA-based PR)

2.3 RapidStream: Parallel Dataflow FPGA Compilation

2.3.1 Motivation and Context

The RapidStream framework [19] was presented at ASPLOS 2022 and addressed a well-established pain point in FPGA-based accelerator development: the hours-long compile time imposed by monolithic vendor tools. Its authors observed that high-level synthesis (HLS) dataflow designs — in which a computation is expressed as a graph of pipelined stages communicating through FIFO channels — exhibit a structural property that vendor tools are unable to exploit: the individual stages are *latency insensitive* with respect to each other, and therefore the placement and routing of each stage can in principle be resolved independently, without knowledge of the placement decisions made for its neighbours.

The key insight is that in a dataflow graph operating under a valid-ready (or token-credit) handshake protocol, inserting additional pipeline register stages on any inter-stage channel does not alter the functional correctness of the computation — it merely increases the end-to-end pipeline latency by the number of added stages. If each inter-stage channel is implemented as a relay register (sometimes termed a *pipeline relay*), the routing problem between any two stages is reduced to finding a path whose latency matches the relay count, rather than requiring those paths to meet a tight wire-length constraint imposed by a global placement solver working across the entire device simultaneously.

RapidStream exploits this property by decomposing the compilation problem into three decoupled phases: floorplan partitioning, parallel island compilation, and inter-island route resolution. Each phase is significantly faster than the equivalent monolithic Vivado flow for designs of comparable complexity.

2.3.2 RapidStream 1.0: Architecture and Pipeline

Floorplan partitioning. Given an HLS dataflow design specified at the C/C++ level (targeting the Vitis HLS toolchain), RapidStream applies a graph-partitioning algorithm to assign each computational module to one of a set of rectangular floorplan *islands* laid out on the device. The island boundaries are aligned to the device's clock region grid — the natural granularity at which routing resources become physically distinct — so that wires crossing an island boundary exercise only the limited set of long-line resources available at that boundary. This alignment guarantees that inter-island routes can be resolved by a targeted routing pass, rather than by a global router that must simultaneously consider all routes across the device.

The number and arrangement of islands is determined by a configurable grid parameter (for instance, a 2×3 or 4×4 arrangement), and modules are assigned to islands such that the total communication volume crossing any island boundary is minimised — equivalent to a balanced graph partitioning

objective with boundary-cut as the cost function. The partitioning algorithm itself, based on a multi-level min-cut heuristic, runs in seconds and produces a stable floorplan that is then fixed for the remainder of the compilation.

Parallel island compilation. With the floorplan determined, each island is compiled independently and in parallel. For each island, Vivado synthesises and implements only the logic assigned to that island, with the island's boundary treated as a set of I/O ports. Because island boundaries are aligned to clock region edges, the inter-island nets do not participate in global placement and routing at this stage: they are simply exported as port connections. Each island compilation runs as a fully independent Vivado process, with no shared global state, making the entire set of island compilations embarrassingly parallel. On a build server with sufficient cores, all islands complete roughly simultaneously, and the aggregate wall-clock time is bounded by the slowest island rather than the sum of all islands.

Inter-island stitching. Once all islands have been individually placed and routed, RapidWright's stitching and routing infrastructure is invoked to connect the inter-island nets. Relay pipeline registers are inserted at the boundary of each island — their count is determined during floorplanning based on the worst-case routing distance between boundary ports — and the inter-island wires are then routed using RWRoute [17], operating only on the long-line routing resources that cross clock region boundaries. Because the intra-island routes are already fixed, the inter-island router operates on a much smaller problem and completes quickly.

The final assembled design is exported as a Vivado design checkpoint (.dcp) that is indistinguishable from one produced by a conventional monolithic implementation run, and can be taken directly to bitstream generation without any additional tool steps.

Reported results. The RapidStream 1.0 paper reported compile-time reductions of approximately one order of magnitude compared to monolithic Vitis compilation for a range of HLS dataflow benchmarks on UltraScale+ devices, with quality-of-results (in terms of clock frequency and resource utilisation) within a few percent of the monolithic baseline. For the largest benchmarks evaluated — including SpMM sparse matrix kernels and SGEMM dense matrix multiplications targeting the Alveo U250 and U280 accelerator cards — monolithic compile times of six to twelve hours were reduced to under one hour.

2.3.3 RapidStream 2.0: HBM-Based Boards and SLR-Crossing Routing

The successor work, presented at ACM FPGA 2023 [20], extended the RapidStream framework to high-bandwidth memory (HBM) based boards and multi-SLR (Super Logic Region) UltraScale+ devices, addressing a new class of routing challenge that arises at SLR boundaries.

The SLR-crossing problem. In multi-SLR UltraScale+ devices (such as the Alveo U250, which contains four SLRs), wires crossing an SLR boundary must use a dedicated set of *super long-line* (SLL) routing resources implemented through metal stacks that span the interposer. SLL resources are scarce relative to the volume of cross-SLR communication in a densely mapped design, and their timing characteristics differ from intra-SLR routes, making them a bottleneck for both compile time and timing closure. The monolithic Vivado router must simultaneously compete for SLL resources across the entire design, leading to congestion and long routing runtimes.

RapidStream 2.0 introduces a dedicated SLR-boundary router that handles cross-SLR wires as a separate post-processing phase. In this approach, the isles are partitioned at the SLR granularity, each SLR is compiled independently (further subdivided into the existing clock-region-level islands), and the resulting per-SLR checkpoints are then assembled by explicitly routing the SLL wires using

a pre-computed assignment that matches each source-sink pair to a specific SLL channel. This SLL assignment is computed as a min-cost flow problem over the bipartite graph of available SLL channels and required connections, and is solved offline in seconds.

HBM pseudo-channel assignment. A second contribution specific to HBM boards is an HBM pseudo-channel assignment algorithm that co-optimises the mapping of HBM accesses to physical pseudo-channels with the island floorplan, so that spatial locality between computation islands and their associated HBM channels is maximised. This spatial co-optimisation reduces the average length of HBM-to-logic wires, improving both timing and the congestion profile of the resulting placement.

Reported results. RapidStream 2.0 reported compile-time reductions of 5–12× compared to monolithic Vitis for HBM-based benchmarks, with quality-of-results competitive with or exceeding the monolithic baseline on frequency for most benchmarks. The SLR-crossing router in particular was shown to add less than ten minutes of compile time even for designs with thousands of SLR-crossing connections.

2.3.4 Relevance and Limitations with Respect to the Present Work

RapidStream addresses compile-time rather than runtime reconfiguration. Its compilation model — fixed islands, fixed routing, static bitstream — produces a conventional monolithic bitstream that is loaded once at device startup and cannot be partially updated at runtime without regenerating the entire bitstream. The framework therefore does not provide any runtime DPR capability and cannot be used as a basis for the CFU swap scenario described in this project.

Nevertheless, the architectural principles identified in RapidStream — particularly the latency-insensitive partitioning of a dataflow graph into independently compiled islands, and the use of RapidWright as a bitstream-level assembly tool — are directly relevant to the HPR framework reviewed in §2.4, which applies the same principles to a DPR context. In the context of the present work, RapidStream serves as a benchmark for compile-time improvement methodology, establishing the theoretical ceiling for how far island-based parallelism can reduce FPGA build times before being applied to the partial reconfiguration domain.

Implementation Note: Hands-on evaluation of RapidStream was not carried out as part of this project. The RapidStream team proceeded to productise the framework following its academic publication, and in doing so revoked all public API keys and access repositories. The tool is no longer available for independent evaluation or reproduction. Coverage in this report is accordingly limited to a literature-based review of the published papers.

2.4 Hierarchical Partial Reconfiguration (HPR)

2.4.1 Motivation: Fixed Pages and Their Limitations

The HPR framework [21], presented at FCCM 2022, addresses the compile-time and flexibility limitations of existing partial reconfiguration-based dataflow acceleration approaches. The foundational observation motivating HPR is that prior DPR systems for dataflow applications — including the approach characterised as *PR with NoC* in the HPR paper, and also implicit in the RapidStream island model — employ *fixed-size* PR pages: rectangular regions of fixed dimensions whose floorplan is determined at system design time and cannot be changed without rebuilding the static partition.

Fixed-size pages create a fundamental tension. If pages are sized to accommodate large operators, the area assigned to small operators is left unutilised, wasting fabric resources that could otherwise be assigned to concurrently executing operators. If pages are sized optimally for small operators, large operators must be manually decomposed into sub-operators that each fit within a single page — a labour-intensive process that frequently degrades performance because it introduces artificial boundaries in the operator's internal routing. Neither outcome is satisfactory in a system where operator sizes vary significantly and evolve during iterative development.

A related limitation concerns compile-time: in a fixed-page PR system, any change to an operator — even a minor HDL edit — necessitates re-synthesising, re-placing, and re-routing the modified operator's new Reconfigurable Module within its designated page. If the operator fits into a different-sized page after the change (for example, because an optimisation reduced its resource footprint), the existing static floorplan cannot accommodate the relocation without a full rebuild of the static shell. This dependency between operator size and static floorplan is the root cause of the long round-trip compilation times observed in fixed-page DPR systems.

2.4.2 Core Concept: Recombinable PR Pages

The central innovation in HPR is the introduction of *hierarchically recombinable PR pages* — rectangular regions that can be logically combined at runtime to form larger compound pages. Three canonical page sizes are defined:

- **Single page (S):** the smallest atomic page, occupying one unit of the floorplan grid;
- **Double page (D):** two horizontally adjacent single pages, treated as a single logical region;
- **Quad page (Q):** four single pages arranged in a 2×2 grid, treated as a single logical region.

The key property of this hierarchy is that the physical layout of the pages, including the routing resources that cross their boundaries, is fixed in the static partition — but an operator can be assigned to any page size at or above its minimum fitting size. A small operator that fits in a Single page is placed there. A medium operator that requires a Double page occupies two adjacent Single pages, and the inter-page routing resources within that Double are part of the static implementation. A large operator that requires a Quad page occupies four Single pages arranged in a 2×2 group.

Critically, the assignment of operators to page sizes is determined automatically at compile time by the greedy fitting algorithm (§2.4.4), not by the designer. The designer does not need to manually decompose operators or pre-assign them to fixed-size regions. This eliminates the key source of developer friction in fixed-page PR systems.

2.4.3 The Hoplite-BF Network-on-Chip

The interconnection between operators in the HPR system is provided by a *Hoplite-BF* (Butterfly) variant of the Hoplite on-chip network [22]. The Hoplite family of NoCs is designed specifically for FPGA deployment and achieves high routing efficiency by mapping logical flits directly to FPGA routing resources rather than requiring dedicated tile-based routers. The Hoplite-BF variant extends the standard Hoplite bidirectional bus-based topology to support the hierarchical page structure of HPR.

Within the HPR context, the Hoplite-BF NoC serves three distinct roles:

1. **Inter-operator routing**: Data tokens flowing from one operator's output to another's input traverse the NoC as flits addressed by virtual channel and destination tag. The NoC provides

flow control through a credit-based scheme, maintaining backpressure when a downstream operator's input FIFO approaches capacity.

2. **Reconfiguration isolation**: When a page is being reconfigured — that is, when a new operator's partial bitstream is being loaded into that page — the NoC's flow control mechanism automatically stalls producers of data destined for that page, preventing corrupted data from entering the partially reconfigured logic. This property is analogous to the function of the DFX Decoupler in the system described in Chapter 4, but implemented at the NoC protocol level rather than through explicit isolation logic.
3. **Runtime operator connection**: When a new operator has been loaded into a page, the NoC routing tables are updated to reflect the new operator's logical address. This step completes the operator swap and allows the pipeline to resume — making operator replacement a purely software-configured event at the NoC control plane, invisible to other operators in the dataflow graph that are not directly connected to the replaced operator.

2.4.4 Compilation Workflow

The HPR compilation workflow is structured as follows, and is designed to maximise parallelism across all phases:

Phase 1 — HLS synthesis (per operator, fully parallel). Each operator in the dataflow graph is described in C/C++ and synthesised using Vitis HLS to produce a synthesised netlist. Crucially, each HLS run is independent: operators have no shared state and do not depend on each other's synthesis outputs. All operators are therefore synthesised simultaneously, and the wall-clock time for Phase 1 is bounded by the slowest individual operator synthesis, rather than the sum of all operators.

Each HLS synthesis also produces a *resource utilisation report* enumerating the number of LUTs, flip-flops, BRAMs, and DSP slices required by the operator at the target clock frequency. This report is consumed by the greedy page assignment algorithm in Phase 2.

Phase 2 — Greedy page assignment. Following synthesis, the tool parses each operator's resource utilisation report and compares its reported requirements against the capacity of each page size (S, D, Q). The greedy algorithm assigns each operator to the smallest page size that can accommodate its resource footprint, iterating over operators in decreasing order of resource demand. If no page of sufficient size is available — because all pages of that size are already allocated — the operator is assigned to the next larger available size. If the operator's requirements exceed the capacity of a Quad page, an error is raised and the designer must either optimise the operator or partition it manually.

The output of Phase 2 is a complete operator-to-page assignment that is guaranteed to be satisfiable given the static floorplan. This assignment is deterministic and reproducible, making it suitable for incremental recompilation: if only one operator changes between iterations, only that operator is re-synthesised and reassigned; all other assignments remain valid.

Phase 3 — Page-level PnR (per assigned page, fully parallel). With the operator-to-page assignment determined, each operator is implemented within its assigned page. Implementation is performed by a Vivado run that uses the page's pre-defined Pblock constraint, which is part of the static shell. As with Phase 1, all page implementations are independent and are therefore executed in parallel. The wall-clock time for Phase 3 is bounded by the slowest page implementation.

For a Double or Quad page, the implementation run treats the compound page as a single Pblock and applies standard Vivado placement and routing within the enlarged region. The intra-compound

routing resources that cross the Sub-page boundaries are available to the router as ordinary LUT-to-LUT routes, without requiring special treatment, because those resources are not used by the static shell at the internal page boundaries.

Phase 4 — Bitstream generation and NoC configuration. Once all page implementations are complete, partial bitstreams are generated for each operator independently. Each partial bitstream targets only the frames within the assigned page's Pblock, and is stored as a binary file indexed by operator identifier and page address. The NoC routing configuration — specifying the virtual channel assignments from producer operators to consumer operators — is computed from the dataflow graph's edge list and written to an initialisation file for the NoC control software.

2.4.5 Runtime Reconfiguration with the Hoplite-BF NoC

At runtime, operator replacement proceeds as follows:

1. The runtime manager (executing on a host processor or embedded soft-core) identifies the operator to be replaced and issues a *deactivation* command to the NoC control plane for the target page's input virtual channels. The NoC stalls all data destined for the page until the channel credit count reaches zero, ensuring no in-flight data will be corrupted by the reconfiguration.
2. The partial bitstream for the new operator is DMA-transferred to the FPGA's configuration memory through the available configuration port (PCAP on Zynq-based platforms, or an ICAP controller on pure FPGA platforms). The target page's existing operator is overwritten.
3. The runtime manager issues an *activation* command, updating the NoC routing tables to reflect the new operator's dataflow connections. The NoC releases the stalled credits and data flow resumes with the new operator in place.

The latency of this operator-swap sequence — from issuing the deactivation command to the resumption of data flow — comprises: (a) drain time for in-flight data (bounded by the FIFO depths multiplied by the pipeline latency of the data path from last producer to the target page), (b) bitstream transfer time (dominated by the partial bitstream size and the ICAP/PCAP throughput), and (c) NoC routing table update time (negligible, typically a few clock cycles). In the benchmarks reported in the HPR paper, total swap latency for a Single page was on the order of tens of milliseconds when using a software-initiated PCAP transfer on a Zynq device.

2.4.6 Comparison with Fixed-Page PR and RapidStream

The HPR paper includes explicit comparisons with two baseline approaches: a conventional fixed-size page PR system (labelled *PR with NoC* in the paper's diagrams), and the RapidStream island-based compilation model. The three-way comparison is summarised as follows:

The key differentiator of HPR relative to *PR with NoC* is area efficiency: because each operator occupies only the smallest page that fits it, a larger number of operators can coexist simultaneously on the device compared to a system in which every operator is allocated a fixed-size page sized for the largest operator in the library. In a library of operators with highly varied resource footprints — which is typical of machine learning accelerators, where tasks range from simple data formatters requiring tens of LUTs to convolutional cores requiring thousands of LUTs and many BRAM tiles — this adaptive sizing can substantially increase system throughput.

Relative to RapidStream, HPR offers runtime adaptability at the cost of a slightly more rigid floorplan structure (the page grid must be defined at static shell compile time) and an increased static shell

Criterion	PR with NoC (Fixed)	RapidStream	HPR
Page / island size	Fixed at design time	Fixed grid, configurable	Adaptive: S / D / Q
Operator decomposition	Manual, by designer	Automatic partitioning	Automatic greedy fitting
Runtime reconfiguration	Yes (per fixed page)	No (static bitstream)	Yes (per adaptive page)
Compile parallelism	Per-RM, sequential or manual	Per-island, fully parallel	Per-operator + per-page, fully parallel
NoC role	Routing only	N/A (fixed wire routes)	Routing + reconfiguration control
Area efficiency	Low for small operators	N/A	High (smallest fitting page)

area overhead (the Hoplite-BF NoC and page boundary logic consume a non-trivial fraction of device resources). RapidStream, by contrast, imposes no static overhead and achieves higher raw clock frequencies by allowing the router complete freedom over inter-island wire placement. The two frameworks are therefore complementary rather than competing: RapidStream targets high-throughput static deployments where compile time is the bottleneck, while HPR targets adaptive deployments where operator diversity and runtime replaceability are the primary concerns.

2.4.7 The PLD Predecessor: Incremental Refinement Compilation

A predecessor to the full HPR framework — published at the same research group under the title *Fast FPGA Compilation to Make Reconfigurable Acceleration Compatible with Modern Incremental Refinement Software Development* (PLD, 2022) [23] — addressed an earlier facet of the compile-time problem: the mismatch between software development workflows, which support rapid iterative refinement with sub-second edit-compile-run cycles, and FPGA development workflows, which impose hour-long compilation barriers even for minor changes.

The PLD work demonstrated that for designs structured around modular HLS accelerators with fixed interfaces, it is possible to maintain a *compiled shell* of the static infrastructure (memories, interconnect, processor interfaces) and iteratively recompile only the changed accelerator modules, assembling the result using RapidWright stitching. This incremental model reduced per-iteration compile times from hours to minutes for the benchmark set evaluated.

The HPR framework can be understood as extending the PLD incremental model to the DPR domain: the static shell becomes not just a compilation convenience but a runtime-deployable base bitstream, and the individual module compilations produce not just assembly artefacts but deployable partial bitstreams. The conceptual lineage from PLD to HPR to the present work's static-plus-PR architecture is therefore direct, with each step adding a layer of runtime adaptability to a foundation of incremental parallel compilation.

2.4.8 Relevance to the Present Work and the Decision to Limit Coverage

The architectural principles demonstrated in HPR — specifically the separation of a fixed static shell (containing NoC and reconfiguration infrastructure) from independently compiled Reconfigurable Modules, and the use of a page-granularity DFX Decoupler-equivalent isolation mechanism during

bitstream loading — are directly reflected in the system architecture described in Chapter 4. The DFX Decoupler placed in `cfu.v` serves precisely the isolation function that the Hoplite-BF NoC's credit-drain mechanism serves in HPR: preventing corrupted data from propagating to the static side of the interface during the partial reconfiguration window.

However, the reproduction of the HPR toolflow was attempted but could not be completed during this project. The primary obstacles were:

- **Vivado version sensitivity**: The HPR toolflow is documented against a specific Vivado release, and the use of even a minor version increment introduces structural changes in the DCP format that cause the RapidWright-based stitching step to fail silently, producing invalid bitstreams.
- **Linux kernel and driver compatibility**: The toolflow's dependency on a specific version of the Linux XRT (Xilinx Runtime) driver stack introduced conflicts with the available build system's kernel version, preventing the Vitis HLS compilation stage from completing.
- **Compute resource constraints**: The place-and-route steps for Quad pages on the target device pushed the available build machine to its memory limits, causing router jobs to be terminated by the OS or to produce degenerate routing results after exhausting available RAM.

For these reasons, HPR is treated in this report as a reference for architectural principles rather than a reproduced implementation. Its concepts directly informed the design of the CFU controller architecture (Chapter 4), particularly with respect to decoupling strategy and the structural separation of static and reconfigurable partitions.

2.5 ZyPR: End-to-End Build Tool and Runtime Manager for PR at the Edge

2.5.1 Motivation and Positioning

ZyPR [24], published in ACM Transactions on Reconfigurable Technology and Systems (TRETS) in 2023, is authored by Alex R. Bucknall and Suhaib A. Fahmy — the latter being the same researcher responsible for the foundational DPR survey [2] and the ZyCAP controller on which ZyPR's runtime infrastructure is built. In contrast to both RapidStream (which targets cloud-scale UltraScale+ accelerators) and HPR (which targets dataflow graphs of HLS operators), ZyPR is designed specifically for edge FPGA SoC deployments: resource-constrained platforms such as the Zynq-7000 family, where the available logic, memory, and I/O budgets are significantly smaller than datacenter boards, and where power consumption and deployment simplicity are primary concerns.

The central problem ZyPR addresses is the fragmented nature of existing edge DPR workflows. A designer wishing to implement partial reconfiguration on a Zynq board must independently manage: (a) the Vivado implementation flow and floorplanning, (b) the PetaLinux device-tree overlay (DTBO) driver stack that makes the FPGA Manager aware of reconfigurable partitions, (c) the AXI-connected soft IP that controls the ICAP or PCAP configuration port, and (d) the runtime software that triggers bitstream loads and monitors completion. Each of these layers requires specialist knowledge and significant manual integration effort. ZyPR's contribution is to automate and integrate all four layers into a single end-to-end flow, exposing a simple specification interface to the designer and producing — without manual intervention — a deployable hardware bitstream, a Linux device-tree overlay, and a runtime management library.

2.5.2 ZyCAP: The High-Throughput PCAP/ICAP Controller

At the heart of ZyPR's runtime infrastructure is the **ZyCAP** IP core [25], a partial reconfiguration controller developed specifically for Zynq and Zynq UltraScale+ SoC platforms. It is the most directly comparable existing work to the ICAP-based reconfiguration controller described in this project, and is therefore reviewed in some depth.

Architecture. ZyCAP is implemented in the Zynq PL and exposes two AXI interfaces to the PS:

- **GP Port (AXI4-Lite, via the General Purpose AXI master port):** The PS writes configuration transactions to ZyCAP through this lightweight control channel. Each transaction specifies the base address in DRAM of the target partial bitstream and its size in bytes. The ZyCAP controller validates the transaction, checks its internal bitstream cache for a previously transferred version of the requested bitstream, and issues a DMA command if the bitstream is not cached.
- **HP Port (AXI4 Full, via the High-Performance AXI slave port):** The ZyCAP DMA engine uses this port to read the partial bitstream data from DRAM through the Zynq memory interconnect. The AXI4 burst transactions achieve the full available memory bandwidth, sustaining a bitstream transfer rate of up to **400 MB/s** to the ICAP primitive — matching the ICAP's maximum rated throughput on Zynq-7000 devices. This is in sharp contrast to the baseline Zynq Linux FPGA Manager, which transfers bitstreams through the PCAP (Processor Configuration Access Port) via the Configuration Security Unit (CSU), a path that occupies the ARM Trusted Firmware and Platform Management Unit and blocks the PS during the entire transfer.

Non-blocking operation. A critical property of ZyCAP is that the DMA transfer to the ICAP is executed entirely in hardware without processor involvement. Once the PS has written the bitstream address and size to the GP port registers, the PS is free to execute other tasks. The ZyCAP DMA engine fetches the bitstream from DRAM independently, writes it to the ICAP primitive, and asserts an interrupt on the GP port interrupt line when the transfer is complete. This non-blocking model allows the application software to overlap useful computation with the reconfiguration latency — an important property for real-time edge deployments where reconfiguration frequency may be high.

DRAM-based bitstream caching. ZyCAP implements a configurable Least Recently Used (LRU) bitstream cache in DRAM. When a partial bitstream is requested for the first time, it is fetched from persistent storage (typically SD card or eMMC) into a reserved DRAM region, and the cache entry is recorded. On subsequent requests for the same bitstream, ZyCAP reads directly from the DRAM cache rather than re-reading from persistent storage, reducing reconfiguration latency by the time that would otherwise be spent on the slower storage access. The LRU policy ensures that the most recently and frequently used bitstreams remain in the cache while infrequently used ones are evicted when the cache fills.

Comparison with the Zynq FPGA Manager. The standard Linux FPGA Manager interface on Zynq uses the PCAP port, which requires the PS ARM cores to coordinate with the ARM Trusted Firmware (ATF) and the Platform Management Unit (PMU) via a sequence of register reads and writes to the Configuration Security Unit (CSU) registers. This coordination: (a) occupies the PS during the entire reconfiguration window, blocking application execution; (b) achieves substantially lower effective bitstream transfer rates than the ZyCAP DMA path (approximately 10–100 MB/s in practice, compared to ZyCAP's rated 400 MB/s); and (c) requires a clean teardown and re-initialisation of the Linux device driver for each RM load, adding latency overhead. ZyCAP eliminates all three limitations through its DMA-based architecture.

2.5.3 ZyPR Toolflow: Optimisation Levels and Build Pipeline

ZyPR exposes a tiered optimisation model with three main levels, balancing compile time and quality-of-results:

O0 — Soft-core execution: At this optimisation level, operators are not mapped to hardware accelerators at all. The dataflow graph is executed entirely on the on-chip soft-core processor (or ARM PS). No DPR infrastructure is synthesised. This level is used during early-stage functional verification, where the overhead of hardware synthesis and implementation would impede rapid iteration.

O1 — Fixed PR pages with Hoplite-BF interconnect: At O1, each operator in the dataflow graph is mapped to a fixed-size Reconfigurable Partition page, interconnected by the Hoplite-BF NoC (the same interconnection fabric used in HPR and described in §2.4.3). Each page is independently synthesised and implemented, enabling parallel compilation. Operators can be swapped at runtime via ZyCAP without halting operators in other pages — the NoC credit-drain protocol handles safe isolation before each bitstream load. O1 provides a good balance between compile time and runtime reconfigurability, and is the primary target for iterative edge development.

O3 — Monolithic Vitis compilation: At O3, the full design is compiled as a single monolithic Vivado implementation run, treating all operators as statically instantiated components rather than reconfigurable modules. No partial bitstreams are generated. This level produces the highest achievable clock frequency and resource efficiency — equivalent to a conventional non-DPR FPGA design — at the cost of full compile time and loss of runtime reconfigurability. O3 is intended as the final production compilation step once the design has been validated at O1.

Build pipeline. The ZyPR build pipeline automates the following sequence, driven by a single user-provided `spec.json` configuration file that lists operator sources, target device, clock constraint, and optimisation level:

1. **Verilog parsing (Pyverilog):** The operator's top-level Verilog or SystemVerilog interface is parsed using Pyverilog [26] to extract port declarations. This step identifies the required handshake signals, data widths, and clock/reset connections that must be preserved when the operator is placed into a PR slot.

2. **EDA backend invocation (Edalize):** The open-source Edalize Python library [27] drives Vivado in batch mode, invoking synthesis and implementation runs for each operator. Edalize generates the tool-specific project files and invocation scripts, abstracting the designer from Vivado's internal TCL project structure.

3. **TCL injection for RP/RM definition:** ZyPR injects custom TCL scripts into the Vivado implementation run to define the Reconfigurable Partition bounding box (the Pblock) and associate the operator's synthesised netlist with the RM role. This step automates what a designer would otherwise perform manually in the Vivado GUI — the most error-prone and time-consuming manual step in a conventional DPR workflow.

4. **PetaLinux DTBO and driver generation:** For the runtime deployment on a Zynq running Linux, ZyPR generates the device-tree overlay (`.dtbo`) file that describes the PR regions to the Linux FPGA Manager, and a set of kernel module driver files that expose each PR slot as a character device accessible through the standard `/dev/fpga*` interface. The designer does not need to write or modify any device-tree source.

5. **Runtime manager integration:** ZyPR generates a Python runtime library that provides a high-level API for operator swap operations. A call such as `manager.load_rm("conv_1x1",`

`slot=0)` triggers the full ZyCAP-managed reconfiguration sequence, abstracts the underlying DTBO driver calls, and blocks (or returns a future handle, in non-blocking mode) until confirmation of successful reconfiguration.

Chained region support. A distinguishing feature of ZyPR relative to simpler single-RP flows is its support for *chained PR regions*: multiple reconfigurable partitions whose interface signals form a directed pipeline. In a chained configuration, operator A in slot 0 feeds its output directly into operator B in slot 1, without routing the signal through the NoC. ZyPR's TCL injection handles the interface constraints at the inter-slot boundary, ensuring that the routing of the chain connection is preserved irrespective of which operator is currently loaded in each slot.

2.5.4 Performance Benchmarks and Comparison

The ZyPR paper reports experimental results on a Zynq-7000 SoC running Linux. Key findings include:

- **ZyCAP reconfiguration throughput**: Measured at approximately 350–390 MB/s sustained throughput for typical partial bitstream sizes (4–48 MB), approaching the rated 400 MB/s ceiling. The Linux FPGA Manager baseline achieved 8–12 MB/s on the same device, confirming the order-of-magnitude throughput advantage of the DMA-based ZyCAP path.
- **Reconfiguration time**: For a representative partial bitstream of 12 MB (typical of a medium-complexity convolution accelerator on a Zynq-7020), ZyCAP's measured reconfiguration time was approximately 31 ms, compared to approximately 280 ms for the Linux FPGA Manager — a 9× improvement.
- **Non-blocking benefit**: By overlapping DRAM-to-ICAP DMA with PS-side computation, the effective reconfiguration overhead (as visible to the application) was reduced further, approaching the memory bandwidth limit rather than the compute limit.

2.5.5 Relevance to the Present Work and Implementation Note

ZyCAP is the most architecturally proximate existing work to the ICAP controller infrastructure described in Chapter 4. The design principles — DMA-based bitstream transfer, interrupt-signalled completion, DRAM-based bitstream staging, and decoupler-based isolation — map directly to the intended autonomous reconfiguration flow for the Arty A7-35T described in §4.7. Specifically:

- The ZyCAP GP-port control interface corresponds to the intended role of the `system_wrapper` module in the present work: receiving a reconfiguration trigger (in ZyCAP from the PS, in the present work potentially from the VexRiscv soft-core) and orchestrating the ICAP write sequence.
- The ZyCAP HP-port DMA corresponds to the intended DDR-to-ICAP path described as the "next step" in §4.7: reading partial bitstreams from DDR memory and streaming them to the ICAP primitive.
- The ZyCAP LRU DRAM cache corresponds conceptually to the DDR partial bitstream storage pool that is the target of the second stage roadmap item in §4.7.

The key distinction between ZyCAP's deployment context and the present system is platform: ZyCAP is designed for Zynq SoCs (where the ARM PS, PCAP, DDR controller, and ICAP all exist on a single chip and are connected by high-bandwidth AXI switched interconnect), whereas the Arty A7-35T is a pure FPGA platform with no embedded ARM core and with ICAP as the only avail-

able configuration port. The Zynq PS and PCap are replaced by the VexRiscv soft-core and the `system_wrapper` ICAP controller — a fundamentally equivalent architectural role implemented in PL logic rather than in hard IP.

Implementation Note: Hands-on use of ZyPR was not achieved during this project. The ZyPR toolflow is dependent on a single proprietary tool for which an educational licence was required; the application for that licence was unsuccessful. A manual workaround — running the underlying Pyverilog and Edalize scripts by hand and treating their intermediate outputs as correct — was explored, but the absence of the full ZyPR integration prevented reliable generation of the DTBO and driver artefacts, rendering the resulting outputs unverifiable. The approach was therefore discontinued. ZyPR and ZyCAP are covered in this report through literature analysis of the published paper [24][25].

2.6 Other Related Works

The five works reviewed in this section are each independently relevant to the system presented in this report. Unlike the toolflow papers of §§2.3–2.5, which collectively address the *how* of building DPR systems, these works address the *why* and *what* — the application motivations and architectural precedents that directly informed the design choices documented in Chapters 3 and 4.

2.6.1 DML: Dynamic Partial Reconfiguration with Scalable Task Scheduling for Multi-Application FPGAs

The DML framework [28] (published in IEEE Transactions on Computer-Aided Design, 2022) addresses the multi-application scheduling problem that arises when a single physical FPGA must serve as a shared resource for multiple concurrent software applications, each of which submits hardware acceleration requests and expects results within bounded latency.

DML models each application as a *task graph*: a directed acyclic graph (DAG) in which nodes represent hardware functions to be executed and edges represent data dependencies. Tasks are mapped to a pool of identically sized Reconfigurable Partitions, and the DML runtime scheduler — implemented in software — determines the order of RM loads based on a priority function that accounts for task deadline, data dependency resolution, and RP availability. When no RP is free for a high-priority task, the scheduler employs a *preemption* mechanism: it evicts the RM currently occupying the target RP (saving its output state if necessary), loads the higher-priority task's RM, and re-queues the preempted task.

The scalability claimed in DML's title refers to the property that the scheduling algorithm's computational complexity grows linearly with the number of tasks and RPs, rather than exponentially (as would be the case for an optimal schedule-search formulation). This linear scalability is achieved by decomposing the scheduling problem into independent per-RP queues managed by per-RP arbiters, with a global priority sorter mediating between arbiters when multiple queues compete for the same RM.

Relevance: DML describes the scheduling layer directly above the controller layer implemented in this project. The `system_wrapper` ICAP controller described in Chapter 4 corresponds to the RP-level bitstream loader that DML's arbiter would invoke. A future extension of the present work — specifically the CFU library management problem of selecting which CFU RM to load based on

the current instruction mix in the RISC-V pipeline — maps naturally to a single-RP instance of the DML scheduling framework.

2.6.2 AMPER-X: Adaptive Mixed-Precision RISC-V Core for Embedded Applications

AMPER-X [29] (IEEE Transactions on Circuits and Systems, 2024) is the most closely related prior work to the system described in this report in terms of its primary thesis. The paper demonstrates an adaptive mixed-precision RISC-V core in which the arithmetic unit — specifically, the multiply-accumulate engine — is designated as a Reconfigurable Partition. At runtime, the active precision format for arithmetic operations (8-bit integer, 16-bit floating-point bfloat16, or 32-bit single-precision float) is selected by loading one of three pre-compiled Reconfigurable Modules into the RP via partial reconfiguration.

This precision-switching architecture is motivated by embedded inference workloads in which different layers of a neural network benefit from different numerical precisions: early convolutional layers can often operate in 8-bit integer arithmetic without loss of accuracy, while later fully-connected layers may require 16- or 32-bit precision to preserve activation dynamic range. By associating each layer's inference pass with the appropriate RM load, AMPER-X achieves a balance between inference throughput (higher for lower precision) and accuracy (higher for higher precision) that cannot be achieved by a monolithic fixed-precision implementation.

The AMPER-X implementation uses a standard Xilinx DFX Decoupler at the boundary between the static RISC-V core and the precision-switchable arithmetic RP, and manages reconfiguration through PCAP-based bitstream loading on a Zynq-based test platform. The measured reconfiguration overhead for a precision switch was reported as approximately 45 ms for the smallest RM (8-bit integer mode), which the authors characterised as acceptable for the inter-layer granularity of neural network inference.

Relevance: AMPER-X provides direct architectural precedent for the CFU partitioning strategy described in Chapter 4. The AMPER-X system's separation of a fixed static RISC-V core from a dynamically swappable arithmetic RM is structurally equivalent to the present work's separation of the fixed VexRiscv SoC (in `digilent_arty`) from the dynamically swappable `cfu_compute` RM. The key distinction is that AMPER-X operates on a single operator (the arithmetic unit) with a small, fixed set of precision configurations, while the present work targets a more general CFU library with functionally distinct accelerator modules (byte manipulation, fixed-point multiply-accumulate, and custom DSP templates) rather than precision variants of the same function. Both systems use a DFX Decoupler for isolation, confirming that this is the canonical solution for RISC-V–RP interface isolation.

2.6.3 RV-CAP: Enabling Dynamic Partial Reconfiguration for FPGA-Based RISC-V SoC

RV-CAP [30] (published at IEEE FPL 2021) is a direct architectural predecessor to the present work. It demonstrated the integration of ICAP-based partial reconfiguration management into a VexRiscv soft-core SoC, enabling the CPU itself to initiate and monitor RM loads without host intervention. The contribution of RV-CAP is specifically the *control-plane integration*: defining a memory-mapped register interface through which the VexRiscv (or any RISC-V soft-core with access to the memory map) can write bitstream addresses to the ICAP controller, trigger the DMA transfer, and poll a status register for completion.

The RV-CAP system was implemented on a Xilinx Artix-7-based board (closely matching the Arty A7-35T used in this project) and validated with a set of synthetic RM benchmarks. Reconfiguration latency was measured at approximately 28 ms for a 4 MB partial bitstream, using an SRAM-backed bitstream buffer local to the PL fabric. The authors noted that the principal bottleneck in their system was the SRAM access bandwidth (limited to approximately 140 MB/s for the on-chip BRAM-based buffer used), rather than the ICAP's rated 400 MB/s — motivating the use of off-chip DDR memory with DMA as the path to maximum throughput.

Relevance: RV-CAP is the most direct hardware-architecture precedent for the `system_wrapper` ICAP controller described in Chapter 4. The register interface design, the decouple-trigger-monitor control flow, and the STARTUPE2 EOS monitoring strategy described in the present work draw on the design patterns established in RV-CAP. The present work extends RV-CAP's architecture by integrating it within the CFU Playground framework, adding the `recon_counter` EOS-based latency measurement, and targeting the CFU interface partition specifically — rather than the generalised RM slot used in RV-CAP.

2.6.4 DORA: A Low-Latency Partial Reconfiguration Controller

DORA [31] (IEEE FPT 2024) presents a hardware-only partial reconfiguration controller — implemented entirely in FPGA PL without any embedded processor — that achieves sub-millisecond reconfiguration latency for small partial bitstreams stored in on-chip BRAM. Its design motivation is latency-sensitive applications (such as real-time control systems and high-frequency trading accelerators) in which the multi-millisecond reconfiguration times typical of DMA-based controllers are unacceptable.

DORA achieves low latency by preloading partial bitstreams into on-chip BRAM at system startup and streaming them to the ICAP primitive through a purpose-built FSM controller that operates at the maximum ICAP clock rate (typically 100 MHz for 7-series devices). For partial bitstreams small enough to fit in on-chip BRAM (under approximately 4 MB for a small RP), DORA achieves reconfiguration times of 0.5–2 ms — roughly an order of magnitude faster than the SRAM-based approach in RV-CAP and two orders of magnitude faster than the Linux FPGA Manager baseline.

The primary limitation of DORA is capacity: on-chip BRAM is a scarce resource, and preloading a large library of partial bitstreams would quickly exhaust the available BRAM budget. DORA's authors acknowledge this and position the controller as appropriate for systems with a small RM library (fewer than four or five modules) and stringent latency requirements, as opposed to large-library systems that must tolerate higher reconfiguration latency in exchange for off-chip storage capacity.

Relevance: DORA establishes the hardware lower bound on reconfiguration latency for 7-series FPGAs — approximately 0.5 ms for small bitstreams from on-chip BRAM, versus 28–45 ms for DMA-based off-chip approaches. This bound informs the latency expectations for the `recon_counter` measurement described in Chapter 4. The present work does not implement a DORA-class controller (the primary reconfiguration path is JTAG-based or planned to be DDR-DMA-based), but the DORA results provide a reference point for evaluating the headroom available for autonomous full-throughput reconfiguration in future iterations.

2.6.5 ACNNE: Adaptive Convolution Engine for CNN Acceleration with DPR

ACNNE [32] (IEEE TCAS-II, 2024) targets a domain-specific application of DPR: accelerating convolutional neural network inference on FPGA by adaptively selecting the hardware configuration of the convolution engine based on the kernel size and feature map dimensions required by each layer. The system defines a family of convolution engine RMs — each optimised for a different kernel size (1×1, 3×3, 5×5) — and loads the appropriate RM at the beginning of each layer's inference pass.

The ACNNE architecture is implemented on a Zynq-7000 series board (ZedBoard, XC7Z020) and uses PCAP-based reconfiguration managed by a bare-metal ARM PS application. Resource utilisation measurements report that the DPR-based ACNNE design uses approximately 40% fewer LUTs and DSP slices compared to a monolithic implementation that instantiates all three convolution engine variants simultaneously — a direct demonstration of the hardware time-multiplexing benefit claimed in §1.1 of this report.

Reconfiguration overhead in ACNNE is reported at approximately 36 ms per RM swap, comparable to the ZyCAP baseline measurements in §2.5. The authors note that for convolutional networks with moderate layer counts (ResNet-style architectures with 16–50 convolutional layers), the reconfiguration overhead constitutes less than 5% of total inference time — confirming that DPR-based hardware multiplexing is practically viable for this application class.

Relevance: ACNNE provides empirical validation for the resource-reduction argument that motivates the use of CFU swapping in the present work. The 40% area reduction reported for a three-variant convolution engine library has a direct analogue in the CFU domain: a CFU library containing byte-manipulation, multiply-accumulate, and DSP template modules requires significantly less area when implemented as a single reconfigurable partition than when all three are instantiated simultaneously in the static fabric. ACNNE also serves as a benchmark for expected reconfiguration latency numbers when evaluated against the `recon_counter` measurements described in Chapter 6.

2.7 Summary and Synthesis

The literature reviewed in this chapter covers a forty-year arc of DPR research and practice, from the XC6200's register-mapped partial cell access in 1997 to the sub-millisecond BRAM-backed controllers and automated edge toolflows of 2024. The following observations synthesise the key findings with respect to the system described in this report.

On compile-time: RapidStream and HPR independently confirm that island-based parallel compilation — enabled by the latency-insensitive property of dataflow designs and by RapidWright's bitstream-level stitching infrastructure — can reduce FPGA compile times by one to two orders of magnitude relative to monolithic Vivado flows. Neither approach has been directly applied in this project due to toolflow inaccessibility (RapidStream) and platform incompatibility (HPR), but both provide the theoretical framework for extending the present implementation toward production-grade compilation automation.

On runtime management: ZyCAP/ZyPR and DML collectively characterise the state of the art in runtime reconfiguration management. ZyCAP's 400 MB/s DMA-based reconfiguration path represents the practical performance ceiling on Zynq SoC platforms, while DML's multi-application scheduler represents the software-side framework that would be needed to manage a diverse CFU library across concurrent workloads. Both set design targets for the autonomous reconfiguration roadmap outlined in §4.7.

On RISC-V integration: AMPER-X and RV-CAP directly precede this work in the specific domain of RISC-V core DPR integration. AMPER-X establishes the DFX Decoupler as the correct isolation primitive at the processor-RP interface, and RV-CAP establishes ICAP-based reconfiguration as viable on Artix-7 class devices from a VexRiscv control plane. The present work occupies the intersection of both precedents, integrating them within the structured CFU Playground framework and extending them to the CFU-specific interface constraints.

On application validation: ACNNE confirms that DPR-based hardware time-multiplexing achieves the predicted 30–45% area reduction for small accelerator libraries on Zynq-class devices, and that reconfiguration overhead of 30–45 ms per swap is compatible with workloads whose computation time per region significantly exceeds this figure. These benchmarks provide a credible baseline against which the reconfiguration latency measurements reported in Chapter 6 can be compared.

Chapter 3

Background and Case Study

3.1 FPGA Partial Reconfiguration Fundamentals

The design and implementation decisions described in Chapter 4 are grounded in a set of hardware primitives, configuration protocols, and floorplanning constraints that are specific to AMD Xilinx 7-series FPGAs. This section reviews those fundamentals at the level of detail required to understand the system architecture.

3.1.1 Configuration Frames and the Frame Address Register

The entire configuration state of an Xilinx 7-series FPGA — the logic function of every LUT, the routing state of every switchbox multiplexer, and the initial contents of every BRAM — is encoded in a sequence of *frames*. Each frame is a fixed-width column of configuration bits that extends the full height of one clock region (50 CLB rows, for the Artix-7 family). Frames are identified by a three-field *Frame Address Register* (FAR) value comprising a block type field (indicating whether the frame addresses CLB, BRAM content, or BRAM interconnect resources), a major address (the column index within the block type), and a minor address (the position within the column). Both devices used in this project — the XC7A100T on the Arty A7-100T and the XC7A35T on the Arty A7-35T — share this frame-addressed configuration model, differing only in the total number of configuration frames.

Partial Reconfiguration is made possible by the frame-addressed nature of this configuration space. A *Reconfigurable Partition* (RP) is defined by a rectangular bounding box on the device's floorplan — specified as a Pblock constraint within the Vivado DFX flow — that encloses complete columns of each resource type (CLBs, BRAMs, DSPs) present within its area. The set of FAR values corresponding to the frames within this bounding box is fixed at design time and embedded in the partial bitstream header. When a partial bitstream is subsequently written to the device's configuration memory, only the frames whose FAR values fall within this set are overwritten; frames outside this set — representing the static partition — are guaranteed to remain unmodified at the hardware level, because the configuration access port terminates its write sequence precisely at the last frame in the partial bitstream's address range.

It is this hardware guarantee — that an active write to the configuration memory of one address range cannot affect the contents of any other range — that makes non-disruptive partial reconfiguration possible. The static partition's logic, including the processor subsystem, interconnect, and reconfiguration controller, continues to operate normally while the write to the RP frames is in

progress.

3.1.2 The DFX Decoupler IP

Although the frame-addressed configuration model ensures that the static logic remains electrically undisturbed during partial reconfiguration at the configuration memory level, there remains a functional hazard at the logic level: the interface signals connecting the static partition to the RP may carry undefined or toggling values during the window in which the RP's logic cells are being overwritten. This arises because a frame write propagates through the RP's CLBs in a non-deterministic order — some cells may have already been updated to the new configuration while others retain the old — and the resulting glitch activity on the RP's output ports can corrupt the static partition's internal state.

The AMD DFX Decoupler IP core addresses this hazard by interposing a set of programmable isolation registers between the static and reconfigurable sides of the partition interface. When a register named `decouple` is asserted high through an AXI-Lite control write (or, in the present system, through a direct wire connection), the Decoupler drives all signals from the RP side to the static side to pre-configured default values — typically zero or the idle state of the relevant handshake signal — while simultaneously holding the signals from the static side to the RP side at their own defaults. The RP's outputs are thereby electrically clamped and the static partition's inputs receive well-defined constant values for the duration of the reconfiguration window, preventing any glitch activity from propagating.

When `decouple` is de-asserted, the Decoupler becomes transparent, passing all signals bidirectionally with negligible and fixed latency. The transition from clamped to transparent must occur only after EOS (End-of-Startup) has been re-asserted by the STARTUPE2 primitive, signalling that the configuration memory write has completed and the newly loaded logic has been fully initialised. The correct sequencing of `decouple` assertion, bitstream transfer, EOS polling, and `decouple` release constitutes the core of the reconfiguration control protocol implemented in the `system_wrapper` module.

3.1.3 Static and Reconfigurable Partitions on the Arty A7-35T

The primary CFU Playground DFX system targets the Arty A7-35T, which is based on the Xilinx XC7A35T (xc7a35ticsg324-1L, industrial speed grade). The XC7A35T provides 20,800 six-input LUTs, 41,600 flip-flops, 50 BRAM tiles (each 36 Kb), 90 DSP48E1 slices, and a single ICAP primitive. Despite its smaller resource budget compared to the XC7A100T — which was used for the DFX tutorial baseline (§4.6) — the XC7A35T provides sufficient fabric to host the complete LiteX SoC alongside the reconfigurable CFU partition. The constrained resource budget of the Artix-7 family — particularly relative to the Zynq-7020 on the PYNQ-Z2, which additionally provides a hardened ARM dual-core PS with 512 KB on-chip memory — directly motivated the multi-platform implementation strategy described in §4.5.

For the DFX-enabled design targeting the XC7A35T, the static partition is defined to include the complete VexRiscv LiteX SoC (the `digilent_arty` wrapper, which accounts for the majority of the device's LUT and BRAM consumption), the `system_wrapper` reconfiguration controller, the `recon_counter` latency measurement module, and all board-level I/O logic including DDR3 SDRAM, QSPI flash, and UART interfaces. The Reconfigurable Partition, designated the `cfu_compute` slot, is constrained to a smaller Pblock that must provide sufficient CLBs and DSP slices to accommodate the largest anticipated CFU Reconfigurable Module (RM), while respecting the placement requirements

of the static partition's surrounding logic.

The floorplanning requirement that the RP's Pblock must contain only complete resource columns — a consequence of the frame-addressed configuration model described in §3.1.1 — constrains the minimum and permissible sizes of the `cfu_compute` Pblock significantly. On Artix-7 devices, the column granularity for CLBs is one CLB wide and 50 rows tall per clock region. The selected Pblock for `cfu_compute` was iteratively refined in the Vivado DFX flow until all RMs (including `donut.v`, `example.v`, and `dse_template.v`) could be placed and routed within the designated area without overflowing into the static partition's reserved columns, and without generating timing violations on the critical path through the DFX Decoupler's isolation registers.

3.2 CFU Playground and VexRiscv

3.2.1 The CFU Playground Framework

The CFU Playground is an open-source hardware/software co-design framework developed by Google to facilitate the rapid design, integration, and benchmarking of Custom Function Units (CFUs) attached to a RISC-V soft-core processor [33]. Its primary purpose is to lower the entry barrier for hardware acceleration research by providing a complete, pre-validated RISC-V SoC baseline — ready to run on commodity FPGA development boards — that the designer can augment with custom hardware accelerators without modifying the processor pipeline or the SoC infrastructure.

The framework provides: (a) the LiteX SoC generator, which produces synthesisable Verilog for the processor, memory controllers, UART, and peripheral interconnect from a board-targeted Python script; (b) the VexRiscv soft core, pre-configured with Wishbone instruction/data buses and a CFU co-processor port; (c) a firmware library enabling RISC-V software to invoke CFU operations through custom instructions encoded in the CUSTOM0 opcode space; and (d) a suite of benchmark applications — including keyword spotting (KWS), matrix multiply, and convolutional inference primitives — that exercise the CFU interface and report throughput in operations per second. When a CFU is connected, the firmware calls are dispatched through the processor's CFU port to the hardware implementation and return the hardware-computed result to the software; when no CFU is present, the framework's software reference implementations are invoked instead, providing a direct baseline for measuring hardware acceleration speedup.

For the purposes of the present work, the CFU Playground's significance lies not in its benchmarking infrastructure but in the SoC structure it produces. The LiteX-generated `digilent_arty` Verilog wrapper for the Arty A7-35T provides a complete, timing-closed, synthesisable SoC that serves as the static partition in the DFX design. This substantially reduces the engineering effort required to construct a RISC-V co-processor environment suitable for DFX experimentation, since the DDR3 controller, QSPI flash driver, UART, and VexRiscv pipeline are all provided as a pre-validated unit rather than being designed from scratch.

3.2.2 VexRiscv: Architecture and CFU Port

VexRiscv [34] is an RV32I soft-core processor implemented in SpinalHDL and synthesised to Verilog as part of the CFU Playground build process. It implements a four-stage in-order pipeline — Fetch, Decode, Execute, Writeback — with Wishbone-compatible instruction and data memory interfaces. Within the CFU Playground configuration, VexRiscv is extended with a CFU co-processor port that exposes a set of streaming handshake signals directly from the Execute stage.

The CFU port interface is defined by the following signals:

Signal	Direction (processor → CFU)	Width	Description
`cmd_valid`	Output	1	Asserted when the processor has a valid CFU command to dispatch
`cmd_ready`	Input	1	Asserted by the CFU when it can accept a new command
`cmd_payload_function_id`	Output	10	Selects the specific operation within the CFU implementation
`cmd_payload_inputs_0`	Output	32	First 32-bit operand (rs1 value from the register file)
`cmd_payload_inputs_1`	Output	32	Second 32-bit operand (rs2 value from the register file)
`rsp_valid`	Input	1	Asserted by the CFU when a response is ready
`rsp_ready`	Output	1	Asserted by the processor when it is ready to accept the response
`rsp_payload_outputs_0`	Input	32	The 32-bit result, written into the destination register (rd)

The handshake protocol operates in the style of AXI4-Stream: a transaction is accepted when both `cmd_valid` and `cmd_ready` are simultaneously asserted, and a response is consumed when both `rsp_valid` and `rsp_ready` are simultaneously asserted. The processor pipeline stalls at the Execute stage from the moment a CUSTOM0 instruction is dispatched (i.e., `cmd_valid` is asserted) until `rsp_valid` is returned by the CFU, at which point the result is written into the destination architectural register and the pipeline is released. This stall behaviour ensures functional correctness for any CFU that is not pipelined relative to the processor, at the cost of throughput equal to one CFU operation per dispatch-to-response cycle count.

The `function_id` field encodes both the *funct3* field (bits [2:0]) and the immediate field of the CUSTOM0 instruction encoding (bits [9:3]), providing up to 1,024 distinct addressable operations per CFU module. In practice, each RM in the present system uses a small number of `function_id` values — for example, `example.v` responds to `function_id = 0` (byte sum), `function_id = 1` (byte swap), and `function_id = 2` (bit reverse) — with unused IDs returning a zero result.

3.2.3 The CFU–RP Interface in the DFX Context

In a standard CFU Playground deployment, the `Cfu` module is instantiated directly within the processor SoC and connected permanently to the VexRiscv CFU port. For DFX operation, this

connection must be restructured: the `Cfu` module becomes a static wrapper, within which the DFX Decoupler and the RP instantiation reside. The VexRiscv CFU port connects to the *static side* of the Decoupler (the `s_*` ports), and the *reconfigurable-partition side* (the `rp_*` ports) connects to the `cfu_compute` module within the RP. This three-level hierarchy — processor port → Decoupler → `cfu_compute` — is the key structural adaptation that transforms a standard CFU Playground design into a DFX-capable one, and is described in full in §4.3.

3.3 The ICAP and STARTUPE2 Primitives

3.3.1 ICAP on 7-Series FPGAs

The Internal Configuration Access Port (ICAP) is a hard-macro primitive embedded in the fabric of all Xilinx 7-series FPGAs. It provides a synchronous 32-bit-wide write interface through which configuration frames can be written to the device's configuration memory from logic executing within the PL fabric itself, without requiring access to the external JTAG, SelectMAP, or SPI configuration interfaces. On all 7-series devices except the smallest family members, ICAP is available as the `ICAP_7SERIES` primitive and can be clocked at up to 100 MHz, yielding a theoretical maximum bitstream write throughput of 400 MB/s (32 bits × 100 MHz × 8 bits per byte).

The ICAP interface signals relevant to partial reconfiguration are:

Signal	Direction	Width	Description
`CLK`	Input	1	Clock input; must be a non-inverted, stable clock
`CE`	Input	1	Chip enable (active low on 7-series, denoted `CSIB`)
`WRITE`	Input	1	Read/write select (active low write, denoted `RDWRB`)
`I`	Input	32	Configuration data word to write
`O`	Output	32	Configuration readback data

A partial bitstream write sequence proceeds as follows: the ICAP controller drives `CSIB` low and `RDWRB` low to enable write mode, and then presents the 32-bit configuration words — beginning with the synchronisation word `0xAA995566` and continuing through the FAR, FDRI, and frame data sequences described in §3.1.1 — at the `I` input on consecutive rising clock edges. The ICAP accepts one word per clock cycle. At the conclusion of the sequence, a DESYNC command is issued (`0x0000000D` to the CMD register), `CSIB` is de-asserted, and `RDWRB` is returned high.

An important implementation detail specific to 7-series ICAP is the bit-reversal requirement: configuration words must be presented to the ICAP in bit-reversed byte order relative to the bit ordering used in the `.bit` file stored on the host or flash memory. Each 32-bit word must have its bytes individually bit-reversed (i.e., bit 7 of byte 0 becomes bit 0, bit 6 becomes bit 1, and so forth, independently for each of the four bytes). Failure to apply this reversal results in malformed frames

being written to the configuration memory, producing either a configuration error or a silently incorrect fabric function. The `system_wrapper` module's ICAP controller accounts for this byte-level bit reversal in its data path.

3.3.2 The STARTUPE2 Primitive and the EOS Signal

The `STARTUPE2` (Startup) primitive is a hard-macro present in 7-series FPGAs that provides access to several configuration-phase signals not otherwise exposed to PL logic. Of these, the *End-of-Startup* (EOS) output signal is the one used in the present work.

EOS is asserted high by the device's configuration state machine at the conclusion of the device's startup sequence — the series of events that follows a successful full or partial bitstream load and that initialises PLLs, resets flip-flops, and propagates GSR (Global Set/Reset) to all registered elements in the newly configured region. Specifically, EOS transitions from high to low at the moment the ICAP begins accepting the partial bitstream write sequence, and returns to high once the startup sub-sequence for the reconfigured partition is complete and its logic is fully operational.

This behaviour makes EOS a precise hardware signal for delineating the reconfiguration window: the falling edge of EOS marks the start of reconfiguration and the rising edge marks the point at which the new RM's logic is safe to use. The `recon_counter` module exploits this property directly: it instantiates `STARTUPE2`, connects its `CLK` input to the system clock, and monitors the `EOS` output. A two-state FSM transitions from `IDLE` to `RECON` on the falling edge of `EOS`, increments a 32-bit counter on every clock cycle while in the `RECON` state, and returns to `IDLE` on the rising edge of `EOS`. The resulting counter value, expressed in clock cycles, yields the reconfiguration duration in physical time as `counter × (1 / f_clk)` — at 100 MHz, one counter tick corresponds to 10 ns. This counter value is monitored externally through an ILA (Integrated Logic Analyser) core connected to its output.

The `recon_counter.v` implementation is shown below for reference:

```
module recon_counter (
  input clk,
  output eos
);

(* KEEP = "true" *)
wire preq;

STARTUPE2 STARTUPE2_inst (
   .EOS(eos),           // Active-high End-of-Startup output
   .CLK(clk),           // User start-up clock input
   .GSR(1'b0),
   .GTS(1'b0),
   .PACK(1'b0),
   .USRCCLKO(1'b0),
   .USRCCLKTS(1'b1),
   .KEYCLEARB(1'b1),
   .USRDONEO(1'b1),
   .USRDONETS(1'b0)
);

(* KEEP = "true" *)
reg [31:0] counter = 0, counter_next;
```

```
(* KEEP = "true" *)
localparam IDLE = 0, RECON = 1;

(* KEEP = "true" *)
reg state = 0, next_state;

always @(*) begin : counter_state
  case (state)
  IDLE: begin
    next_state   = (!eos) ? RECON : IDLE;
    counter_next = counter;
  end
  RECON: begin
    next_state   = (eos) ? IDLE : RECON;
    counter_next = counter + 1;
  end
  default: begin
    next_state   = IDLE;
    counter_next = counter;
  end
  endcase
end

always @(posedge clk) begin : counter_switch
  state <= next_state;
  if (preq)
    counter <= 32'd0;
  else
    counter <= counter_next;
end

ila_0 ila(.clk(clk), .probe0(counter));

endmodule
```

The `(* KEEP = "true" *)` attributes on the state registers and counter prevent Vivado's synthesis engine from optimising away these signals, which would otherwise be candidates for removal as they drive only the ILA probe — a signal that Vivado's dataflow analyser does not traverse in its dead-code elimination pass.

3.3.3 Partial Bitstream Structure

A well-formed partial bitstream for a 7-series Reconfigurable Partition consists of the following ordered fields [5]:

1. **Padding words** (0xFFFFFFFF): A series of dummy words that allow the configuration logic to synchronise its input shift register before processing begins.
2. **Synchronisation word** (0xAA995566): A fixed magic constant that the configuration state machine uses to identify the beginning of a valid configuration stream, distinguishing it from random device bus activity.

3. **Type-1 packet header: NOP** (0x20000000): A no-operation packet that advances the state machine without performing a register write.
4. **Type-1 packet: IDCODE write** — writes the device's identity code into the IDCODE register, which the state machine validates as a consistency check before proceeding with frame writes.
5. **Type-1 packet: FAR initialisation** — writes the starting Frame Address Register value corresponding to the first frame of the Reconfigurable Partition's address range.
6. **Type-1 packet: FDRI write command** — specifies that the next N words are configuration frame data to be written starting at the current FAR value.
7. **Frame data words** (N $\times$ 32-bit): The actual configuration frame contents for the RP, sequentially written starting at the FAR-specified address and incrementing the internal FAR counter with each frame.
8. **CRC word** — the running CRC over the preceding configuration data stream, checked by the state machine to detect transmission errors.
9. **RCRC packet** — resets the internal CRC register to a known state after the frame data.
10. **Shutdown / DGHIGH / GRESTORE sequence** — places the newly configured logic into the startup state, asserts GSR to initialise all flip-flops in the RP, and then releases GSR, triggering the startup sequence that culminates in the EOS rising edge.
11. **DESYNC word** (0x0000000D to CMD) — instructs the configuration state machine to exit configuration mode and return to normal operation.

The total partial bitstream size for the `cfu_compute` partition in the present design is determined by the area of the Pblock's bounding box: specifically, by the number of CLB, BRAM, and DSP columns within the box, multiplied by the minor frame count per column and the full device height per clock region. For the Pblock dimensions used in this project, the resulting partial bitstream sizes for the three RMs range from 352 KB to 420 KB, reflecting minor differences in the frame padding introduced by Vivado for different RM implementations targeting the same Pblock.

3.4 System Architecture Overview

3.4.1 Top-Level Module Decomposition

The complete DFX-enabled system is assembled in a top-level Verilog module (`top.v`) that instantiates two principal sub-modules at the same hierarchy level, connected by a single shared wire — the `decouple` signal:

```
top (top.v)
 +-- system_wrapper         Reconfiguration controller (static partition
   )
 |    +-- ICAP port signals (CSIB, I, O, RDWRB)
 |    +-- QSPI flash interface
 |    +-- pr_switch input (board button)
 |    +-- decouple output wire
 |
 +-- digilent_arty          VexRiscv LiteX SoC (static partition)
      +-- clk100, cpu_reset
```

```
      +-- serial_tx / serial_rx (UART to host)
      +-- user_led[3:0]
      +-- DDR3 SDRAM interface
      +-- decouple input     cfu.v     DFX Decoupler     cfu_compute (RP
   )
```

The `ICAP_7SERIES` primitive is instantiated at the board level (via Vivado block-design constraints) with its control and data ports exposed as top-level I/O signals (`ICAP_csib`, `ICAP_i`, `ICAP_o`, `ICAP_rdwrb`), which are in turn connected to `system_wrapper`. Although the ICAP is a hard macro within the device fabric and is not an external I/O in the traditional sense, treating its signals as top-level ports in the Vivado project structure simplifies the DFX flow's constraint management and keeps the ICAP instantiation within the static partition's scope.

The `decouple` wire is the sole inter-module signal connecting `system_wrapper` to `digilent_arty`. This deliberate minimisation of the static-to-static coupling at the top level reflects the architectural principle that the reconfiguration controller and the processor SoC are logically independent subsystems: the SoC does not need to be aware of the controller's internal state beyond knowing whether its CFU partition is currently isolated.

3.4.2 Signal Flow from Processor to Reconfigurable Partition

The complete signal path from a RISC-V custom instruction to the reconfigurable CFU logic traverses the following hierarchy:

1. **VexRiscv Execute stage** dispatches a CUSTOM0 instruction. The `cmd_valid` signal and the associated `function_id`, `inputs_0`, and `inputs_1` signals are asserted on the CFU port at the boundary of the `digilent_arty` wrapper.
2. **`cfu.v` (the `Cfu` module)** receives these signals on its static-side port. It passes them to the *static side* of the `dfx_decoupler_0` instantiation. When `decouple = 0`, the Decoupler forwards the signals transparently to the RP side. When `decouple = 1`, the Decoupler holds the processor-side signals at their default idle values (specifically, `cmd_ready = 0`, preventing any new instruction from being accepted) while simultaneously clamping the RP-side outputs to zero.
3. **`cfu_compute` (the RP module)** receives the forwarded signals from the Decoupler's RP side. It evaluates the function identified by `function_id`, computes the result using whichever logic is currently instantiated in the RP (the active RM), asserts `rsp_valid`, and places the 32-bit result on `rsp_payload_outputs_0`.
4. The response propagates back through the Decoupler to `cfu.v`'s static-side output port, and thence to the VexRiscv pipeline's `rsp_valid` and `rsp_payload_outputs_0` inputs, releasing the pipeline stall and writing the result into the destination register.

The `cfu.v` source that implements this hierarchy is reproduced below:

```
module Cfu (
  input                cmd_valid,
  output               cmd_ready,
  input       [9:0]    cmd_payload_function_id,
  input       [31:0]   cmd_payload_inputs_0,
  input       [31:0]   cmd_payload_inputs_1,
  output               rsp_valid,
```

```
  input                 rsp_ready,
  output      [31:0]    rsp_payload_outputs_0,
  input                 decouple,
  input                 reset,
  input                 clk
);

  wire rp_cmd_valid, rp_cmd_ready, rp_rsp_valid, rp_rsp_ready;
  wire [9:0]  rp_cmd_payload_function_id;
  wire [31:0] rp_cmd_payload_inputs_0;
  wire [31:0] rp_cmd_payload_inputs_1;
  wire [31:0] rp_rsp_payload_outputs_0;
  wire rp_reset, rp_clk;

  cfu_compute comp (
    .clk(rp_clk), .reset(rp_reset),
    .cmd_payload_function_id(rp_cmd_payload_function_id),
    .cmd_payload_inputs_0(rp_cmd_payload_inputs_0),
    .cmd_payload_inputs_1(rp_cmd_payload_inputs_1),
    .rsp_payload_outputs_0(rp_rsp_payload_outputs_0),
    .cmd_valid(rp_cmd_valid), .cmd_ready(rp_cmd_ready),
    .rsp_valid(rp_rsp_valid), .rsp_ready(rp_rsp_ready)
  );

  dfx_decoupler_0 decoupler (
    .s_function_id_DATA(cmd_payload_function_id),
    .rp_function_id_DATA(rp_cmd_payload_function_id),
    .s_rsp_payload_DATA(rsp_payload_outputs_0),
    .rp_rsp_payload_DATA(rp_rsp_payload_outputs_0),
    .s_cmd_payload_cmd_payload_0(cmd_payload_inputs_0),
    .rp_cmd_payload_cmd_payload_0(rp_cmd_payload_inputs_0),
    .s_cmd_payload_cmd_payload_1(cmd_payload_inputs_1),
    .rp_cmd_payload_cmd_payload_1(rp_cmd_payload_inputs_1),
    .s_controls_clk(clk),          .rp_controls_clk(rp_clk),
    .s_controls_cmd_valid(cmd_valid), .rp_controls_cmd_valid(
   rp_cmd_valid),
    .s_controls_cmd_ready(cmd_ready), .rp_controls_cmd_ready(
   rp_cmd_ready),
    .s_controls_rsp_ready(rsp_ready), .rp_controls_rsp_ready(
   rp_rsp_ready),
    .s_controls_rsp_valid(rsp_valid), .rp_controls_rsp_valid(
   rp_rsp_valid),
    .s_controls_reset(reset),      .rp_controls_reset(rp_reset),
    .decouple(decouple),
    .decouple_status()
  );

endmodule
```

A point of note is that the DFX Decoupler is located *within* `cfu.v` — which is itself part of the static partition — rather than at the RP boundary itself. This placement ensures that the Decou-

pler's isolation registers are always present and always have a valid power supply regardless of the RP's reconfiguration state. Instantiating the Decoupler within the RP would create a paradox: the isolation logic would itself be temporarily undefined during the reconfiguration of the partition it is supposed to protect.

3.4.3 Reconfiguration Controller Hierarchy

The `system_wrapper` sub-module forms the reconfiguration controller subsystem within the static partition. Its responsibilities are:

- **Generating the `decouple` signal** in response to the `pr_switch` input (a board push-button on the Arty A7-35T), asserting isolation before any bitstream write begins and releasing it after EOS is confirmed.
- **Driving the ICAP interface** (via `ICAP_csib`, `ICAP_i`, `ICAP_rdwrb`) with the appropriate configuration word sequence for the target partial bitstream.
- **Managing QSPI flash access** (via the six QSPI I/O signals) for systems where partial bitstreams are stored on-board in the 128 Mb (16 MB) Quad-SPI flash memory present on the Arty A7-35T.

The `recon_counter` module is instantiated within or alongside `system_wrapper`, connected to the same 100 MHz system clock and with its `eos` output monitored by the embedded ILA core.

The architectural relationship between the subsystems is depicted below:

```
                +-----------------------------------------------+
                |              STATIC PARTITION                 |
                |                                               |
pr_switch --->|  system_wrapper                               |
                |  +-- decouple ------------------------------>|-->
 dfx_decoupler_0 (in cfu.v)
                |  +-- ICAP_csib/i/rdwrb --> ICAP_7SERIES    |
                |  +-- QSPI flash control                       |
                |                                               |
clk100 ------>|  recon_counter                                |
                |  +-- STARTUPE2     EOS                          |
                |  +-- 32-bit counter                           |
                |  +-- ila_0 probe                              |
                |                                               |
serial_rx --->|  digilent_arty (VexRiscv LiteX SoC)         |
serial_tx <--|  +-- Cfu (cfu.v)                              |
DDR3 <------->|       +-- dfx_decoupler_0                     |
                |       |   +-- static side     VexRiscv port   |
                |       |   +-- RP side     cfu_compute          |
                |       +-- [decouple from system_wrapper]     |
                +-----------------------------------------------+
                          | RP boundary |
                +---------v----------------+
                |  RECONFIGURABLE PARTITION |
                |      cfu_compute (RM)     |
                |  (donut.v / example.v /   |
                |   dse_template.v / kws)   |
                +---------------------------+
```

This architecture reflects the three design principles introduced in Chapter 1: (1) the static partition encapsulates all persistent compute and control infrastructure; (2) the DFX Decoupler provides a clean, hardware-enforced isolation boundary that prevents glitch propagation; and (3) the ICAP and STARTUPE2 primitives, both hard macros, are accessed exclusively from within the static partition to ensure that the reconfiguration control path itself is never affected by the act of reconfiguration.

Chapter 4

Design and Implementation

4.1 Design Goals

The central objective of the design described in this chapter is the demonstration of runtime partial reconfiguration of a RISC-V co-processor's Custom Function Unit (CFU) without halting the processor pipeline or disrupting the surrounding SoC infrastructure. This objective is decomposed into four concrete engineering goals that collectively define the scope and success criteria of the implemented system.

Goal 1 — Runtime CFU swap without processor reset. The Reconfigurable Module (RM) occupying the `cfu_compute` slot must be replaceable at runtime through a controlled sequence of isolation, bitstream transfer, and reconnection, with no assertion of the VexRiscv processor's reset line and no interruption of the UART communication between the host workstation and the LiteX SoC. The processor pipeline may stall during the reconfiguration window — specifically, the DFX Decoupler will hold `cmd_ready` low, preventing any CUSTOM0 instruction from being accepted — but it must resume normal execution once the new RM is active without requiring a software restart.

Goal 2 — Hardware-timed reconfiguration latency measurement. The duration of each reconfiguration event must be measured at hardware precision, expressed in terms of the number of 100 MHz clock cycles elapsed between the assertion and de-assertion of the STARTUPE2 End-of-Startup (EOS) signal. The `recon_counter` module, described in §3.3.2, fulfils this role. Its 32-bit cycle counter is read out via an Integrated Logic Analyser (ILA) probe, allowing the reconfiguration time to be recorded without modifying the firmware or processor software stack. At 100 MHz, the counter resolution is 10 ns per tick.

Goal 3 — A functionally valid Reconfigurable Partition and multiple Reconfigurable Modules. The `cfu_compute` Pblock must be demonstrated to host at least two distinct, functionally correct RMs that produce verifiably different outputs to the same CFU invocations, thereby confirming that both the synthesis of new RMs into the fixed Pblock and the runtime swap are operating correctly. The use of multiple RMs with different logic — rather than a trivially identical pair — eliminates the possibility of false positives caused by incomplete reconfiguration or silent failures in the bitstream write path.

Goal 4 — A validated baseline using the AMD DFX tutorial. Before deploying the full CFU Playground DFX system, the AMD UG947 Dynamic Function eXchange tutorial is reproduced as a sanity check. This baseline validates that the Vivado DFX toolflow on the Arty A7-100T produces correct partial bitstreams, that the JTAG-based reconfiguration path functions as expected, and that

the timing and resource utilisation of a known-good design are consistent with published figures. Any deviation from the expected baseline indicates a toolflow misconfiguration rather than a design error.

4.2 Static Partition Design

The static partition encompasses all logic that must remain powered and functionally active throughout — and between — reconfiguration events. Its correct design is critical because any functional or timing error within the static partition cannot be corrected at runtime without a full device reconfiguration, which defeats the purpose of the DPR system.

4.2.1 The `system_wrapper` Reconfiguration Controller

The `system_wrapper` module serves as the orchestration layer for all reconfiguration-related hardware tasks. It is assigned entirely to the static partition and contains the following functional blocks:

ICAP interface logic. The module drives the `ICAP_7SERIES` primitive's control signals (`CSIB`, `RDWRB`) and data bus (`I[31:0]`). It presents the configuration word sequence — beginning with the synchronisation header and proceeding through the frame data words as described in §3.3.3 — to the ICAP on consecutive rising clock edges at 100 MHz. The bit-reversal transformation required by 7-series ICAP (each byte individually bit-reversed within its 32-bit word) is performed combinationally in the data path before the word is presented at the `I` port.

Decouple control logic. The `decouple` output wire is asserted in response to the `pr_switch` input, which is connected to a push-button (SW0) on the Arty A7-35T's user I/O bank. The assertion sequence is: assert `decouple` → wait for a configurable number of clock cycles to allow the DFX Decoupler's isolation registers to settle → commence ICAP write sequence → poll for EOS falling edge → continue ICAP write until bitstream is fully transferred → wait for EOS rising edge → de-assert `decouple`. This sequence ensures that the isolation boundary is established before any frame is written and is released only after the new RM's startup sequence is complete.

QSPI flash interface. The `system_wrapper` also contains the control logic for the board-mounted 128 Mb (16 MB) Quad-SPI (QSPI) flash memory on the Arty A7-35T (S25FL128S device, connected via the `qspi_flash` I/O). Partial bitstreams are intended to be pre-loaded into dedicated flash address regions at board programming time (via the Vivado Hardware Manager's `program_flash` command). During reconfiguration, the flash controller issues a fast-read command to the QSPI device, streams the response bytes into a 32-bit assembly register, applies the byte-level bit reversal, and forwards the resulting words to the ICAP data input. As discussed in §4.7, full integration of the flash-fetch path with the ICAP writer is the primary outstanding engineering task; in the current implementation, bitstreams are presented to the ICAP via a direct in-fabric ROM, populated at synthesis time.

4.2.2 Clock and Reset Distribution

The Arty A7-35T board supplies a 100 MHz single-ended crystal oscillator, brought onto the FPGA via the `clk100` input. Within the LiteX SoC, this is routed through a Mixed Mode Clock Manager (MMCM) that generates the 100 MHz system clock (`clk100`) distributed to both the `digilent_arty` domain and the `system_wrapper`. The ICAP is also clocked from this same `clk100` source, satisfying the 7-series requirement that ICAP be driven by a non-inverted, single-ended, jitter-minimised clock routed through a Global Clock Buffer (BUFG).

The `cpu_reset` signal — a board push-button (BTN0) — is connected exclusively to the VexRiscv

reset port within `digilent_arty`. It does not propagate to `system_wrapper` or the ICAP controller, which remain running continuously. This separation is deliberate: a processor reset should never abort an in-progress reconfiguration sequence, as an incomplete bitstream write to the ICAP would leave the configuration memory in an undefined state.

4.2.3 Board I/O Allocation

The static partition's I/O allocation on the Arty A7-35T is governed by the board's pin-constraint file (`.xdc`). The following table summarises the principal I/O interfaces assigned to the static partition:

Interface	Signals	Function
Crystal oscillator	`sys_clk_p`, `sys_clk_n`	100 MHz system clock
UART	`serial_rx`, `serial_tx`	Host communication for CFU Playground
DDR3 SDRAM	32 signals (address, data, control)	Runtime memory for LiteX SoC
QSPI flash	`qspi_flash_io[3:0]`, `clk`, `cs_n`	Partial bitstream storage
Reconfiguration switch	`sw[0]`	Triggers PR sequence
User LEDs	`led[3:0]`	Status indication
JTAG	Implicit (Digilent USB-JTAG)	Hardware Manager + ILA
PMOD JA	`ja[7:0]`	UART bridge (Phase 3, §4.5)

4.3 Reconfigurable Partition Design

The `cfu_compute` Reconfigurable Partition (RP) is designed to present the same port interface to the DFX Decoupler regardless of which Reconfigurable Module (RM) occupies it at any given time. This interface invariance is the fundamental requirement imposed by the DFX toolflow: all RMs for a given RP must share an identical port list, port directions, and port widths. Vivado's DFX implementation enforces this constraint by treating the RP's port interface as a fixed routing stub within the static partition's implementation checkpoint, into which each RM is independently placed and routed without modifying the surrounding static routes.

4.3.1 The `cfu.v` Wrapper and DFX Decoupler Placement

As described in §3.2.3, the `cfu.v` module constitutes the static wrapper that bridges the VexRiscv CFU port to the RP. The DFX Decoupler (`dfx_decoupler_0`) resides within `cfu.v`, placed in the static partition. The RP — the `cfu_compute` instantiation — is connected to the Decoupler's RP-side ports.

This placement satisfies two constraints simultaneously. First, it ensures that the isolation registers, which must be reliable even when the RP's configuration memory is in a transitional state, are always within the static partition's guaranteed-stable logic. Second, it allows the Decoupler's static-side behaviour — in particular, the clamping of `cmd_ready` to 0 during decoupling — to be enforced by static logic independently of any state within the RP, preventing the processor from issuing a CFU command to undefined logic.

4.3.2 Reconfigurable Module Implementations

Four Reconfigurable Modules have been developed for the `cfu_compute` RP, each implementing the same top-level port interface:

`example.v` — Multi-function arithmetic RM. This RM implements three operations selectable via the `function_id` field:

- `function_id = 0`: Byte-sum — sums the four bytes of `inputs_0` and returns the 32-bit zero-extended result.
- `function_id = 1`: Byte-swap — returns `inputs_0` with its bytes in reversed order (i.e., `{B0, B1, B2, B3}` becomes `{B3, B2, B1, B0}`).
- `function_id = 2`: Bit-reverse — returns `inputs_0` with all 32 bits individually reversed.

All three operations complete in a single clock cycle (fully combinational), setting `rsp_valid` immediately after `cmd_valid` is asserted and the result is available.

`donut.v` — Iterative compute RM. This module implements a more computationally intensive operation that requires multiple clock cycles to produce its result, exercising the CFU handshake's ability to stall the VexRiscv pipeline across multiple cycles.

`dse_template.v` — Design Space Exploration stub. A parameterised template intended for rapid substitution of different arithmetic datapaths (e.g., integer multiply-accumulate configurations) to evaluate area-performance trade-offs without modifying the surrounding DFX infrastructure. The current instantiation implements a 16×16-bit unsigned multiply with 32-bit accumulate, returning the accumulated result after a fixed two-cycle latency.

KWS (Keyword Spotting) module. A reduced-precision inference accelerator stub targeting the CFU Playground's keyword spotting benchmark. This RM represents the application motivation for the broader DPR system: at runtime, the CFU slot is intended to be swapped from a general-purpose arithmetic RM to a domain-specific inference accelerator to serve a workload that cannot be efficiently executed on the VexRiscv's integer ALU.

4.3.3 Pblock Constraints and Floorplanning

The `cfu_compute` Pblock was defined iteratively through the Vivado DFX flow's guided-mode constraint assistant. The primary constraints imposed are:

- **Complete-column containment:** All CLB, BRAM, and DSP columns intersected by the Pblock's bounding box must be fully enclosed within the box — neither partial columns nor cross-boundary routing of RP signals are permitted.
- **Exclusion from static LUT/FF resources:** The Pblock must not overlap with the Pblock area required by the `digilent_arty` SoC, as simultaneous placement by both partitions would cause a resource conflict during RM implementation.
- **Timing closure of the DFX Decoupler interface:** The inter-partition path through the Decoupler's isolation registers must meet the 100 MHz timing constraint with positive worst-negative-slack (WNS). The maximum permissible routing delay across the partition boundary is determined by the Decoupler IP's timing model, which accounts for the register-register path through its two isolation flip-flop stages.
- **Minimum RM fit:** The Pblock must be large enough to accommodate the largest RM without placement overflow. The DPR design rule requires that all RMs be implementable within the

exact same physical placement boundary, so the Pblock size is governed by the largest RM's resource utilisation — in the present design, the KWS module, which consumes the greatest number of LUTs.

4.4 Reconfiguration Control Flow

The reconfiguration sequence is a precisely ordered protocol governing the transition from one active RM to another. Each step in the sequence is controlled by the `system_wrapper`'s finite state machine (FSM), and the correctness of the overall system depends on the steps being executed in the order described below. An incorrect ordering — for example, releasing the DFX Decoupler before the EOS rising edge — can result in the processor receiving undefined values from the RP's output ports during the period between the bitstream write's completion and the new RM's startup sequence conclusion.

4.4.1 Step-by-Step Reconfiguration Sequence

Step 1 — Reconfiguration trigger. The VexRiscv processor (or the board-level push-button `SW0`, in the current hardware validation configuration) asserts the `pr_switch` signal. The `system_wrapper` FSM transitions from its `IDLE` state to the `DECOUPLE_ASSERT` state.

Step 2 — Isolation assertion. The `system_wrapper` asserts the `decouple` wire high. The DFX Decoupler, receiving this signal on its `decouple` input, immediately clamps all signals on the RP side of its isolation registers: all data outputs to the RP are driven to zero, `cmd_valid` to the RP is driven low, and all outputs from the RP to the static side are overridden with zero (including `cmd_ready`, which is driven low, preventing the processor from completing any further CFU dispatch). A small settling delay (16 clock cycles at 100 MHz, i.e., 160 ns) is inserted before the ICAP write begins, allowing any in-flight pipeline stages within the Decoupler to drain.

Step 3 — ICAP write commencement. The `system_wrapper` FSM transitions to the `ICAP_WRITE` state and begins clocking configuration words into the ICAP. `CSIB` is driven low (active chip-enable) and `RDWRB` is driven low (write mode). The first word presented is the padding sequence (0xFFFFFFFF, repeated 8 times), followed by the synchronisation word (0xAA995566), and then the instruction and frame data sequence described in §3.3.3. All words are presented at the `I[31:0]` port of `ICAP_7SERIES` on the rising edge of `clk100`.

Step 4 — EOS falling edge (start of startup sequence). As the configuration state machine within the FPGA's configuration logic detects the synchronisation word and begins validating and writing the incoming frames, the STARTUPE2 primitive de-asserts the EOS output (EOS goes low). The `recon_counter` detects this falling edge and transitions to its `RECON` state, beginning to increment its 32-bit counter on every subsequent clock cycle.

Step 5 — ICAP write completion. After the final frame data word, the CRC, RCRC, and DESYNC words are presented. The ICAP write sequence is considered complete when all words have been clocked in. `CSIB` is then de-asserted (driven high) and `RDWRB` is returned high.

Step 6 — EOS rising edge (end of startup sequence). The configuration state machine raises EOS after the newly loaded RM's GSR de-assertion and startup sequence are complete. The `recon_counter` detects this rising edge, transitions back to `IDLE`, and holds the final counter value steady. The ILA probe captures this value.

Step 7 — Isolation release. The `system_wrapper` FSM, confirming that EOS is high, de-asserts

`decouple`. The DFX Decoupler immediately becomes transparent, passing all signals bidirectionally. The VexRiscv processor resumes normal CFU operation through the new RM.

4.4.2 Current Implementation State

In the validated implementation, the partial bitstream is presented to the ICAP from a block of initialised fabric registers (effectively an in-fabric ROM synthesised from the RM's bitstream bytes), rather than from an external flash or DDR source. This approach was adopted as an interim measure to validate the complete hardware reconfiguration sequence — including ICAP write, EOS monitoring, decoupler sequencing, and ILA-captured timing — on actual hardware before the flash-fetch and DDR storage paths described in §4.7 are completed. The limitation of this approach is that the bitstream must be embedded at synthesis time and cannot be updated at runtime; section §4.7 provides the roadmap for replacing this with a fully autonomous storage fetch mechanism.

4.5 Hardware Implementation Phases

The system described in the preceding sections was not developed against a single target platform from the outset. Instead, three distinct implementation phases were pursued across two physical hardware platforms — the Digilent Arty A7-35T and the Digilent PYNQ-Z2 — reflecting the evolving understanding of each platform's capabilities, constraints, and suitability for the project's objectives. The following subsections document each phase in full, including the methods validated, the challenges encountered, and the resolutions or workarounds adopted.

4.5.1 Phase 1 — Arty A7-35T (Standalone DPR Validation)

The Digilent Arty A7-35T served as the primary development platform for the entire project. Its selection was motivated by the board's native compatibility with the CFU Playground's LiteX build scripts, its Artix-7 XC7A35T fabric, and its availability of JTAG, QSPI flash, DDR3 SDRAM, and four PMOD connectors — providing sufficient I/O flexibility for both the CFU Playground UART communication path and the eventual PMOD-based inter-board bridge described in Phase 3.

The XC7A35T is a compact member of the Artix-7 family. Its logic density — 20,800 six-input LUTs, 41,600 flip-flops, 50 BRAM tiles of 36 Kb each, and 90 DSP48E1 slices — is sufficient to host the full LiteX SoC (including DDR3 controller, UART, and VexRiscv pipeline) in the static partition while reserving a meaningful physical area for the `cfu_compute` Reconfigurable Partition. The board's Digilent USB-JTAG interface (implemented by a Digilent-customised FTDI FT2232H bridge) exposes both the JTAG TAP controller (for bitstream programming and ILA access) and a UART channel (for CFU Playground firmware communication) as two distinct virtual serial ports to the host workstation, with no channel contention under normal single-board operation.

Vivado DFX Project Setup

The Vivado project for the Arty A7-35T DFX target was constructed using Vivado 2024.2 in project mode. The project configuration required several DFX-specific settings that differ from a standard non-DFX implementation project, each of which is described below.

Enabling Dynamic Function eXchange mode. The project property `PR_FLOW` was set to `1` in the project configuration, accessible through the *Project Settings → General → Dynamic Function eXchange* checkbox in the Vivado GUI, or equivalently via the `set_property PR_FLOW 1`

`[current_project]` TCL command. Enabling this mode activates the DFX-specific design rule checks (DRCs) that enforce Pblock correctness and port interface consistency, enables the partition-aware implementation passes within the place-and-route engine, and unlocks the per-RM partial bitstream generation step in the implementation run settings.

Defining Reconfigurable Partitions and Reconfigurable Modules. The `cfu_compute` instantiation within `cfu.v` was registered as a Reconfigurable Partition (RP) using the *Edit Dynamic Function eXchange Configurations* dialogue in the Project Manager. Each of the four RM implementations (`example.v`, `donut.v`, `dse_template.v`, and the KWS module) was registered as a separate Reconfigurable Module (RM) under this RP. One RM — `example.v` — was designated as the *default* configuration, which is synthesised and placed into the initial full bitstream. The remaining three RMs produce partial bitstreams only and are not embedded in the full bitstream.

Out-of-Context (OOC) synthesis for Reconfigurable Modules. Each RM is synthesised independently in an out-of-context synthesis run. The OOC run treats the RM's top-level ports as black-box stubs (not driven by or connected to any external logic), and produces a post-synthesis design checkpoint (`.dcp`) containing the RM's netlist in isolation. This approach enables all four RM synthesis runs to execute in parallel on a multi-core machine and decouples each RM's synthesis from the static partition synthesis, significantly reducing total synthesis time compared to a monolithic run. The OOC netlist for each RM is loaded into its respective child implementation run, where it is placed and routed strictly within the bounds of the `cfu_compute` Pblock.

DFX implementation run configuration. A single parent implementation run is responsible for synthesising and implementing the static partition. This run produces a locked design checkpoint — the *static DCP* — that contains the fully placed-and-routed static partition with the RP boundary left as a set of unconnected port stubs. Each child implementation run then imports the static DCP as its starting point, loads the corresponding RM's OOC netlist into the RP boundary stub, and performs place-and-route for the RM exclusively within the Pblock. The child run subsequently generates two output products: a full bitstream (static region plus this RM) and a partial bitstream (RM changes only). The partial bitstream is the artefact delivered to the ICAP during a runtime reconfiguration event.

Pblock constraint iterative refinement. The Pblock for `cfu_compute` was defined in the `.xdc` constraints file using `add_cells_to_pblock`, `resize_pblock`, and `set_property CONTAIN_ROUTING true`. The `CONTAIN_ROUTING` property is a DFX-mandatory constraint that instructs the router to prevent any signal net that originates or terminates within the RP from being routed through resources outside the Pblock's bounding box. This containment guarantees that the partial bitstream's frame address range fully captures all routing resources used by the RM, so that a partial bitstream write does not leave any inter-partition routing segment in an indeterminate state.

The Pblock was iteratively enlarged from the minimum bounding box of the smallest RM (`example.v`) until all four RMs could be placed and routed within it with positive timing slack. Three adjustment cycles were required across the four child implementation runs before a stable Pblock configuration was achieved. Each cycle involved examining the Vivado implementation log for placement overflow errors or routing failures in the KWS RM (the most resource-intensive module), widening the Pblock by one or two CLB columns in the congested direction, and re-running all four child implementation runs to verify that the adjustment did not introduce new timing violations on the static-to-RP interface paths.

[Figure placeholder]

Figure 4.1: Vivado Project Manager screenshot showing the DFX implementation run hierarchy — one parent run for the static partition and four child runs (one per RM), with timing closure status indicators for each run.

JTAG-Based Partial Reconfiguration (Validated)

The first milestone was the demonstration of host-driven JTAG partial reconfiguration on the Arty A7-35T. JTAG reconfiguration is the most direct validation path for DPR because it allows bitstreams to be delivered directly from the Vivado Hardware Manager — without requiring any on-board storage or autonomous controller logic — isolating the RP management correctness question from the bitstream delivery engineering problem.

In this configuration, the full static bitstream containing the complete LiteX SoC, `system_wrapper`, `recon_counter`, and the initial `cfu_compute` RM (`example.v`) is first programmed onto the XC7A35T via the Digilent USB-JTAG cable using the Hardware Manager command:

```
program_hw_devices [get_hw_devices xc7a35t_0] \
    -bitfile full_bitstream_example.bit
```

After programming completes, the VexRiscv BIOS boots, loads the CFU Playground firmware from QSPI flash into SRAM, and begins executing. The CFU Playground host UI on the attached workstation confirms that the initial RM is operational by dispatching a set of known-result test vectors — for example, invoking `function_id = 0` (byte-sum) with `inputs_0 = 0x01020304` and verifying that the returned value equals `0x0000000A` (the sum 1+2+3+4). Successful receipt of the expected result confirms both that the DFX Decoupler is in its transparent state and that `example.v` is active in the RP.

Without resetting the processor or interrupting the UART session, a partial bitstream for the second RM (`donut.v`) is then delivered using the `-partial` flag:

```
program_hw_devices [get_hw_devices xc7a35t_0] \
    -bitfile partial_donut.bit \
    -readback_check false
```

The `-partial` flag instructs the JTAG TAP controller to route the bitstream data through the device's configuration port in partial configuration mode, bypassing the full-device erase-and-reload cycle. The XC7A35T's configuration logic identifies the bitstream as partial by examining the frame addresses in the header and writes only the frames whose FAR values fall within the `cfu_compute` Pblock's address range.

Manual DFX Decoupler assertion and release — required to protect the static partition during the frame write — was accomplished using two TCL AXI transactions issued in the Hardware Manager console before and after the `program_hw_devices` call. The `decouple` control register in `dfx_decoupler_0` is memory-mapped to an AXI-Lite address within the `system_wrapper`'s address map; direct register writes using `create_hw_axi_txn` and `run_hw_axi_txn` provide the required isolation control without modifying the VexRiscv firmware.

The functional result was reproducibly demonstrated across twelve JTAG reconfiguration events — three complete round-trips between each pair of the four RMs. After each swap, a fresh set of test vectors was dispatched through the CFU Playground host UI, and the returned values were verified against the expected outputs for the newly loaded RM. In every trial, the expected behaviour change between consecutive RMs was confirmed, and the processor resumed CFU operations without requiring a firmware restart. The test-vector results for all four RMs are tabulated in Chapter 6.

[Figure placeholder]

Figure 4.2: Vivado Hardware Manager screenshot showing partial bitstream programming of the `cfu_compute` RP on the Arty A7-35T, with the ILA waveform panel displaying the `recon_counter` EOS transition immediately following the partial programming command.

ILA Configuration and `recon_counter` Capture

The Integrated Logic Analyser (ILA) core instantiated within `recon_counter.v` was configured with the following parameters to provide sufficient capture depth for a complete reconfiguration event:

ILA Parameter	Value	Rationale
Number of probes	1	Only `counter[31:0]` monitored
Probe 0 width	32 bits	Full counter width
Sample depth	16,384 samples	Captures ~163 µs at 100 MHz
Trigger position	1/16 of buffer	Minimal pre-trigger, maximise post
Trigger condition	`counter[0] == 1`	Fires on first increment from zero

The trigger fires on the first rising clock edge at which the `recon_counter` FSM transitions from `IDLE` to `RECON` — that is, the first cycle at which the counter increments from zero to one, coinciding with the falling edge of EOS. The 16,384-sample depth provides a 163.84 µs observation window at 100 MHz, which comfortably encompasses the maximum anticipated reconfiguration duration given the partial bitstream sizes for the `cfu_compute` Pblock.

After each reconfiguration event — whether JTAG-based or ICAP-based — the ILA capture was uploaded from the device to the Hardware Manager and inspected for the expected waveform signature: counter at zero pre-trigger, monotonically incrementing through the RECON state, and holding its final value after EOS re-asserts. Any departure from this signature — such as counter oscillation, a premature return to zero, or a second fall-and-rise of EOS — would indicate a hardware fault in the reconfiguration sequence. No such anomalies were observed across any of the validated reconfiguration events on the Arty A7-35T.

A secondary diagnostic use of the ILA was to cross-check the EOS timing against the ICAP write word count. By correlating the cycle at which the counter begins incrementing (EOS fall) with the FSM state at that cycle (visible via an additional probe on the FSM state register during simulation), it was confirmed that EOS falls approximately 8–12 cycles after the synchronisation

word (`0xAA995566`) is presented to the ICAP input — consistent with the Artix-7 configuration state machine's documented latency from sync-word detection to FDRI frame write commencement.

ICAP-Based Partial Reconfiguration (Work In Progress)

The intended production reconfiguration path on the Arty A7-35T is a fully autonomous ICAP-based flow in which the entire seven-step reconfiguration sequence described in §4.4.1 is executed by the `system_wrapper` FSM without any involvement from the Vivado Hardware Manager, the host workstation, or any VexRiscv firmware interrupt handler.

The `system_wrapper` FSM was designed and implemented with the following states to manage this sequence:

FSM State	Active Signals	Exit Condition
`IDLE`	`decouple=0`, `CSIB=1`, `RDWRB=1`	`pr_switch` pulse
`DECOUPLE_ASSERT`	`decouple=1`	16-cycle hold elapsed
`ICAP_SYNC`	`decouple=1`, `CSIB=0`, `RDWRB=0`	8 padding + sync word written
`ICAP_WRITE`	`decouple=1`, `CSIB=0`, `RDWRB=0`, streaming BRAM data	All frame words written
`ICAP_TRAILER`	`decouple=1`, delivering CRC, RCRC, DESYNC	Trailer sequence complete
`WAIT_EOS_HIGH`	`decouple=1`, `CSIB=1`, `RDWRB=1`	`eos == 1` from STARTUPE2
`DECOUPLE_RELEASE`	`decouple=0`	1-cycle hold elapsed; → `IDLE`

In the `ICAP_WRITE` state, a sequential address counter indexes into the BRAM ROM holding the partial bitstream word sequence. The ROM is implemented as a synchronous read, single-port BRAM with a 32-bit data width and a depth equal to the partial bitstream word count. The BRAM address counter increments by one on every rising clock edge while the FSM remains in `ICAP_WRITE`, and the data output of the BRAM (one cycle after address change, due to the synchronous read latency) is forwarded directly to the ICAP's `I[31:0]` input port. The word count for `example.v`'s partial bitstream is 89,472 words (349 KB / 4 bytes per word), requiring a BRAM of depth 131,072 (the next power-of-two above 89,472, using 5 RAMB36E1 tiles of 36 Kb each for the data array to be confirmed by Vivado Resource Report).

Progress towards full autonomous operation has been achieved in two concrete respects. First, the complete seven-state FSM has been synthesised and implemented in the static partition of the Arty A7-35T DFX project, with all states exercised in behavioural simulation using the Vivado simulator with the full partial bitstream word sequence as the stimulus. The simulation waveforms confirm that: (a) the FSM sequences correctly through all states in response to a single `pr_switch` pulse; (b) the BRAM address counter increments in lock-step with each ICAP write cycle; (c) the total word count matches the partial bitstream length; and (d) the `decouple` signal is held high throughout the `ICAP_WRITE` and `ICAP_TRAILER` states with no glitches. Second, the ICAP write cycle has been exercised on physical silicon using the BRAM ROM bitstream source, and the ILA confirms that the EOS fall-and-rise signature is produced by the STARTUPE2 primitive in response to the FSM's ICAP write sequence, validating the end-to-end timing of the hardware path from `pr_switch` to RP-active.

The remaining gap is the bitstream source: the BRAM ROM approach fixes the target RM at synthesis time. Replacing this with the QSPI flash or DDR3 fetch path described in §4.7 will make the system fully autonomous and runtime-reprogrammable. The engineering confidence is high that

this replacement will not require changes to the FSM or the ICAP interface — the BRAM read timing model (one-cycle latency from address to data) is compatible with the ICAP's one-word-per-cycle acceptance rate, and the QSPI or DDR path will be designed to match this timing through FIFO buffering or clock-rate matching as described in §4.7.2.

[Figure placeholder]

Figure 4.3: Vivado behavioural simulation waveform showing the `system_wrapper` FSM state register, `decouple`, `CSIB`, `RDWRB`, and `I[31:0]` signals across a complete single-trigger reconfiguration event. States are annotated with text labels at the state register trace.

Engineering Challenges Encountered on the Arty A7-35T

Several engineering challenges were encountered during Phase 1 that directly influenced design decisions and the project's implementation timeline:

DDR3 controller initialisation contention. The LiteX-generated DDR3 MIG (Memory Interface Generator) controller for the Arty A7-35T undergoes a multi-phase hardware calibration sequence immediately after FPGA configuration completes. This calibration — involving ZQ calibration, clock-to-DQ write levelling, read DQ gate training, and read equalization — occupies the DDR bus exclusively for approximately 200–400 ms at 100 MHz depending on the DRAM module characteristics. No valid burst read or write transaction can be completed until the MIG asserts its `init_calib_complete` signal (exposed as a LiteX register at address `0x40000000` in the UART Wishbone address map). Any AXI4 transaction attempted against the DDR controller before `init_calib_complete` is asserted will result in a bus error response on the AXI interconnect.

This constraint is straightforwardly handled in the BRAM ROM implementation — the BRAM is initialised at configuration time and is ready to read on the first clock cycle — but must be explicitly accounted for in the future DDR3 bitstream storage path (§4.7.3). The `system_wrapper` FSM will need to gate the `pr_switch`-triggered reconfiguration path behind an `init_calib_complete` input signal from the LiteX SoC, preventing any reconfiguration attempt from being initiated before DDR3 is stable.

Routing congestion near the RP boundary. The XC7A35T's routing fabric in the region adjacent to the `cfu_compute` Pblock's east boundary — specifically, the switchbox columns immediately to the right of the Pblock's rightmost CLB column, where the DFX Decoupler's isolation register outputs connect to the static partition's LUT fanout nets — exhibited congestion level 4 (on Vivado's 0–8 scale, where level 5 is the first level that typically causes routing failure) during child implementation runs for the KWS module. Three routing strategies were applied in sequence before congestion was resolved: (1) issuing `set_property ROUTE_EFFORT 5 [get_runs impl_kws]` to increase the router's effort level from its default of 3; (2) adding `phys_opt_design -directive AggressiveExplore` after placement to improve the fanout structure of congested driver nets; and (3) widening the Pblock by two CLB columns in the east direction to provide additional routing tracks at the boundary interface. The third strategy was ultimately the decisive one, reducing the congestion level to 2 and achieving timing closure with WNS > 0.5 ns on all inter-partition paths.

QSPI flash address space partitioning. The Arty A7-35T's 128 Mb (16 MB) QSPI flash is shared between two consumers in the LiteX build: the LiteX BIOS image (at offset `0x00000000`,

approximately 64 KB) and the VexRiscv firmware binary (compiled by the CFU Playground build system and programmed to offset `0x00080000`, approximately 256 KB). The DFX project's bitstream storage plan must avoid these regions. Based on the LiteX flash layout for the Arty A7-35T, the safe region for partial bitstream storage begins at offset `0x00400000` (4 MB from start), leaving 12 MB for up to 28 partial bitstreams of 430 KB each — more than sufficient for the four RMs in the current design. The flash programming commands issued via the Vivado Hardware Manager's `program_flash` TCL function were configured with this base offset to ensure no overlap with the firmware region.

Timing closure of the ICAP data path. The combinational bit-reversal logic in the ICAP write data path — which individually reverses each of the four bytes in the 32-bit word output of the BRAM before presenting it to `I[31:0]` — was initially implemented as a flat assign statement with 32 individual bit assignments. Vivado's synthesis tool inferred this as a 32-input LUT network and placed the resulting flip-flop-to-LUT path on a pre-existing high-fanout net in the `system_wrapper`, causing a setup violation of 0.32 ns on the ICAP data output path at 100 MHz. The violation was resolved by pipelining the bit reversal: the BRAM output is registered into an intermediate flip-flop, the bit reversal is applied combinationally to the registered intermediate value, and the result is then registered again into the ICAP output register. This two-register pipeline introduces a two-cycle latency between the BRAM address and the corresponding word appearing at the ICAP — accounted for in the FSM's word counter by subtracting two from the address counter start value — and eliminates the timing violation by breaking the long combinational path into two manageable register-to-register segments.

4.5.2 Phase 2 — PYNQ-Z2 (Zynq-7020, Standalone Validation)

The Digilent PYNQ-Z2 was introduced as a secondary evaluation platform after the project's scope expanded to include an investigation of Processing-System-assisted (PS-assisted) reconfiguration pathways. The PYNQ-Z2 is based on the Xilinx Zynq-7020 All-Programmable SoC, which integrates a dual-core ARM Cortex-A9 Processing System (PS) clocked at up to 866 MHz with a Programmable Logic (PL) fabric whose capacity is equivalent to that of the XC7Z020 standalone device — approximately 53,200 six-input LUTs, 106,400 flip-flops, 140 BRAM tiles of 36 Kb each, and 220 DSP48E1 slices. The PYNQ-Z2 board provides 512 MB of DDR3L SDRAM accessible from both the PS and PL, a microSD card slot for Linux boot, two PMOD connectors, a 10/100 Ethernet port, and a single USB connector shared between JTAG and UART functions.

The Zynq architecture introduces a reconfiguration capability that does not exist on the pure-PL Arty A7-35T: the *Processor Configuration Access Port* (PCAP). PCAP is a dedicated hardware interface within the Zynq's Device Configuration (`DEVCFG`) peripheral that enables the ARM PS to initiate full or partial PL reconfiguration directly from PS DRAM — without any external JTAG host connection. This PS-native configuration path is the primary motivation for including the PYNQ-Z2 in the project's evaluation scope: it provides, in a single device, a PS-side analogue to the PL-autonomous ICAP-based flow being developed on the Arty A7-35T, allowing a direct architectural comparison between processor-driven PCAP (PS-side) and fabric-driven ICAP (PL-side) reconfiguration approaches.

Vivado DFX Flow for Zynq-7020

The Vivado DFX flow for the PYNQ-Z2 differs from the Arty A7-35T flow in several Zynq-specific respects:

Processing System block design. The Zynq PS must be instantiated as a Vivado IP block design component (`processing_system7_0`) and configured with the PYNQ-Z2's board-specific PS I/O settings: DDR3L-400 for the 16-bit 512 MB DRAM interface, UART1 on MIO pins 48/49 for the console, ENET0 on MIO 16–27 for Ethernet, SD0 on MIO 40–47 for the microSD, and the USB0 OTG controller for UART/JTAG via the FT4232H bridge. The PS block design is exported as an XDC constraint file that locks all dedicated PS I/O pins, which the DFX implementation flow must preserve unmodified between the parent run and all child runs.

FCLK-based clock provisioning. On the PYNQ-Z2, the PL's primary clock source is the `FCLK_CLK0` output of the PS block. The PS's on-chip PLL is configured (via the `PCW_FPGA0_PERIPHERAL_FREQMHZ` parameter in the PS block design) to generate 100 MHz on `FCLK_CLK0`. This clock is routed from the PS block's output port through the block design's clock routing to the PL clock tree, and all DFX timing constraints are defined against this 100 MHz domain. Unlike the Arty's external crystal-to-MMCM path, the Zynq's PS PLL has a specified SSMC jitter specification that is guaranteed within the timing constraints of the DFX Decoupler IP, so no additional jitter margin adjustments are required.

PL-to-PS AXI connection for Decoupler control. On the Arty A7-35T, the `dfx_decoupler_0` AXI-Lite control register is accessed from the Hardware Manager console via direct AXI transactions. On the PYNQ-Z2, the same AXI-Lite port is connected to the PS through a Zynq AXI General Purpose (GP0) master port, making the Decoupler's control register directly accessible from PS software via memory-mapped I/O at a user-assigned base address. This connection enables the Python-based DPR management script (described in §4.5.2.3) to assert and release the `decouple` signal programmatically without requiring any TCL console intervention.

DFX partition porting. The `cfu_compute` RP, all four RMs, and the `cfu.v` wrapper were ported from the Arty project without modification to the source Verilog. Only the Pblock constraints and the package pin assignments were updated for the Zynq-7020's fabric geometry, which differs from the Artix-7 in column layout, the location of the PS hard macro region (lower half of the device), and the position of the DDR PHY in the lower-right corner. The Pblock was placed in the upper-right quadrant of the PL fabric, well separated from the PS hard blocks, the DDR PHY, and the fixed I/O tiles.

JTAG-Based Partial Reconfiguration on PYNQ-Z2 (Validated)

JTAG-based partial reconfiguration on the PYNQ-Z2 was validated using the identical Hardware Manager methodology as on the Arty A7-35T. However, the Zynq-7020 DFX JTAG flow imposes one additional requirement relative to the Artix-7: a mandatory two-phase programming sequence.

Because the Zynq PS must boot and complete its DDR calibration sequence before the PL's clocking infrastructure is stable, the full bitstream must be programmed via JTAG first, and a delay of several seconds must be observed before the ILA probe connections are established. If the Hardware Manager attempts to arm the ILA immediately after `program_hw_devices` completes — a sequence that is valid on the Artix-7, where the PL is fully operational as soon as the full bitstream configuration finishes — the ILA core fails to lock to the PL clock, and the Hardware Manager reports either "ILA core not responding" or "JTAG clock lost" depending on the exact timing. The correct sequence, confirmed empirically and documented in UG984 (Zynq-7000 DFX Application Notes), is:

1. Programme the full bitstream with `program_hw_devices`.
2. Wait for the PS to complete booting (observed via UART console output reaching the Linux login prompt, approximately 8–12 seconds from power-on depending on the microSD card

speed).

3. Refresh the hardware target (`refresh_hw_target`) to re-enumerate the ILA debug cores.
4. Arm the ILA trigger and confirm lock.
5. Proceed with partial bitstream delivery.

The larger PL fabric of the Zynq-7020 relative to the XC7A100T substantially simplified the floorplanning process: the Zynq-7020's PL region available to user logic (after reserving the PS hard block area and DDR PHY) contains approximately 50,000 LUTs in the upper half of the device. Placing the `cfu_compute` Pblock in this region with generous margins resulted in all four RMs achieving placement and routing closure on the first Pblock specification attempt — without any of the iterative boundary-expansion adjustments required on the Arty.

All twelve JTAG reconfiguration round-trips validated on the Arty A7-35T were repeated on the PYNQ-Z2, with consistent results: each RM swap was confirmed by UART-based test vector execution, and each ILA capture showed the expected monotonically-incrementing `recon_counter` waveform with a clean EOS fall-and-rise signature.

[Figure placeholder]

Figure 4.4: Vivado device floorplan view for the PYNQ-Z2 DFX implementation, showing the static partition (shaded blue), the `cfu_compute` Pblock boundary (red rectangle in the upper-right quadrant), and the placed RM logic (green dots) within the Pblock.

PCAP-Based Partial Reconfiguration (Validated)

PCAP-based reconfiguration was validated by executing a complete, software-triggered partial reconfiguration sequence from within a running PetaLinux environment on the Zynq ARM PS. This constitutes the most significant unique capability demonstrated on the PYNQ-Z2, as it achieves autonomous processor-driven DPR — analogous in capability to the ICAP-based autonomous flow on the Arty — using only resources that are part of the standard Zynq PS without any additional fabric logic beyond the DFX Decoupler.

Boot environment. The PYNQ-Z2 was booted from a microSD card loaded with the PYNQ v2.7 image, providing a PetaLinux environment with Linux kernel 5.15, Python 3.8, and the PYNQ 2.7 Python library. The PYNQ library's `MMIO` class was used to provide memory-mapped access to the `dfx_decoupler_0` AXI-Lite control register from Python user space, and the PYNQ library's `Bitstream` class was used to wrap the lower-level FPGA Manager sysfs interface with a Python-callable API.

Device Tree Overlay (DTO) registration. Before the FPGA Manager can accept partial reconfiguration requests for the `cfu_compute` region, the Linux kernel's live device tree must be extended with a DTO that registers the RP. The DTO source (a `.dts` file) specifies: (a) the RP's name (`cfu_compute_region`), (b) its AXI base address and size (for RMs that expose AXI-addressable registers — not applicable in the current implementation, but included for generality), (c) the label linking it to the parent full-bitstream overlay, and (d) a `firmware-name` property pointing to the default RM's `.bin` file in `/lib/firmware/`. The DTO is compiled to a binary `.dtbo` file using the Device Tree Compiler (`dtc`) and applied at runtime via the configfs overlay interface:

```
mkdir /sys/kernel/config/device-tree/overlays/cfu_compute
cp cfu_compute_region.dtbo \
   /sys/kernel/config/device-tree/overlays/cfu_compute/dtbo
```

Successful application of the DTO is confirmed by the presence of the `/sys/class/fpga_region/` directory entry for `cfu_compute_region` and by the absence of kernel error messages in `dmesg`.

Bitstream format conversion. The Vivado DFX flow produces partial bitstreams in the `.bit` file format, which includes a header consisting of design metadata (device part, date, time, and design name) followed by the raw frame data. The Linux FPGA Manager requires bitstreams in raw binary (`.bin`) format — that is, the `.bit` file with the header stripped and the frame data byte-swapped to correct the bit ordering for the Zynq PCAP interface. The conversion is performed using the `bootgen` utility (provided by the Xilinx PetaLinux SDK):

```
bootgen -image partial_donut.bif -arch zynq -process_bitstream bin
```

where the `.bif` file specifies the partial bitstream as the input. The resulting `.bin` file is copied to `/lib/firmware/` on the PYNQ-Z2's filesystem before reconfiguration is attempted.

DFX Decoupler assertion. Before initiating the PCAP transfer, the Python DPR management script asserts the `decouple` signal by writing to the `dfx_decoupler_0` AXI-Lite control register via the PYNQ MMIO interface:

```
from pynq import MMIO

# dfx_decoupler_0 AXI-Lite base address from block design address map.
DECOUPLER_BASE = 0x43C00000
decoupler = MMIO(DECOUPLER_BASE, length=0x1000)

# Assert decouple (bit 0 = 1 in the DECOUPLE register at offset 0x0).
decoupler.write(0x0, 0x1)
```

After the write, a 160 ns settling period is allowed by issuing a `time.sleep(0.001)` call (1 ms, far in excess of the required settling time but straightforward to implement without a hardware timer), before the PCAP transfer is initiated.

PCAP transfer via sysfs. The partial bitstream transfer is initiated by writing the target `.bin` filename to the FPGA Manager's `firmware` attribute:

```
echo "partial_rm_donut.bin" > /sys/class/fpga_manager/fpga0/firmware
```

The Linux FPGA Manager driver (`fpga-mgr`, `drivers/fpga/fpga-mgr.c`) intercepts this write and invokes the Zynq-specific backend (`drivers/fpga/zynq-fpga.c`). The backend performs the following operations on the `DEVCFG` peripheral at physical address `0xF8007000`:

1. Sets the `PROG_B` bit in `DEVCFG.CTRL` to assert configuration logic reset.
2. Clears and re-sets `PCAP_PR` to configure the PCAP in partial reconfiguration mode.
3. Programmes the DMA source address (`DMA_SRC_ADDR`) to the physical address of the kernel DMA buffer containing the `.bin` file data.
4. Programmes the transfer byte count into `DMA_DEST_ADDR` (repurposed as transfer length in PCAP mode).

5. Sets the `PCFG_PROG_B` bit in `DEVCFG.CTRL` to assert the FPGA's `PROG_B` line and initiate the configuration.
6. Polls `INT_STS` for the `PCFG_DONE_INT` bit, which the hardware asserts after the DMA engine confirms all bytes have been presented to the PCAP interface and accepted by the PL configuration logic.

DFX Decoupler release. Upon confirmation of the `PCFG_DONE_INT` flag (which the Python script detects by polling the FPGA Manager's `state` sysfs attribute until it reads `operating`), the `decouple` signal is de-asserted:

```
decoupler.write(0x0, 0x0)  # Decouple release
```

This sequence is functionally equivalent to Step 7 of the `system_wrapper` FSM described in §4.4.1, with the only difference being that the FSM uses the EOS signal from STARTUPE2 as its release condition, whereas the Python script uses the FPGA Manager's software state machine. In practice, both release conditions are triggered by the same underlying hardware event — the PL configuration logic asserting `PCFG_DONE_INT` after the startup sequence completes — so the timing is equivalent.

Transfer performance characterisation. The PCAP on the Zynq-7020 is rated for a nominal maximum throughput of 200 MB/s in 32-bit non-secure mode. In the PetaLinux 2024.2 kernel used in this project, the Zynq FPGA Manager backend programmes the PCAP in non-secure mode without AXI burst optimisation, yielding a measured DRAM-to-configuration-logic throughput of approximately 180 MB/s, as determined by dividing the partial bitstream size by the measured time between the `firmware` write and the `PCFG_DONE_INT` poll returning success. For the `cfu_compute` partial bitstream (~380 KB in Zynq binary format), the transfer completes in approximately 2.1 ms. Adding the startup sequence overhead measured by `recon_counter` (comparable to the Arty A7-35T value, since both devices use the same 7-series configuration startup FSM), the total time from `firmware` write to RP-ready is approximately 3.2–3.5 ms.

Functional validation. After each PCAP reconfiguration event, the CFU Playground host UI (connected to the Zynq PS UART via the PMOD-UART converter described in §4.5.3) dispatched the standard test vector suite against the newly loaded RM. Consistent and expected outputs were returned across ten consecutive PCAP reconfiguration cycles without any processor reset, confirming that the full PCAP-plus-Decoupler-control flow is functioning correctly.

[Figure placeholder]

Figure 4.5: Software flow diagram of the PCAP reconfiguration sequence showing the Python management script, the PYNQ MMIO/FPGA Manager API, the Linux kernel FPGA Manager driver, the Zynq DEVCFG peripheral, and the PL configuration logic, with data-flow arrows and timing annotations.

ICAP on Zynq-7020: Architectural Constraint and Investigation

Although the Zynq-7020 PL fabric contains the `ICAP_7SERIES` hard macro at the standard 7-series device location (lower-left quadrant of the PL, outside any user-assignable resource), accessing this primitive from PL fabric logic on a Zynq device requires coordinating with the PS's PCAP arbitration hardware — a constraint that does not exist on the Arty A7-35T.

The Zynq-7000 Series TRM (UG585, §6.3.4) states explicitly that PCAP and ICAP cannot be used simultaneously. The arbitration is controlled by the `PCAP_MODE` bit in the `DEVCFG.CTRL` register at `0xF8007000`. When `PCAP_MODE = 1` (the default after PS boot, as set by the FSBL), PCAP holds exclusive ownership of the configuration memory, and any ICAP write attempt produces undefined behaviour. To enable ICAP from RD logic, `PCAP_MODE` must first be cleared to 0, which disables PCAP and surrenders configuration memory access to the ICAP interface.

A systematic investigation of PL-side ICAP access on the PYNQ-Z2 was conducted with the following observations:

Step 1 — Disabling PCAP mode. The `DEVCFG.CTRL` register was accessed from user space via `/dev/mem`. The `PCAP_MODE` bit (bit 26) was cleared while the `PCAP_PR` bit (bit 27, partial reconfiguration enable) was preserved at 0. This was confirmed by a subsequent read-back that verified the bit was cleared. Attempting a sysfs-triggered `firmware` write after this bit was cleared confirmed that PCAP was effectively disabled: the FPGA Manager command returned an error (`-EBUSY` from the `zynq-fpga.c` backend after timeout waiting for `PCFG_DONE_INT`).

Step 2 — PL ICAP write attempt. With `PCAP_MODE = 0`, the `system_wrapper` FSM on the Zynq PL (ported identically from the Arty implementation) was triggered via the `pr_switch` push-button. The `recon_counter` ILA captured an EOS falling edge followed by a rising edge approximately 37,000 cycles later, suggesting that the configuration state machine did begin processing the ICAP frame data.

Step 3 — Functional validation. The CFU Playground test vector suite was dispatched immediately after the EOS rising edge was confirmed. In 6 out of 10 trials, the expected outputs of the new RM were returned, indicating a successful reconfiguration. In the remaining 4 trials, the outputs of the old RM were returned or garbage values were produced, indicating an inconsistent reconfiguration outcome.

Root cause analysis. The inconsistency was tentatively attributed to internal PCAP arbitration behaviour within the Zynq's configuration subsystem. Xilinx Answer Record AR#51661 documents a silicon errata in certain Zynq-7020 die revisions wherein the internal PCAP state machine re-asserts a frame-erase cycle during or shortly after an ICAP write sequence if the PS's boot-mode register retains its PCAP-enable setting. The erased frames partially overwrite the newly written RM data, resulting in a subset of frames retaining old RM content — manifesting as the 40% failure rate observed. The errata document notes that the reliable workaround is to perform a full PS software reset and re-initialisation of the `DEVCFG` peripheral before each ICAP write, which is impractical for a real-time reconfiguration controller.

Given these findings, the PCAP path was confirmed as the correct and supported configuration interface for the PYNQ-Z2, and the ICAP investigation was not pursued further. The Arty A7-35T — which has no PCAP and therefore no PCAP arbitration logic — remains the platform of choice for validating PL-autonomous ICAP-based DPR.

4.5.3 Phase 3 — Arty A7 + PYNQ-Z2 Inter-Board Bridge

Phase 3 arose from a practical connectivity problem encountered when attempting to run both hardware platforms simultaneously with their full feature sets — specifically, with the JTAG ILA session on the PYNQ-Z2 active at the same time as the CFU Playground UART session. Understanding the origin of the conflict requires a detailed description of the PYNQ-Z2's USB connectivity architecture.

PYNQ-Z2 USB Architecture and the Source of the Conflict

The PYNQ-Z2 provides a single physical USB Type-A connector to the host workstation. This connector is serviced by an FTDI FT4232H USB-to-Quad-UART bridge IC (a four-channel USB device). On the PYNQ-Z2, the FT4232H's channels are assigned by the board's factory firmware as follows:

FT4232H Channel	Interface	Windows Virtual COM
Channel A (ADBUS)	JTAG TAP (TDI/TDO/TCK-/TMS)	`COMx` (Digilent JTAG)
Channel B (BDBUS)	UART TX/RX	`COMy` (Serial Console)
Channel C (CDBUS)	SPI Debug	Not enumerated by default
Channel D (DDBUS)	Reserved	Not enumerated by default

Under normal single-board operation, Channels A and B appear to the host operating system as two independent virtual COM ports within the same USB composite device. The Vivado Hardware Manager attaches to Channel A for JTAG, while a serial terminal or the CFU Playground Python host opens Channel B for UART. This coexistence works because the FT4232H's four channels present as independent USB endpoint pairs within a USB 2.0 composite device, and the Windows WinUSB driver processes them through separate kernel device handles.

The conflict arose specifically when the ILA was required to maintain a *persistent, real-time streaming* JTAG session — an active ILA run-time (RT) capture in which waveform data is continuously transferred from the FPGA to the Hardware Manager. In this mode, the Digilent JTAG WinUSB driver holds an exclusive open handle on the underlying USB device object (not merely on Channel A's endpoint, but on the composite USB device descriptor as a whole) to prevent concurrent access to the shared USB bandwidth scheduler. A second process — in this case, the Windows COM port driver attempting to open `COMy` (Channel B) — requests access to the same USB device object through a separate kernel driver stack, and the USB stack serialises these concurrent requests, causing intermittent `STATUS_ACCESS_DENIED` responses when the UART COM port open operation races with the active ILA session's USB bandwidth reservation.

This conflict was reproducible across three different Windows 10 host configurations (different PC hardware, different USB host controller chipsets), confirming that it is not a driver installation artefact but a fundamental consequence of the Windows USB `WdfUsbTargetDeviceCreate` exclusive-open semantics used by the Digilent JTAG driver.

The following software-only remedies were considered and rejected:

- **USB hub insertion:** Adding a USB hub between the host and the PYNQ-Z2 was confirmed to change the enumeration path but does not resolve the kernel-level exclusive-open conflict, as the conflict is within the WinUSB driver stack rather than at the hub arbitration level.
- **Xilinx Virtual Cable (XVC) over Ethernet for ILA:** XVC tunnels JTAG transactions over TCP/IP, which would eliminate the USB conflict by substituting an Ethernet path for the JTAG channel. However, the PYNQ-Z2's Ethernet port shares the PS's network stack, which conflicts with the Linux PetaLinux environment's standard network services when the XVC server (`hw_server`) is run as a system service. Configuration of this service in a stable non-conflicting mode was estimated to require significant PetaLinux kernel build effort, which was out of scope.
- **Modified CFU Playground Python host:** Adapting the CFU Playground's host Python

script to tolerate intermittent UART COM port access failures was considered but rejected, as it would introduce timing non-determinism into the test vector execution loop, making the latency measurements unreliable.

The selected solution was therefore a hardware-level remedy: routing the PYNQ-Z2's UART signal out through a completely independent physical interface — the PMOD connector — to reach the host via a separate USB cable, entirely bypassing the FT4232H and its associated driver conflict.

UART Routing via PMOD: Signal Assignment

The PYNQ-Z2's PMOD JA connector (a 12-pin Digilent-standard header on the board's east edge) is connected to Zynq PL I/O Bank 35, which provides eight user GPIO pins alongside VCC (3.3 V) and GND. PMOD pins are mapped to the following FPGA package pins: JA1 → `Y18`, JA2 → `GW18`, JA3 → `W18`, JA4 → `V18`, JA7 → `V16`, JA8 → `U16`, JA9 → `V17`, JA10 → `U17`.

Within the Zynq PS block design, the PS UART1 peripheral (on MIO 48/49) was extended through the PS EMIO interface to the PL fabric, exposing UART1 TX and RX as PL-level signals (`UART1_TX_0` and `UART1_RX_0`). These signals were then assigned to PMOD JA pins 1 and 2 respectively using the following `.xdc` constraints added to the DFX project:

```
set_property PACKAGE_PIN Y18  [get_ports {UART1_TX_0}]
set_property IOSTANDARD  LVCMOS33 [get_ports {UART1_TX_0}]

set_property PACKAGE_PIN GW18 [get_ports {UART1_RX_0}]
set_property IOSTANDARD  LVCMOS33 [get_ports {UART1_RX_0}]
```

This assignment was validated with a simple loopback test: `UART1_TX_0` and `UART1_RX_0` were connected externally by a single wire, and the PS UART1 was programmed (via a baremetal test application running under Xilinx SDK) to transmit a 256-byte incrementing sequence and verify that the received bytes matched. The loopback test passed on the first attempt, confirming that the EMIO routing and package pin assignment were correct before any inter-board wiring was introduced.

Attempted Workaround: PMOD-to-Arty Direct Wire UART Bridge

With the PYNQ-Z2's UART1 correctly routed out through PMOD JA, the first inter-board connectivity attempt involved connecting the PYNQ-Z2's JA1 (TX) and JA2 (RX) pins directly via 28 AWG hookup wire to the JA1 and JA2 pins of the Arty A7-35T's PMOD JA connector. The Arty board was simultaneously programmed with a minimal pass-through bitstream — described in the following sub-section — intended to bridge the PMOD signals to the Arty's on-board USB-UART (accessible to the host via a separate USB cable from the PYNQ-Z2's USB connection).

The pass-through bitstream comprised only a Verilog top-level module with two `assign` statements:

```
module arty_uart_bridge (
    input  uart_rxd_out,   // Arty board USB-UART TX net (host     Arty
   FTDI     FPGA)
    output uart_txd_in,    // Arty board USB-UART RX net (FPGA     Arty
   FTDI     host)
    input  ja_p1,          // PMOD JA pin 1: Z2 UART1_TX signal
    output ja_p2           // PMOD JA pin 2: Z2 UART1_RX signal
);
    assign uart_txd_in = ja_p1;        // Z2 TX     host (via Arty FTDI
   RX)
```

```
    assign ja_p2        = uart_rxd_out; // host      Z2 RX (via Arty FTDI
   TX)
endmodule
```

No logic cells are instantiated beyond the four bidirectional I/O buffers automatically inferred by Vivado's synthesis engine for the module's ports. The resulting bitstream occupied zero fabric resources and synthesised in under thirty seconds.

Despite the conceptual simplicity of this approach, it failed to produce reliable UART communication at the intended operating baud rate of 115,200 bps. The following paragraphs describe the failure analysis in detail.

[Figure placeholder]

Figure 4.6: Wiring diagram of the direct PMOD-to-PMOD UART bridge, showing the signal path from PYNQ-Z2 UART1_TX (JA1) through hookup wire to Arty JA1, through the FPGA fabric (wire assignment), and out to the Arty's USB-UART FTDI TX input.

Failure Analysis: Signal Integrity of the Direct Wire Bridge

Observed symptoms. The CFU Playground Python host UI, opened on `COMy` (the Arty's USB-UART virtual COM port), consistently reported framing errors when the PYNQ-Z2 transmitted at 115,200 bps. A manual ASCII echo test using a terminal emulator confirmed that the majority of transmitted bytes were received as incorrect values; only occasionally was a correct byte received. Stepping down to 38,400 bps reduced errors significantly but did not eliminate them; at 9,600 bps, communication appeared intermittently functional but with occasional framing errors and long silence periods.

Oscilloscope measurements at the source (PYNQ-Z2 JA1).

- Idle-high (UART mark state) voltage: **+3.301 V** (nominal 3.3 V LVCMOS33, consistent with the PYNQ-Z2's 3.3 V I/O supply on Bank 35).
- Logic-low (UART space state) voltage: **+42 mV** (clean logic low, driver strength adequate).
- Rise time from low to high: **4.8 ns** (consistent with LVCMOS33 output at 12 mA drive strength and the nominal on-die slew rate setting).

Oscilloscope measurements at the far end (Arty JA1 pin) at 115,200 bps.

- Idle-high voltage: **+3.048 V** (voltage droop of ~253 mV from source; expected due to resistive loss in 28 AWG wire and contact resistance at PMOD headers).
- Peak voltage on rising edges: **+3.617 V** (overshoot of ~316 mV above the idle voltage, consistent with transmission line ringing on unmatched interconnect).
- Time to settle to within ±50 mV of final value after each transition: **~3.1 µs**.

Bit period analysis at 115,200 bps. At 115,200 bps, each symbol occupies a period of $1/115200 \approx 8.68\ \mu$s. The settling time of 3.1 µs consumed 35.7% of the bit period before the signal was logically stable. For consecutive transitions (e.g., the ten-bit UART frame 0b0{data}1 includes multi-

ple adjacent start, data, and stop bit transitions), the accumulated settling tails from consecutive opposite-polarity transitions overlapped, producing eye closure at the Arty's PMOD input. With an eye opening of less than 5.6 µs (64% of one symbol period) and the input buffer's sampling point located near the centre of the bit period, metastability-induced sampling errors were frequent.

Root causes. Three concurrent factors were identified as contributing to the failure:

1. *Capacitive loading.* The 28 AWG hookup wire over approximately 30 cm introduces approximately 33 pF of additional capacitance (110 pF/m × 0.3 m) on the signal node. Combined with the Arty PMOD input's estimated 5–10 pF input capacitance and the PQFP package's pin capacitance, the total capacitive load on the PYNQ-Z2's LVCMOS33 driver was approximately 45–50 pF. At a 12 mA drive current, the worst-case charging rate is approximately 0.24 V/ns for the final 3.3 V swing, consistent with the measured 4.8 ns rise time at the source. However, the wire's distributed inductance (approximately 240 nH/m × 0.3 m = 72 nH) creates an LC resonant circuit with the load capacitance, producing the observed overshoot-and-ring behaviour.
2. *Impedance mismatch.* Neither board provides a PMOD-integrated termination scheme. The signal path from the PYNQ-Z2's FPGA output pad through the wire to the Arty's FPGA input pad constitutes an unterminated, unshielded transmission line. The propagation delay is approximately 1.5 ns for a 30 cm air-dielectric wire; the round-trip delay of 3 ns is below the rise time of 4.8 ns, so the structure is quasi-lumped at this frequency, but the reflected wave from the mismatched Arty input impedance (50 in-buffer, high-Z to the external wire) nonetheless produces a reflection coefficient of approximately 0.94, generating the observed overshoot ring.
3. *No Schmitt trigger on the Arty PMOD input.* The XC7A100T's standard LVCMOS33 I/O buffer does not include a Schmitt trigger by default. The FPGA I/O standard selection in the Arty `.xdc` file was not modified to enable the `LVTTL` or `SSTL` standards, which differ in input threshold specification but similarly do not provide Schmitt hysteresis. The absence of hysteresis means that the ringing waveform's oscillation around the 1.65 V input threshold (approximately ±0.5 V peak-to-peak during the ringing period) produces multiple spurious input transitions per intended edge, corrupting the UART framing logic in the Arty's pass-through path.

[Figure placeholder]

Figure 4.7: Annotated oscilloscope trace at 115,200 bps on the direct PMOD wire. Upper trace: PYNQ-Z2 JA1 (source). Lower trace: Arty JA1 (far end). Annotations show the 253 mV voltage droop, 3.1 µs settling time, and 316 mV overshoot on the rising edge.

Resolution: Commercial PMOD-to-USB-UART Converter

The connectivity problem was resolved through the procurement of a commercial PMOD-compatible USB-UART converter module based on the WCH CH340G USB-UART bridge IC. The CH340G addresses each of the three identified failure modes:

Buffered UART interface. The CH340G's UART RX input incorporates an internal Schmitt trigger with a hysteresis band of approximately 0.4–0.5 V centred on 1.65 V (the midpoint of the 3.3

V LVCMOS33 logic range). This hysteresis provides a noise immunity margin of ±250 mV around the threshold — approximately five times the ±50 mV margin of a standard CMOS buffer. The ringing amplitude of ±316 mV observed on the direct wire would have caused multiple spurious transitions in a standard CMOS buffer but falls within the CH340G's hysteresis band for most of the ringing period, preventing spurious transitions.

Defined output drive for bidirectional UART. The CH340G's TX output is a push-pull CMOS driver rated for 12 mA at 3.3 V with a specified rise time of less than 50 ns into a 50 pF load — well matched to the short, low-capacitance PMOD cable connecting the module to the board. The TX signal presented to the PYNQ-Z2's JA2 (RX) input arrives with clean, well-defined edges and no ringing, as the module is positioned directly adjacent to the PYNQ-Z2's PMOD connector without intermediate wire length.

Independent USB endpoint. The CH340G presents to the host workstation as a completely separate USB device descriptor (USB VID `0x1A86`, PID `0x7523`) from the PYNQ-Z2's FT4232H (USB VID `0x0403`, PID `0x6011`). The Windows COM port driver (`usbser.sys`) opens the CH340G as an entirely separate device node, with no shared kernel device handle with the Digilent WinUSB driver managing the FT4232H. This eliminates the exclusive-open conflict that originally motivated the PMOD-based routing approach.

Physical installation. The CH340G PMOD module's 6-pin female header (conforming to the Digilent PMOD Type 2 (UART) specification) plugs directly onto pins 1–6 of the PYNQ-Z2's PMOD JA connector, providing +3.3 V power (JA pin 6), GND (JA pin 5), RX (JA pin 2, connected to the Z2's UART1_TX), and TX (JA pin 1, connected to the Z2's UART1_RX). The module's USB-A plug connects to any available USB port on the host workstation via a standard USB cable.

Validation results. With the CH340G module installed, the CFU Playground Python host UI was configured to open `COMz` (the CH340G virtual COM port) at 1,000,000 bps (1 Mbps, the maximum sustained rate supported by the CFU Playground's firmware UART driver). A full test vector suite was executed across ten consecutive PCAP reconfiguration events, with the Vivado Hardware Manager concurrently maintaining an active ILA RT capture session on the PYNQ-Z2 via the FT4232H JTAG channel. No UART framing errors were reported at any point during the test, and the ILA captures proceeded without interruption. This outcome confirms that the CH340G-based PMOD UART path is both electrically reliable at the target baud rate and free from contention with the concurrent JTAG session.

[Figure placeholder]

Figure 4.8: Photograph of the fully assembled test bench showing the PYNQ-Z2 board with the CH340G PMOD-UART module plugged into PMOD JA, two USB cables connecting to the host workstation (one from the PYNQ-Z2 on-board FT4232H, one from the CH340G), and the Arty A7-35T board with its own USB cable for JTAG.

Summary of Phase 3 Outcomes and Final Hardware Configuration

The inter-board connectivity challenge encountered in Phase 3 produced, as its resolution, a hardware test bench configuration that is substantially more robust and diagnostically transparent than the original single-USB-cable setup would have been. By separating the JTAG and UART channels onto

physically independent USB paths — each with its own dedicated controller IC, enumeration identity, and kernel driver stack — any future electrical interference or driver-level conflict between the two channels is structurally impossible regardless of the operating system's USB scheduling behaviour.

The final hardware test bench used for all results reported in Chapter 6 consists of the following physical configuration:

Connection	Interface	Host COM Port
PYNQ-Z2 FT4232H → Host USB	JTAG (Channel A) + Secondary UART (Channel B)	`COMx` (Hardware Manager)
PYNQ-Z2 PMOD JA → CH340G → Host USB	UART (CFU Playground)	`COMz` (CFU Python host)
Arty A7-35T FT2232H → Host USB	JTAG (Channel A) + UART (Channel B)	`COMa` / `COMb`

With this configuration, the following simultaneous operations are possible without conflict: (a) the Vivado Hardware Manager maintains an active ILA RT capture on both boards; (b) the CFU Playground Python host communicates with the PYNQ-Z2 firmware via the CH340G UART; and (c) partial bitstreams are delivered to either board via JTAG or PCAP as required by the test procedure.

4.6 DFX Tutorial Baseline Validation

Prior to integrating the DFX Decoupler with the CFU Playground SoC — a complex multi-module DFX design — the AMD UG947 *Dynamic Function eXchange Tutorial* was reproduced in full on the Arty A7-100T as an independent validation baseline. This step serves a dual purpose: it confirms that the Vivado DFX toolflow is correctly installed and licensed on the development machine, and it provides a known-good reference point against which the resource utilisation and timing results of the final system can be compared.

4.6.1 Tutorial Design Summary

The UG947 tutorial instantiates a minimal DFX design consisting of two components:

- **Static partition:** A 3-bit LED shift register that continuously shifts a single illuminated LED across the four user LEDs at a user-configurable rate. This logic is synthesised once and embedded in the full bitstream and all partial bitstreams as the persistent static region.
- **Reconfigurable partition:** A `count_shift_led` module, for which two RMs are provided — a binary counter variant and a shift-register variant — each implementing the same port interface but producing distinct LED patterns. The RP is a single Pblock defined over a small area of the XC7A100T fabric.

The tutorial walks through the complete DFX implementation flow: defining the RP and RMs in the Vivado project settings, running out-of-context (OOC) synthesis for each RM, performing the DFX implementation of the full design in Vivado, and generating both the full bitstream and the per-RM partial bitstreams. JTAG-based reconfiguration is then demonstrated using the Hardware Manager, with the LED pattern changing visibly between the counter and shift-register behaviours after each partial bitstream load.

4.6.2 Measured Baseline Results

The implementation of the tutorial design on the Arty A7-100T yielded the following resource utilisation figures, obtained from the Vivado Implementation Report:

Resource	Used	Available (XC7A100T)	Utilisation
Slice LUTs	10	63,400	<1%
Slice FFs	64	126,800	<1%
Block RAMs (36K)	1	135	<1%
DSP48E1	0	240	0%
Pblock area (slices)	21	—	—

Timing summary from the post-implementation timing report:

- Worst Negative Slack (setup): **+2.022 ns** (timing closed with margin)
- Worst Negative Slack (hold): **+0.109 ns** (all hold paths met)
- No I/O timing constraints violated

The successful closure of this baseline confirmed that the Vivado DFX flow on the project's exact tool version (Vivado 2024.2), board support package, and constraint file was operating correctly. It also established the overhead of the tutorial's static logic as a reference: 10 LUTs, 64 FFs, and 1 BRAM tile for the shift counter and pattern storage, leaving effectively all of the device's fabric available for the subsequent CFU Playground DFX implementation.

4.6.3 Partial Bitstream Validation

Partial bitstreams for both RMs were generated and verified by issuing the `.bit` files through the Vivado Hardware Manager's partial programming interface. Switching between the counter RM and the shift-register RM without resetting the static LED shifter logic was confirmed to work correctly on three consecutive round-trip reconfiguration events, with the ILA monitoring the EOS transition confirming a clean startup sequence for each RM swap. The total partial bitstream size for each RM in the tutorial design was approximately 340 KB, consistent with the frame count implied by the Pblock's bounding box dimensions on the XC7A100T device.

4.7 Partial Bitstream Storage — Current State and Roadmap

The automation of partial bitstream retrieval and delivery to the ICAP is the final engineering step separating the current implementation from a fully autonomous, software-triggered DPR system. This section describes the current state of the bitstream storage subsystem, the constraints that have delayed its completion, and the concrete technical roadmap for achieving full autonomy.

4.7.1 Current State: In-Fabric ROM (Synthesis-Time Embedding)

In the present implementation, partial bitstreams are stored in initialised BlockRAM tiles within the static partition. The content of each BRAM tile is specified using Verilog's `$readmemh` system task during synthesis — the bitstream bytes are hex-encoded and stored in a `.mem` file that is read by the synthesis tool and compiled into the BRAM initialisation attributes (`INIT_00` through `INIT_3F`

parameters on the `RAMB36E1` primitives). At runtime, the `system_wrapper` FSM reads words from this BRAM sequentially and forwards them to the ICAP interface.

This arrangement successfully validates the complete reconfiguration sequence — DFX Decoupler assertion, ICAP write, EOS monitoring, and Decoupler release — on real hardware, and the ILA-captured `recon_counter` values in this configuration represent the true hardware reconfiguration latency of the ICAP primitive itself (exclusive of any bitstream fetch latency, since the BRAM read latency is negligible relative to the ICAP word rate). The limitation is that the bitstream is fixed at synthesis time: modifying the target RM requires resynthesising the static partition, which negates the runtime flexibility that is the core objective of DPR.

4.7.2 Near-Term Plan: QSPI Flash Fetch

The near-term target is to replace the BRAM ROM with a QSPI flash read path. The Arty A7-35T's on-board Quad-SPI flash (S25FL128S, 128 Mb / 16 MB) is already accessible from within the `system_wrapper` via the board's `qspi_flash` I/O bank. The implementation plan for this stage is:

1. **Flash programming:** At board-programming time (alongside the full static bitstream via the Hardware Manager), each RM's partial bitstream is written to a reserved flash sector at a fixed, pre-agreed base address (e.g., RM0 at offset `0x00400000`, RM1 at offset `0x00500000`).
2. **Flash controller state machine:** A fast-read (`0x0B`) command is issued to the flash with the appropriate 24-bit address. The S25FL128S returns data bytes sequentially at up to 104 MHz with a one-byte dummy cycle following the address. The `system_wrapper` pipeline collects four consecutive bytes, byte-reverses each (applying the 7-series ICAP bit-reversal requirement), and assembles them into a 32-bit word.
3. **ICAP write coincidence:** The assembled 32-bit words are forwarded directly to the ICAP without intermediate buffering, provided that the QSPI clock rate matches the ICAP's acceptance rate. If the QSPI read rate is slower than the ICAP word rate, a FIFO of sufficient depth must be interposed to absorb the rate difference.

This approach does not require DDR integration, making it the lower-complexity of the two bitstream storage strategies and the one designated as the primary near-term engineering objective.

4.7.3 Longer-Term Plan: DDR3 SDRAM Storage

Following the successful integration of the QSPI flash fetch path, a second storage tier — DDR3 SDRAM — is planned. DDR3 SDRAM provides substantially higher bandwidth than QSPI flash and lower random-access latency, making it suitable for scenarios in which multiple partial bitstreams must be accessed at low latency or in which the partial bitstreams are generated or transferred at runtime from a host.

Two architectural approaches are under consideration for DDR-backed bitstream storage:

Approach A — Boot-time migration. At system startup, after the full bitstream has been loaded and the DDR3 controller is initialised, the `system_wrapper` reads all partial bitstreams from QSPI flash and copies them into pre-allocated DDR3 address regions. Subsequent reconfiguration events fetch bitstreams directly from DDR3 via an AXI4 burst read, benefiting from the DDR's higher sustained bandwidth and lower per-word latency relative to flash.

Approach B — Runtime host transfer. Partial bitstreams are transferred from the host workstation to the FPGA's DDR3 memory at runtime via the existing UART channel. The VexRiscv

firmware receives the bitstream bytes via UART, writes them into DDR3 through the LiteX SoC's Wishbone-to-AXI bridge, and then signals the `system_wrapper` to initiate a reconfiguration event against the newly written address region. This approach provides the maximum flexibility — any RM can be uploaded and instantiated at runtime without any prior flash programming — at the cost of UART-limited transfer bandwidth (approximately 1 MB/s at 10 Mbps UART, implying a transfer time of approximately 350–420 ms for a 350–420 KB partial bitstream).

The DDR3 path is the longer-term objective and is described in the Future Work section (§7.2) as the ultimate target architecture for producing a ZyCAP-comparable autonomous reconfiguration capability on a non-Zynq Artix-7 fabric.

4.7.4 Report Scope Clarification

All reconfiguration latency measurements reported in Chapter 6 reflect the in-fabric BRAM ROM delivery path: the `recon_counter` values capture only the ICAP write and startup sequence duration, not any bitstream fetch latency. This is a deliberate measurement choice: by eliminating memory latency from the measurement, the reported figures represent the lower bound on reconfiguration time — the time attributable to the ICAP primitive and the STARTUPE2 startup sequence themselves — which is the hardware characteristic of primary interest for comparing DPR systems across platforms. When the flash or DDR fetch paths are integrated, total reconfiguration latency (fetch latency + ICAP write + startup) will be separately quantified.

Chapter 5

Software, Firmware, and RTL Source Organisation

5.1 Overview of the Software and HDL Stack

The hardware infrastructure described in Chapter 4 defines the physical boundaries within which reconfiguration occurs, but converting that infrastructure into a working system demands a complementary set of software, firmware, and register-transfer-level (RTL) design files. The present chapter offers a detailed, code-level examination of every source artefact authored or adapted during the course of this project. Two distinct implementation sub-projects comprise the portfolio of work:

1. **CFU Playground integration** — the primary RISC-V co-processor reconfiguration system on the Arty A7-35T, encompassing the top-level wrapper (`top.v`), the DFX Decoupler bridge (`cfu.v`), the reconfiguration-timing measurement module (`recon_counter.v`), three Reconfigurable Module (RM) HDL files (`example.v`, `donut.v`, `dse_template.v`), a fourth RM consisting of a multi-file SystemVerilog keyword-spotting accelerator (`kws/`), associated firmware headers (`cfu.h`), software emulation sources (`software_cfu.c`, `software_cfu.cc`), the interactive test menu (`proj_menu.cc`), and the donut rendering benchmark (`donut.c`).
2. **AMD DFX Tutorial baseline** — a reference DFX design adapted from Xilinx UG947, containing a static top-level design (`top.v`), an MMCM clock-domain module (`clocks.v`), and BRAM-based shift-pattern Reconfigurable Modules (`shift_left.v` and its counterparts).

The sections that follow examine each sub-project in source-level detail. All Verilog and SystemVerilog keywords, module names, signal names, and macro identifiers are reproduced verbatim because they are functional identifiers whose alteration would break the design. Explanatory prose surrounding these identifiers is written in original language to preserve academic integrity across the merged report.

5.2 CFU Playground — Top-Level Integration

The CFU Playground provides the application-level framework within which custom hardware accelerators are designed, compiled, and exercised on a VexRiscv soft-core processor. For this project, the CFU Playground's standard single-module architecture was extended into a DFX-capable configuration. The integration point is the `top.v` module, which serves as the physical top of the FPGA design and coordinates two major subsystems.

5.2.1 `top.v` — System-Level Composition

The top-level module `top.v` declares all off-chip interfaces required by the Arty A7-35T — DDR3 SDRAM buses, QSPI flash, USB-UART, status LEDs, and the ICAP primitive's four-port interface — and instantiates exactly two sub-modules:

```
module top(
    inout qspi_flash_io0_io,
    inout qspi_flash_io1_io,
    inout qspi_flash_io2_io,
    inout qspi_flash_io3_io,
    inout qspi_flash_sck_io,
    inout qspi_flash_ss_io,

    output wire [13:0] ddram_a,
    output wire [2:0] ddram_ba,
    output wire ddram_ras_n,
    output wire ddram_cas_n,
    output wire ddram_we_n,
    output wire ddram_cs_n,
    output wire [1:0] ddram_dm,
    inout  wire [15:0] ddram_dq,
    inout  wire [1:0] ddram_dqs_p,
    inout  wire [1:0] ddram_dqs_n,
    output wire ddram_clk_p,
    output wire ddram_clk_n,
    output wire ddram_cke,
    output wire ddram_odt,
    output wire ddram_reset_n,

    output          ICAP_csib,
    output [31:0]   ICAP_i,
    input  [31:0]   ICAP_o,
    output          ICAP_rdwrb,

    input ck_rst,
    input clk100,
    input pr_switch,
    output wire serial_tx,
    input  wire serial_rx,

    output wire user_led0,
    output wire user_led1,
    output wire user_led2,
    output wire user_led3
);
```

The two instantiated sub-modules are connected by a single internal wire, `decouple`, which serves as the control plane between the PR controller and the LiteX-generated SoC:

```
wire decouple;

system_wrapper system(
```

```
    .ck_rst(ck_rst),
    .clk100(clk100),
    .decouple(decouple),
    .pr_switch(pr_switch),
    .ICAP_csib(ICAP_csib),
    .ICAP_i(ICAP_i),
    .ICAP_o(ICAP_o),
    .ICAP_rdwrb(ICAP_rdwrb),
    .qspi_flash_io0_io(qspi_flash_io0_io),
    .qspi_flash_io1_io(qspi_flash_io1_io),
    .qspi_flash_io2_io(qspi_flash_io2_io),
    .qspi_flash_io3_io(qspi_flash_io3_io),
    .qspi_flash_sck_io(qspi_flash_sck_io),
    .qspi_flash_ss_io(qspi_flash_ss_io)
);

digilent_arty cfu_wrapper (
    .clk100(clk100),
    .cpu_reset(ck_rst),
    .eth_ref_clk(clk100),
    .decouple(decouple),
    .serial_tx(serial_tx),
    .serial_rx(serial_rx),
    .user_led0(user_led0),
    .user_led1(user_led1),
    .user_led2(user_led2),
    .user_led3(user_led3),
    .ddram_a(ddram_a),
    .ddram_ba(ddram_ba),
    .ddram_ras_n(ddram_ras_n),
    .ddram_cas_n(ddram_cas_n),
    .ddram_we_n(ddram_we_n),
    .ddram_cs_n(ddram_cs_n),
    .ddram_dm(ddram_dm),
    .ddram_dq(ddram_dq),
    .ddram_dqs_p(ddram_dqs_p),
    .ddram_dqs_n(ddram_dqs_n),
    .ddram_clk_p(ddram_clk_p),
    .ddram_clk_n(ddram_clk_n),
    .ddram_cke(ddram_cke),
    .ddram_odt(ddram_odt),
    .ddram_reset_n(ddram_reset_n)
);
endmodule
```

Architectural rationale. The `system_wrapper` instance owns the PR-control resources: the ICAP primitive ports, the QSPI flash lines (for potential bitstream fetching), and the `pr_switch` input that triggers reconfiguration. The `digilent_arty` instance (`cfu_wrapper`) is the LiteX-generated SoC containing the VexRiscv processor, the DDR3 MIG controller, and the UART bridge. The sole shared signal between the two halves is `decouple`, which gates the CFU interface; when `system_wrapper` asserts `decouple` high, the DFX Decoupler within the SoC isolates the Reconfigurable Partition from

the processor, allowing a new RM to be loaded without corrupting the pipeline state.

This partitioning strategy means that the PR controller logic and the application processor logic occupy separate hierarchical namespaces, which simplifies floorplanning — the `system_wrapper` can be constrained to a compact region near the ICAP primitive, while the SoC occupies the remainder of the device. The `eth_ref_clk` input on the SoC instance is tied to `clk100` because the Ethernet PHY reference clock must be present for the LiteX-generated MII logic to initialise, even though Ethernet itself is not used in this project.

5.3 CFU Playground — Decoupler Bridge (`cfu.v`)

The `Cfu` module (note: the CFU Playground uses Pascal case for its top-level module name per its coding convention) implements the isolation bridge between the static-side CFU handshake signals and the Reconfigurable Partition identified as `cfu_compute`:

```
module Cfu (
  input                cmd_valid,
  output               cmd_ready,
  input       [9:0]    cmd_payload_function_id,
  input       [31:0]   cmd_payload_inputs_0,
  input       [31:0]   cmd_payload_inputs_1,
  output               rsp_valid,
  input                rsp_ready,
  output      [31:0]   rsp_payload_outputs_0,
  input                decouple,
  input                reset,
  input                clk
);
```

5.3.1 Signal Isolation Through the DFX Decoupler

Internally, the module declares a complete shadow set of the CFU handshake signals — prefixed with `rp_` — representing the reconfigurable-partition-side view of each signal:

```
  wire            rp_cmd_valid, rp_cmd_ready, rp_rsp_valid, rp_rsp_ready;
  wire   [9:0]    rp_cmd_payload_function_id;
  wire   [31:0]   rp_cmd_payload_inputs_0;
  wire   [31:0]   rp_cmd_payload_inputs_1;
  wire   [31:0]   rp_rsp_payload_outputs_0;
  wire            rp_reset;
  wire            rp_clk;
```

The `cfu_compute` RM is connected to the `rp_*` signals rather than to the top-level ports directly:

```
cfu_compute comp (
  .clk(rp_clk),
  .reset(rp_reset),
  .cmd_payload_function_id(rp_cmd_payload_function_id),
  .cmd_payload_inputs_0(rp_cmd_payload_inputs_0),
  .cmd_payload_inputs_1(rp_cmd_payload_inputs_1),
  .rsp_payload_outputs_0(rp_rsp_payload_outputs_0),
```

```
  .cmd_valid(rp_cmd_valid),
  .cmd_ready(rp_cmd_ready),
  .rsp_valid(rp_rsp_valid),
  .rsp_ready(rp_rsp_ready)
);
```

The Xilinx DFX Decoupler IP (`dfx_decoupler_0`) bridges every signal pair with a registered isolation stage:

```
dfx_decoupler_0 decoupler (
  .s_function_id_DATA(cmd_payload_function_id),
  .rp_function_id_DATA(rp_cmd_payload_function_id),

  .s_rsp_payload_DATA(rsp_payload_outputs_0),
  .rp_rsp_payload_DATA(rp_rsp_payload_outputs_0),

  .s_cmd_payload_cmd_payload_0(cmd_payload_inputs_0),
  .rp_cmd_payload_cmd_payload_0(rp_cmd_payload_inputs_0),
  .s_cmd_payload_cmd_payload_1(cmd_payload_inputs_1),
  .rp_cmd_payload_cmd_payload_1(rp_cmd_payload_inputs_1),

  .s_controls_clk(clk),
  .rp_controls_clk(rp_clk),
  .s_controls_cmd_valid(cmd_valid),
  .rp_controls_cmd_valid(rp_cmd_valid),
  .s_controls_cmd_ready(cmd_ready),
  .rp_controls_cmd_ready(rp_cmd_ready),
  .s_controls_rsp_ready(rsp_ready),
  .rp_controls_rsp_ready(rp_rsp_ready),
  .s_controls_rsp_valid(rsp_valid),
  .rp_controls_rsp_valid(rp_rsp_valid),
  .s_controls_reset(reset),
  .rp_controls_reset(rp_reset),

  .decouple(decouple),
  .decouple_status()
);
```

Port-naming convention. The Decoupler IP uses a `s_` prefix for static-side ports and an `rp_` prefix for reconfigurable-partition-side ports. Each data path (function ID, input payloads, output payload) and each control signal (valid, ready, clock, reset) passes through the Decoupler as a separate named interface. This granularity gives the Decoupler the ability to apply per-signal default values during isolation — for instance, holding `cmd_ready` low on the static side so that the VexRiscv pipeline stalls during reconfiguration without receiving spurious acknowledgements.

Clock and reset forwarding. The `s_controls_clk` and `rp_controls_clk` paths allow the Decoupler to gate the clock to the RM during isolation if configured to do so. In the present design, the clock is passed through transparently in both states because the RM's reset signal — also forwarded through the Decoupler — is sufficient to bring the new module into a consistent state upon re-coupling.

`decouple_status` output. This signal is left unconnected in the current implementation. It reports

the Decoupler's internal state and could be routed to the `system_wrapper` in a future revision that verifies isolation before initiating an ICAP write sequence.

5.4 CFU Playground — Reconfiguration Timing Module (`recon_counter.v`)

The `recon_counter` module provides the sole on-chip measurement instrument for reconfiguration latency. Its operation is tied to the `STARTUPE2` primitive, which is a 7-series-specific hard block that exposes device configuration events to user logic.

5.4.1 Complete Source and Annotation

```
module recon_counter (
  input clk,
  output eos
);

(* KEEP = "true" *)
wire preq;

STARTUPE2 STARTUPE2_inst (
   .EOS(eos),              // Active high End-of-Startup
   .CLK(clk),
   .GSR(1'b0),
   .GTS(1'b0),
   .PACK(1'b0),
   .USRCCLKO(1'b0),
   .USRCCLKTS(1'b1),
   .KEYCLEARB(1'b1),
   .USRDONEO(1'b1),        // Drive DONE pin high
   .USRDONETS(1'b0)        // Do not tristate DONE pin
);
```

The `STARTUPE2` primitive is instantiated with deliberately chosen tie-off values. `GSR`, `GTS`, and `PACK` are all held inactive (`0`) because the design does not use the global set/reset, global tristate, or program acknowledge features. `USRCCLKO` is `0` because the user configuration clock output is not used; `USRCCLKTS` is `1` to tristate this output. `KEYCLEARB` is held high to prevent inadvertent key clearing. `USRDONEO` is asserted so that the external DONE pin stays high after startup.

The `preq` wire, marked with the Vivado `KEEP` synthesis attribute, is included to prevent the synthesis tool from optimising away the `STARTUPE2` instance — even though `preq` is not connected to anything in the current design, its presence as a kept wire attached to the module scope ensures the primitive is retained in the netlist. This is a defensive measure observed in several Xilinx reference designs.

5.4.2 Counter State Machine

```
(* KEEP = "true" *)
reg [31:0] counter = 0, counter_next;
```

```
(* KEEP = "true" *)
localparam IDLE = 0, RECON = 1;

(* KEEP = "true" *)
reg state = 0, next_state;

always @(*) begin : counter_state
    case (state)
    IDLE: begin
        next_state = (!eos)? RECON : IDLE;
        counter_next = counter;
    end
    RECON: begin
        next_state = (eos)? IDLE : RECON;
        counter_next = counter + 1;
    end
    default: begin
        next_state = IDLE;
        counter_next = counter;
    end
    endcase
end

always @(posedge clk) begin : counter_switch
  state = next_state;
  if(preq)
    counter <= 32'd0;
  else
    counter <= counter_next;
end

ila_0 ila(.clk(clk),.probe0(counter));

endmodule
```

FSM coding style. The state machine uses a two-block structure: one combinational `always @(*)` block computes `next_state` and `counter_next` based on the current state and the `eos` signal, and one clocked `always @(posedge clk)` block registers these values. This separation is intentional — when the FSM is in `IDLE`, the counter value is held unchanged; when `eos` drops (indicating active reconfiguration), the FSM transitions to `RECON` and the counter increments once per clock cycle. The counter holds its final value indefinitely after `eos` returns high, providing a stable measurement that can be captured by the Integrated Logic Analyzer.

Blocking assignment on `state`. The use of `state = next_state` (blocking assignment) within the clocked block for the `state` register is a known coding choice in this module. While non-blocking assignments are preferred for intra-clock registered logic in synthesisable designs, the behaviour here is functionally equivalent because `state` is not read again within the same clocked block — `counter_next` has already been computed by the combinational block based on the previous cycle's `state`. However, this is worth noting as a potential area for refactoring in future revisions.

`(* KEEP = "true" *)` annotations. All registers and the localparam carry the `KEEP` attribute to

prevent Vivado's optimiser from merging, absorbing, or eliminating the counter registers. Without these annotations, synthesis could replicate or merge the counter with other fan-out structures, yielding incorrect ILA probe connections.

ILA instantiation. The single-line `ila_0 ila(.clk(clk), .probe0(counter))` statement connects the 32-bit counter directly to Probe 0 of a Vivado ILA IP core. This ILA core is configured in the Vivado IP Integrator with a single 32-bit probe width and a capture depth determined by the available BRAM resources. By placing the ILA inside `recon_counter.v` rather than at the top level, the probe connections are preserved across synthesis steps and avoid the need for manual netlist editing in the post-synthesis design.

5.5 CFU Playground — Reconfigurable Modules

Three Verilog partitions and one multi-file SystemVerilog partition occupy the `partitions/` directory. Each partition defines a module named `cfu_compute` (for the three Verilog files) or `Cfu` (for the KWS accelerator), conforming to the port interface expected by the `comp` instance in `cfu.v`. Every RM must expose the same top-level handshake ports (`cmd_valid`, `cmd_ready`, `cmd_payload_function_id`, `cmd_payload_inputs_0`, `cmd_payload_inputs_1`, `rsp_valid`, `rsp_ready`, `rsp_payload_outputs_0`, `clk`, `reset`) so that the Vivado DFX tool can verify port-signature equivalence during the child implementation run.

5.5.1 `example.v` — Multi-Function Combinational Accelerator

This RM implements three independent bit-manipulation operations selected by the `cmd_payload_function_id` field:

```
module cfu_compute (
  input                cmd_valid,
  output               cmd_ready,
  input       [9:0]    cmd_payload_function_id,
  input       [31:0]   cmd_payload_inputs_0,
  input       [31:0]   cmd_payload_inputs_1,
  output               rsp_valid,
  input                rsp_ready,
  output      [31:0]   rsp_payload_outputs_0,
  input                clk,
  input                reset
);

  assign cmd_ready = rsp_ready;

  reg [31:0] rsp_payload;
  reg rsp_valid0, rsp_valid1;
  reg [31:0] cfu_in0, cfu_in1;

  always @(posedge clk) begin
    // stage 1: register inputs
    rsp_valid0 <= cmd_valid;
    cfu_in0 <= cmd_payload_inputs_0;
    cfu_in1 <= cmd_payload_inputs_1;
```

```
    // stage 2: register output
    rsp_valid1 <= rsp_valid0;
    rsp_payload <= cmd_payload_function_id[1] ? cfu2 :
                       ( cmd_payload_function_id[0] ? cfu1 : cfu0);
  end

  assign rsp_valid = rsp_valid1;
```

Pipeline structure. Although the underlying computations are purely combinational, the module introduces a two-stage pipeline using registered intermediates `cfu_in0`/`cfu_in1` (Stage 1) and `rsp_payload` (Stage 2). This pipeline costs two clock cycles of latency but provides timing margin — the combinational datapath between the registered inputs and the output register has a full clock period in which to settle, rather than having to resolve through both the function-select multiplexer and the arithmetic logic within a single cycle.

Operation 0 — Byte sum: adds all eight constituent bytes across both 32-bit inputs:

```
  wire [31:0] cfu0;
  assign cfu0[31:0] =  cfu_in0[7:0]   + cfu_in1[7:0] +
                       cfu_in0[15:8]  + cfu_in1[15:8] +
                       cfu_in0[23:16] + cfu_in1[23:16] +
                       cfu_in0[31:24] + cfu_in1[31:24];
```

Each byte slice (`[7:0]`, `[15:8]`, etc.) is zero-extended to 32 bits by the addition chain; no explicit sign extension is applied, consistent with unsigned byte interpretation. The maximum possible output is `8 × 255 = 2040`, which comfortably fits in 12 bits.

Operation 1 — Byte swap (endianness reversal):

```
  wire [31:0] cfu1;
  assign cfu1[31:24] =     cfu_in0[7:0];
  assign cfu1[23:16] =     cfu_in0[15:8];
  assign cfu1[15:8] =      cfu_in0[23:16];
  assign cfu1[7:0] =       cfu_in0[31:24];
```

This operation reverses the byte ordering of `inputs_0` — the least-significant byte becomes the most-significant byte and vice versa. The result is a 32-bit value whose byte order has been transposed, equivalent to converting between big-endian and little-endian representations.

Operation 2 — Full-word bit reversal:

```
  wire [31:0] cfu2;
  genvar n;
  generate
      for (n=0; n<32; n=n+1) begin
          assign cfu2[n] =     cfu_in0[31-n];
      end
  endgenerate
```

The `generate`/`for` construct maps each bit position to its mirror image across the 32-bit word. Bit 0 of the output receives bit 31 of the input, bit 1 receives bit 30, and so on. This produces a bitwise palindrome of the input operand.

Output multiplexing: The function-select logic at Stage 2 uses `cmd_payload_function_id[1]` and `[0]` to choose between the three operations. Notably, the comment in the source acknowledges that the 3-bit `funct3` subfield is only partially decoded — bit 1 set selects bit reversal regardless of bit 0, bit 0 set (with bit 1 clear) selects byte swap, and neither set selects byte sum.

5.5.2 `donut.v` — Signed Integer Multiply with Shift

The donut RM provides two fixed-point arithmetic operations used by the 3D donut rendering application:

```
module cfu_compute (
  input                cmd_valid,
  output               cmd_ready,
  input       [9:0]    cmd_payload_function_id,
  input       [31:0]   cmd_payload_inputs_0,
  input       [31:0]   cmd_payload_inputs_1,
  output               rsp_valid,
  input                rsp_ready,
  output      [31:0]   rsp_payload_outputs_0,
  input                reset,
  input                clk
);

  assign cmd_ready = rsp_ready;

  reg [31:0] rsp_payload;
  reg rsp_valid0, rsp_valid1;
  reg [31:0] cfu_in0, cfu_in1;

  always @(posedge clk) begin
    rsp_valid0 <= cmd_valid;
    cfu_in0 <= cmd_payload_inputs_0;
    cfu_in1 <= cmd_payload_inputs_1;

    rsp_valid1 <= rsp_valid0;
    rsp_payload <= cmd_payload_function_id[0] ? mul : mulsh;
  end

  assign rsp_valid = rsp_valid1;

  wire [31:0] mul    = $signed(cfu_in0) * $signed(cfu_in1);
  wire [31:0] mulsh  = $signed(mul) >>> 10;

  assign rsp_payload_outputs_0 = rsp_payload;

endmodule
```

`mul` — signed 32×32 multiplication: The `$signed()` cast forces the synthesis tool to infer a signed multiplier, which on the XC7A35T maps to the DSP48E1 hard multiply primitive. Only the lower 32 bits of the 64-bit product are retained in the `mul` wire — the upper 32 bits are discarded, which is acceptable for the donut rendering workload where operands are fixed-point values with

limited dynamic range.

`mulsh` — multiply-then-arithmetic-shift: The arithmetic right shift by 10 positions (`>>> 10`) diverts 10 bits from the fractional part of the product, effectively performing a divide-by-1024 with correct sign extension. This fixed-point scaling is central to the donut application's trigonometric approximation chain, where sine and cosine values are represented as signed integers in a Q10.22 fixed-point format.

Relationship to `donut.c`: The firmware file `donut.c` invokes these two operations through the macros `MULTSHIFT10(a,b)` and `CFUMUL(a,b)`:

```
#define MULTSHIFT10(a,b)      ((int)cfu_op(0, 0, (a), (b)))
#define CFUMUL(a,b)           ((int)cfu_op(1, 1, (a), (b)))
```

The first argument to `cfu_op` is the function ID (0 for multiply-then-shift, 1 for raw multiply). The nested rotation macro `R(mul,shift,x,y)` in `donut.c` demonstrates the real-world use case:

```
#define R(mul,shift,x,y) \
  _=x; \
  x -= CFUMUL(mul,y)>>shift; \
  y += CFUMUL(mul,_)>>shift; \
  _ = 3145728-CFUMUL(x,x)-CFUMUL(y,y)>>11; \
  x = CFUMUL(x,_)>>10; \
  y = CFUMUL(y,_)>>10;
```

Each invocation of `CFUMUL` compiles to a single CUSTOM0 RISC-V instruction. The VexRiscv pipeline stalls until the two-stage pipeline in the RM produces a valid result, then resumes with the 32-bit product available in the destination register.

5.5.3 `dse_template.v` — SIMD Multiply-Accumulate

The DSE (Design Space Exploration) template RM implements a four-lane signed 8-bit multiply-accumulate unit intended for quantised neural-network inference workloads:

```
module cfu_compute (
  input                cmd_valid,
  output               cmd_ready,
  input       [9:0]    cmd_payload_function_id,
  input       [31:0]   cmd_payload_inputs_0,
  input       [31:0]   cmd_payload_inputs_1,
  output reg           rsp_valid,
  input                rsp_ready,
  output reg  [31:0]   rsp_payload_outputs_0,
  input                reset,
  input                clk
);
  localparam InputOffset = $signed(9'd128);

  wire signed [15:0] prod_0, prod_1, prod_2, prod_3;
  assign prod_0 =  ($signed(cmd_payload_inputs_0[7 : 0]) + InputOffset)
                 * $signed(cmd_payload_inputs_1[7 : 0]);
  assign prod_1 =  ($signed(cmd_payload_inputs_0[15: 8]) + InputOffset)
                 * $signed(cmd_payload_inputs_1[15: 8]);
```

```
  assign prod_2 =  ($signed(cmd_payload_inputs_0[23:16]) + InputOffset)
                 * $signed(cmd_payload_inputs_1[23:16]);
  assign prod_3 =  ($signed(cmd_payload_inputs_0[31:24]) + InputOffset)
                 * $signed(cmd_payload_inputs_1[31:24]);

  wire signed [31:0] sum_prods;
  assign sum_prods = prod_0 + prod_1 + prod_2 + prod_3;
```

SIMD lane structure. Each of the four 8-bit byte lanes within `inputs_0` is treated as a signed filter-weight value that receives a constant `InputOffset` of +128 before being multiplied with the corresponding byte of `inputs_1`. This offset maps the unsigned 8-bit input activation range [0, 255] to the signed 9-bit range [128, 383], which is the convention used by the TensorFlow Lite Micro quantisation scheme for the first convolutional layer. The 16-bit products are summed to form a 32-bit accumulated partial result.

Accumulation and handshake logic:

```
  assign cmd_ready = ~rsp_valid;

  always @(posedge clk) begin
    if (reset) begin
      rsp_payload_outputs_0 <= 32'b0;
      rsp_valid <= 1'b0;
    end else if (rsp_valid) begin
      rsp_valid <= ~rsp_ready;
    end else if (cmd_valid) begin
      rsp_valid <= 1'b1;
      rsp_payload_outputs_0 <= |cmd_payload_function_id[9:3]
          ? 32'b0
          : rsp_payload_outputs_0 + sum_prods;
    end
  end
endmodule
```

The accumulator register `rsp_payload_outputs_0` is declared as `output reg` — a departure from the combinational output style of `example.v` and `donut.v`. This is because the accumulation semantics require the register to retain its value across multiple CFU invocations. The `cmd_payload_function_id[9:3]` check provides a reset pathway: when any of the upper 7 bits of the function ID are non-zero, the accumulator is cleared to zero rather than accumulating further, allowing the firmware to reset the accumulator for a new dot-product sequence without issuing a full module reset.

The back-pressure handshake (`cmd_ready = ~rsp_valid`) prevents the processor from issuing a new command while a previous result is waiting to be consumed. This single-cycle-response, accumulating architecture is representative of the pattern used in quantised inference accelerators where many multiply-accumulate operations are streamed sequentially before a single readout.

5.5.4 `kws/` — Keyword Spotting Accelerator (SystemVerilog)

The fourth RM is implemented in SystemVerilog across four files and provides a purpose-built accelerator for the keyword-spotting (KWS) inference pipeline used in the CFU Playground's TinyML benchmarks.

`cfu.sv` — Top-level dispatch: The top module instantiates three sub-modules (MAC, RDH, RCDBPOT) and selects among their outputs using a one-hot encoding on the lowest three bits of the function ID:

```
`include "mac.sv"
`include "rcdbpot.sv"
`include "rdh.sv"

module Cfu (
  input logic clk,
  input logic reset,
  input logic [9:0]  cmd_payload_function_id,
  input logic [31:0] cmd_payload_inputs_0,
  input logic [31:0] cmd_payload_inputs_1,
  output logic [31:0] rsp_payload_outputs_0,
  input  logic cmd_valid,
  output logic cmd_ready,
  output logic rsp_valid,
  input  logic rsp_ready
);
```

The output multiplexer uses a `casez` statement with don't-care bits for priority encoding:

```
  always_ff @(posedge clk) begin
    if (cmd_valid) begin
      casez (cmd_payload_function_id[2:0])
        3'b??1 : rsp_payload_outputs_0 <= mac_output;
        3'b?1? : rsp_payload_outputs_0 <= rdh_output;
        3'b1?? : rsp_payload_outputs_0 <= rcdbpot_output;
        default: rsp_payload_outputs_0 <= '0;
      endcase
    end
  end
```

`mac.sv` — Multiply-Accumulate with configurable offset:

```
module mac (
  input logic layer_one_en,
  input logic simd_en,
  input logic [31:0] input_vals,
  input logic [31:0] filter_vals,
  input logic [31:0] curr_acc,
  output logic [31:0] out
);
  logic signed [8:0] InputOffest = layer_one_en ? -9'sd83 : 9'sd128;
```

The `InputOffest` (note: the original source preserves this spelling) is configurable at runtime through the `layer_one_en` flag, derived from `cmd_payload_function_id[4]`. For the first convolutional layer, an offset of -83 is used (reflecting the quantisation zero-point of the input tensor); for subsequent layers, +128 is used. The `simd_en` flag (bit 3) controls whether all four byte lanes are summed or only the first lane is used, providing both scalar and SIMD operating modes in a single module.

`rdh.sv` — Rounding Doubling High 32 Bits: This module takes a 64-bit value represented as `{top, bottom}`, doubles it, rounds, and outputs only the upper 32 bits:

```
module rdh (
  input logic [31:0] top,
  input logic [31:0] bottom,
  output logic [31:0] out
);
  assign out = (signed'({top, bottom}) + 32'sh40000000) >> 31;
endmodule
```

The constant `32'sh40000000` is the rounding bias for the midpoint of the 64-bit range. The right shift by 31 (not 32) effectively doubles the value while extracting the high word — a pattern taken directly from the TensorFlow Lite reference kernel's fixed-point arithmetic library.

`rcdbpot.sv` — Rounding Clamping Divide by Power of Two: This module divides the accumulated result by a power of two (encoded as a negative exponent) and clamps the output to the range [-128, 127] before applying a final offset of -128:

```
module rcdbpot (
  input logic [31:0] dividend,
  input logic [31:0] negative_exponent,
  output logic [31:0] out
);
  logic [3:0] shift;
  logic signed [31:0] mask, remainder, threshold;

  always_comb begin
    casez (negative_exponent[2:0])
      3'b111 : shift = 9;
      3'b?11 : shift = 5;
      3'b??1 : shift = 7;
      3'b?1? : shift = 6;
      default: shift = 8;
    endcase

    mask = ~({32{1'b1}} << shift);
    remainder = dividend & mask;
    threshold = (mask >> 1) + dividend[31];
    out = signed'(signed'(dividend) >>> shift);

    if (remainder > threshold) out += 1'b1;
    if (out[31]) out = 32'sd0;
    else if (|out[31:8]) out = 32'sd255;
    out -= 32'sd128;
  end
endmodule
```

The `casez`-based shift decoding maps the exponent encoding into one of five possible shift amounts (5 through 9), optimised for the quantisation parameters observed in the KWS model. The rounding logic adds 1 if the truncated remainder exceeds half the divisor (with a sign-dependent adjustment via `dividend[31]`), providing proper round-to-nearest-even semantics for negative dividends. The

final clamping to [0, 255] followed by subtraction of 128 produces a signed 8-bit output in the range [-128, 127], ready for the next quantised layer's input.

5.6 CFU Playground — Firmware Sources

5.6.1 `cfu.h` — Instruction Encoding and SW/HW Selection

The header file `cfu.h` defines the firmware-visible interface to the CFU hardware. It imports the `riscv.h` header (which provides the `opcode_R` macro for raw R-type instruction encoding) and defines both hardware-invoking macros and functionally-equivalent software-emulation macros:

```
#include "riscv.h"
```

Hardware-invocation macros:

```
#define cfu_op0_hw(rs1, rs2)  opcode_R(CUSTOM0, 0, 0, (rs1), (rs2))
#define cfu_op1_hw(rs1, rs2)  opcode_R(CUSTOM0, 1, 0, (rs1), (rs2))
#define cfu_op2_hw(rs1, rs2)  opcode_R(CUSTOM0, 2, 0, (rs1), (rs2))
#define cfu_op3_hw(rs1, rs2)  opcode_R(CUSTOM0, 3, 0, (rs1), (rs2))
#define cfu_op4_hw(rs1, rs2)  opcode_R(CUSTOM0, 4, 0, (rs1), (rs2))
#define cfu_op5_hw(rs1, rs2)  opcode_R(CUSTOM0, 5, 0, (rs1), (rs2))
```

Each `opcode_R` invocation encodes an R-type instruction with opcode `CUSTOM0` (binary `0001011`, decimal 11), the specified `funct3` value (0 through 5), `funct7=0`, and the register-file indices for `rs1` and `rs2`. The resulting machine word is emitted directly into the instruction stream via inline assembly.

Semantically-named aliases provide self-documenting invocations:

```
#define cfu_byte_sum_hw(rs1, rs2)    opcode_R(CUSTOM0, 0, 0, (rs1), (rs2
   ))
#define cfu_byte_swap_hw(rs1, rs2)   opcode_R(CUSTOM0, 1, 0, (rs1), (rs2
   ))
#define cfu_bit_reverse_hw(rs1, rs2) opcode_R(CUSTOM0, 2, 0, (rs1), (rs2
   ))
#define cfu_fib_hw(rs1, rs2)         opcode_R(CUSTOM0, 3, 0, (rs1), (rs2
   ))
#define cfu_donut_mulsh_hw(rs1, rs2) opcode_R(CUSTOM0, 4, 0, (rs1), (rs2
   ))
#define cfu_donut_mul_hw(rs1, rs2)   opcode_R(CUSTOM0, 5, 0, (rs1), (rs2
   ))
```

Software-emulation macros call the C function `Cfu()` defined in `software_cfu.c`:

```
#define cfu_op0_sw(rs1, rs2)  Cfu(0, rs1, rs2)
#define cfu_op1_sw(rs1, rs2)  Cfu(1, rs1, rs2)
```

Compile-time selection between hardware and software execution is controlled by the `CFU_FORCE_SW` preprocessor define:

```
#ifdef CFU_FORCE_SW
#define cfu_op0(rs1, rs2)       cfu_op0_sw(rs1, rs2)
```

```
...
#else
#define cfu_op0(rs1, rs2)        cfu_op0_hw((rs1), (rs2))
...
#endif
```

This mechanism allows the entire firmware test suite to run on a standard RISC-V simulator (without custom hardware) during development, then switch to hardware execution for on-board validation, without modifying any application-level source code.

5.6.2 `software_cfu.c` — C-Language Reference Model

The software emulation function provides a bit-accurate reference implementation for the operations supported by the `example.v` and `donut.v` RMs:

```
uint32_t software_cfu(int funct3, uint32_t rs1, uint32_t rs2)
{
  uint32_t retval = 0;

  if (funct3 & 0x2)
  {
    // bitreverse (rs1)
    for (int i = 0; i < 32; ++i)
    {
      retval |= (((rs1 >> i) & 0x1) << (31 - i));
    }
  }
  else if (funct3 & 0x1)
  {
    // byte swap (rs1)
    for (int i = 0; i < 32; i += 8)
    {
      retval |= (((rs1 >> i) & 0xff) << (24 - i));
    }
  }
  else
  {
    // byte sum
    retval += (rs1 & 0xff) + (rs2 & 0xff);
    rs1 >>= 8; rs2 >>= 8;
    retval += (rs1 & 0xff) + (rs2 & 0xff);
    rs1 >>= 8; rs2 >>= 8;
    retval += (rs1 & 0xff) + (rs2 & 0xff);
    rs1 >>= 8; rs2 >>= 8;
    retval += (rs1 & 0xff) + (rs2 & 0xff);
  }

  return (funct3 & 4)?
            (funct3 & 5) ?
                ((int)rs1) * ((int)rs2)  :
                (((int)rs1) * ((int)rs2) >> 10)  :
                    retval;
```

```
}
```

The final ternary chain merges the donut RM's multiply operations (function IDs 4 and 5) with the `example.v` operations (function IDs 0–2). When `funct3` bit 2 is set, the function returns a signed multiply (ID 5) or a multiply-then-shift-by-10 (ID 4). The `main()` function at the end of the file provides a standalone host-side test harness that exercises the byte-sum path with a grid of input values, confirming reference-model correctness before hardware testing.

5.6.3 `proj_menu.cc` — Interactive Test Menu

The interactive menu provides a UART-driven interface for exercising CFU operations directly on the board:

```
struct Menu MENU = {
    "Project Menu",
    "project",
    {
        MENU_ITEM('e', "exercise example cfu op0",
   do_example_exercise_cfu_op),
        MENU_ITEM('0', "exercise donut cfu op0",
   do_donut_exercise_cfu_op0),
        MENU_ITEM('1', "exercise donut cfu op1",
   do_donut_exercise_cfu_op1),
        MENU_ITEM('g', "grid cfu donut op0", do_donut_grid_cfu_op0),
        MENU_ITEM('G', "grid cfu donut op1", do_donut_grid_cfu_op1),
        MENU_ITEM('h', "say Hello", do_hello_world),
        MENU_END,
    },
};
```

Each menu item maps a keyboard character to a test function. For example, `do_donut_exercise_cfu_op0` performs an exhaustive sweep over a grid of signed 32-bit inputs and compares the hardware CFU output against the expected software result:

```
void do_donut_exercise_cfu_op0(void) {
  puts("\nExercise Donut CFU Op0\n");
  int count = 0;
  for (int32_t a = -0x71234567; a < 0x68000000; a += 0x10012345) {
    for (int32_t b = -0x7edcba98; b < 0x68000000; b += 0x10770077) {
      int32_t cfu = cfu_op0(0, a, b);
      int32_t exp = a*b>>10;
      int32_t ab = a*b;
      int32_t cfuab = cfu_op1(1, a, b);
      printf("a: %08x b:%08x cfu=%08x exp=%08x  ab=%08x\n",
             (unsigned)a, (unsigned)b, (unsigned)cfu,
             (unsigned)exp, (unsigned)ab);
      if (cfu != exp) {
        printf("\n***FAIL\n");
        printf("\nexpect %08x  a*b=%08x  cfu:a*b=%08x\n",
               (unsigned)exp, (unsigned)ab, (unsigned)cfuab);
        return;
      }
```

```
      count++;
    }
  }
  printf("Performed %d comparisons", count);
}
```

The step sizes (`0x10012345`, `0x10770077`) are deliberately chosen as coprime values that span the full signed 32-bit range without requiring billions of iterations. A mismatch between the hardware result and the software expectation terminates the sweep immediately with a diagnostic `FAIL` message, printing both the expected and actual values to aid debugging.

5.6.4 `donut.c` — 3D Donut Rendering Benchmark

The donut benchmark is a fixed-point 3D torus renderer that produces animated ASCII art on the UART terminal. It serves as both a visual demonstration and a performance benchmark for the `donut.v` RM:

```
void donut(void) {
  int sA=1024,cA=0,sB=1024,cB=0,_;
  puts("\nPress any key to exit.  Accelerated version.\n");
  for (;;) {
    unsigned start_cycle = perf_get_mcycle();
    memset(b, 32, 1760);  // text buffer
    memset(z, 127, 1760);   // z buffer
    int sj=0, cj=1024;
    for (int j = 0; j < 90; j++) {
      int si = 0, ci = 1024;
      for (int i = 0; i < 324; i++) {
        int R1 = 1, R2 = 2048, K2 = 5120*1024;

        int x0 = CFUMUL(R1,cj) + R2,
            x1 = MULTSHIFT10(ci,x0),
            ...
```

Each frame renders a 80×22-character ASCII image using nested loops over torus surface parameters `i` and `j`. The `CFUMUL` and `MULTSHIFT10` macros are the critical acceleration points — without hardware CFU support, the signed multiplications would fall back to the VexRiscv's multi-cycle software multiply routine (since the `minimal` variant lacks the M extension). With the donut RM active, each multiply completes in two pipeline stages of the CFU.

The `perf_get_mcycle()` call reads the RISC-V machine-cycle CSR, providing a hardware-accurate cycle count for each rendered frame, which is printed to the terminal as the frame rate metric recorded in Chapter 6.

5.7 AMD DFX Tutorial Baseline Implementation

Before integrating DFX into the full CFU Playground system, a baseline DFX design was constructed following the AMD UG947 tutorial. This provided familiarity with the Vivado DFX toolflow without the complexity of the LiteX SoC.

5.7.1 Tutorial `top.v` — Static Partition

```
module top(
    input        gclk,            // 100MHz input clock
    input        rst_n,           // Reset mapped to center push button
    output [3:0] shift_high_out,  // mapped to general purpose LEDs[4-7]
    output [3:0] shift_low_out    // mapped to general purpose LEDs[0-3]
);

  wire        rst;
  reg  [34:0] count;

  assign rst = rst_n;

  // instantiate module shift for low bits
  shift inst_shift_low (
      .en      (rst),
      .clk     (gclk),
      .addr    (count[34:23]),
      .data_out(shift_low_out)
  );

  // instantiate module shift for high bits
  shift inst_shift_high (
      .en      (rst),
      .clk     (gclk),
      .addr    (count[34:23]),
      .data_out(shift_high_out)
  );

  always @(posedge gclk)
    if (rst) begin
      count <= 0;
    end else begin
      count <= count + 1;
    end

endmodule
```

The static partition contains a 35-bit free-running counter whose upper bits (`count[34:23]`) serve as addresses into two instances of the `shift` RM. The 12-bit address space produces a 4-bit LED output pattern that shifts visually at a rate determined by the counter's division ratio: with a 100 MHz clock, the address MSB toggles at approximately 0.003 Hz (every ~335 seconds), yielding a slow, visually observable shift pattern on the LEDs.

5.7.2 `shift_left.v` — BRAM-Based Reconfigurable Module

The RM uses a `RAMB36` primitive pre-initialised with a repeating LED shift pattern:

```
module shift (
   en,
   clk,
```

```
   addr,
   data_out);

   input en;
   input clk;
   input  [11:0] addr;
   output [3:0]  data_out;

RAMB36 #(
.SIM_MODE("SAFE"),
.DOA_REG(0),
.DOB_REG(0),
.READ_WIDTH_A(9),
...
.INIT_00(256'h0804020108040201080402 01...),
...
) RAMB36_inst (
.DOA(data_out),
.ADDRA({addr[11:0], 3'b 000}),
.CLKA(clk),
.ENA(~en),
.REGCEA(~en),
.SSRA(en),
...
);
endmodule
```

The BRAM initialisation data (`INIT_00` through `INIT_7F`) contains the repeating pattern `08 04 02 01` — corresponding to a left-shifting one-hot LED display. The counterpart RM `shift_right.v` uses the reverse pattern `01 02 04 08`. The DFX toolflow permits swapping between these two RMs at runtime, causing the LED animation to reverse direction without disrupting the static counter logic.

5.7.3 `clocks.v` — MMCM Configuration

The clock module instantiates the `MMCM_ADV` primitive to divide a 200 MHz differential input down to 100 MHz:

```
MMCM_ADV
#(.BANDWIDTH("OPTIMIZED"),
  .DIVCLK_DIVIDE(1),
  .CLKFBOUT_MULT_F(6.000),
  .CLKOUT0_DIVIDE_F(12.000),
  .CLKIN1_PERIOD(5.0),
  ...)
mmcm_adv_inst (...);
```

The feedback multiplication of 6× and output division of 12× produces a net division of 2.0, converting the 200 MHz input to 100 MHz. This module was included in the tutorial baseline but is bypassed in the Arty A7 adaptation (where the board provides a single-ended 100 MHz clock directly, as noted in the comment `// clocks U0_clocks ...` in `top.v`).

5.8 Constraint Files and Physical Floorplanning

5.8.1 Arty A7-35T Master XDC

The Digilent-provided `Arty-A7-100-Master.xdc` serves as the base constraint template for all Arty A7 implementations. Each I/O assignment maps a logical port name to a physical FPGA package pin and I/O standard:

```
## Clock signal
#set_property -dict { PACKAGE_PIN E3  IOSTANDARD LVCMOS33 } [get_ports {
    CLK100MHZ }];
#create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [
   get_ports { CLK100MHZ }];
```

In practice, the specific lines that are uncommented depend on which ports a given design uses. The CFU Playground integration uncomments the clock, UART, DDR3, QSPI, LED, and button assignments while leaving Pmod, Ethernet, and analog headers commented. The `pr_switch` input is mapped to one of the four on-board push buttons.

5.9 Build Flow Summary

The complete build pipeline, from source checkout to on-board validation, follows this ordered sequence:

Step	Action	Key Output
1	Generate LiteX SoC (`python -m litex_boards.targets.digilent_arty ...`)	`digilent_arty.v`, `arty.xdc`, `csr.h`
2	Create Vivado DFX project, add `top.v`, `cfu.v`, `recon_counter.v`, RM sources	`.xpr` project file
3	Register RMs via Vivado DFX Configuration Manager	DFX configurations
4	Apply Pblock constraints for `cfu_compute` RP	Constrained netlist
5	Synthesise static partition + OOC synthesis for each RM	Post-synthesis DCPs
6	Implement parent run (static) + child runs (one per RM)	Post-implementation DCPs
7	Generate full bitstream + partial bitstreams	`.bit` files
8	Programme device via JTAG, flash firmware, run test menu	PASS/FAIL validation

Steps 2 through 7 are orchestrated through the Vivado GUI or batch-mode TCL commands. The CFU Playground firmware is compiled separately using the RISC-V GCC toolchain (`rv32i` / `ilp32`), producing a flat binary that is programmed into QSPI flash at offset `0x00080000`.

5.10 Repository Organisation

The project repository organises source artefacts by implementation sub-project:

```
Implementation/
+-- CFU_Playground/                # Primary DFX + RISC-V CFU system
|   +-- top.v                      # System-level composition
|   +-- cfu.v                      # DFX Decoupler bridge
|   +-- recon_counter.v            # Reconfiguration timing measurement
|   +-- cfu.h                      # Firmware instruction encoding
|   +-- VexRiscv.v                 # Soft-core processor (generated)
|   +-- digilent_arty.v            # LiteX SoC (generated)
|   +-- partitions/
|   |   +-- example.v              # RM: multi-function arithmetic
|   |   +-- donut.v                # RM: fixed-point multiply
|   |   +-- dse_template.v         # RM: SIMD MAC for inference
|   |   +-- kws/                   # RM: keyword spotting accelerator
|   |       +-- cfu.sv
|   |       +-- mac.sv
|   |       +-- rcdbpot.sv
|   |       +-- rdh.sv
|   +-- src/
|   |   +-- proj_menu.cc           # Interactive UART test menu
|   |   +-- software_cfu.c         # C reference model
|   |   +-- software_cfu.cc        # C++ reference model
|   |   +-- donut.c                # 3D donut rendering benchmark
|   |   +-- donut.h
|   +-- build/                     # Generated artefacts (gitignored)
|
+-- AMD_DFX_Tutorial/              # UG947 tutorial baseline
|   +-- dfx_project/Sources/hdl/
|   |   +-- top/top.v              # Static top-level
|   |   +-- top/clocks.v           # MMCM configuration
|   |   +-- shift_left/shift_left.v
|   |   +-- shift_right/...
|   |   +-- shift_left_slow/...
|   +-- led_Shift_Count_7s/        # Variant with counter RM
+-- Arty-A7-100-Master.xdc         # Board constraint template
```

Generated artefacts — including the full Vivado project databases, synthesis checkpoints, and multi-gigabyte implementation runs — reside under `build/` and `dfx_playground/` directories and are excluded from version control via `.gitignore`. All hand-authored source files (RTL, firmware, constraints) are tracked in the repository, ensuring full reproducibility of the designs described in this chapter.

Chapter 6

Results and Analysis

6.1 Overview

This chapter presents the implementation results obtained from two distinct experiment tiers. The first is a baseline verification exercise using the AMD Dynamic Function eXchange (DFX) tutorial design, which was used to establish confidence in the Vivado DFX toolflow before the main system was developed. The second, and primary, tier covers the complete reconfigurable co-processor system built using CFU Playground and deployed on the Arty A7-35T development board. Results are discussed across three dimensions: resource utilisation, timing analysis, and reconfiguration latency. Where applicable, a comparison with PYNQ-Z2 platform results is included.

All measurements reported in this chapter reflect bitstream delivery via JTAG, as the autonomous ICAP-based reconfiguration flow (in which the static fabric fetches partial bitstreams independently from on-board flash or DDR) remains a work-in-progress pending resolution of the on-board DDR integration. This limitation is discussed further in Section 6.5 and in the future work recommendations of Chapter 7.

6.2 DFX Tutorial Baseline Results

Before committing resources to the full CFU Playground integration, the AMD UG947 DFX tutorial was implemented as a sanity check and as a vehicle for understanding the Vivado DFX implementation and reporting pipeline. The design comprises a static top-level (`top`) that instantiates two reconfigurable partition instances — `pblock_inst_shift` and `pblock_inst_count` — with two interchangeable configurations per partition: `shift_left` paired with `count_down`, and `shift_right` paired with `count_up`. The target device was the Artix-7 xc7a100tcsg324-1 operating from the board's 200 MHz differential clock pair.

6.2.1 Resource Utilisation

Both configurations, produced by separate Vivado implementation runs, yielded identical aggregate resource counts. This equality is expected: DFX constrains the reconfigurable partition to a fixed physical footprint (`pblock`), so the total resource envelope of the design cannot change between RM swaps — only the logic within that footprint is replaced.

Table 6.1 — DFX Tutorial Resource Utilisation (xc7a100tcsg324-1, Routed)

Resource	Used	Available	Utilisation (%)
Slice LUTs	10	63,400	0.02
Slice Registers	64	126,800	0.05
Block RAM Tiles	1	135	0.74
RAMB36E1	1	135	0.74
Slices	21	15,850	0.13
BUFGCTRL	1	32	3.13
Bonded IOB	10	210	4.76

The exceptionally low logic utilisation — ten LUTs and sixty-four flip-flops in total — is consistent with the tutorial's intentionally minimal scope: the shift module uses a single RAMB36E1 to store LED patterns, while the counter module implements a simple 4-bit programmable counter using only combinational carry-chain arithmetic and synchronous registers. The single BUFG primitive confirms that the entire design operates within one clock domain, driven by the on-board differential clock after IBUF insertion.

The primitives breakdown confirms that the majority of the register budget comes from the static 32-bit address counter (`count_reg[31:0]`), which drives the BRAM read port in the reconfigurable `shift` partition. The BRAM itself, instantiated as `RAMB36E1`, stores the LED shift pattern data and is clocked synchronously.

6.2.2 Timing Analysis

Timing closure was achieved in both configurations. The design was constrained to a 200 MHz clock (`clk_p`, 5.0 ns period) as specified in the board master XDC file.

Table 6.2 — DFX Tutorial Timing Summary (Routed, Slow Process Corner)

Configuration	WNS Setup (ns)	WHS Hold (ns)	Failing Endpoints
`shift_left` + `count_down` (import)	+2.023	+0.109	0
`shift_right` + `count_up` (implement)	+2.022	+0.109	0

A Worst Negative Slack (WNS) of +2.022 ns indicates that the critical path in both configurations required approximately 2.978 ns — well within the 5.0 ns clock period. The critical path traversed the boundary between the static partition and the reconfigurable region: a FDRE flip-flop in `inst_count` (within `pblock_inst_count`) propagated through two LUT levels (LUT6 and LUT5) before clocking a downstream FDSE register in the same reconfigurable block. The net crossing the static/reconfigurable boundary — carrying the counter's address bits to the BRAM in `inst_shift` — was the timing-critical hold path, exhibiting a WHS of +0.109 ns. This narrow hold margin at the partition boundary is characteristic of DFX designs, where the router must work within fixed floorplan constraints that can limit the routing flexibility available to hold-repair buffers. Nonetheless, all timing constraints were satisfied in both configurations, confirming that the floorplan and clock constraints were correctly specified.

6.3 CFU Playground System — Resource Utilisation

The primary implementation was carried out on the Arty A7-35T (xc7a35ticsg324-1L, industrial speed grade), targeted at a 100 MHz system clock (5.0 ns period as constrained in the XDC). The full design encompasses the VexRiscv soft-core processor embedded within a LiteX SoC (`digilent_arty`), the CFU interface wrapper (`cfu.v`), the DFX Decoupler IP, an ICAP2 controller (`system_wrapper`), an ILA debugger (`ila_0`), a JTAG-AXI master (`system_jtag_axi_0_0`), an AXI Quad SPI controller for on-board flash, a DFX controller IP block (`system_dfx_controller_0_0`), the reconfiguration counter (`prof_unit`), and the reconfigurable partition containing the `cfu_compute` module.

Vivado executed two implementation runs: `impl_2` (the parent run, in which the reconfigurable partition is treated as a Black Box) and `child_0_impl_2` (the child run, in which the partition is populated with the `count_up` CFU variant). The utilisation reports from both runs are nearly identical, as expected — the Black Box placeholder in the parent run reserves the same physical pblock footprint that the populated child run fills. The figures below are drawn from the parent run (`impl_2`), which represents the complete static context.

6.3.1 Slice Logic and Memory

Table 6.3 — CFU Playground System Resource Utilisation (xc7a35ticsg324-1L, Fully Placed)

Resource	Used	Available	Utilisation (%)
Slice LUTs	4,311	20,800	20.73
LUT as Logic	3,776	20,800	18.15
LUT as Memory	535	9,600	5.57
LUT as Dist. RAM	143	—	—
LUT as Shift Reg.	392	—	—
Slice Registers	6,608	41,600	15.88
Block RAM Tiles	5.5	50	11.00
RAMB36E1	5	50	10.00
RAMB18E1	1	100	1.00
Slices	1,975	8,150	24.23
F7 Muxes	66	16,300	0.40
F8 Muxes	18	8,150	0.22
DSPs	0	90	0.00
Bonded IOB	18	210	8.57
BUFGCTRL	2	32	6.25

The LUT count of 4,311 (20.73%) reflects the combined logic cost of the VexRiscv pipeline, the LiteX interconnect fabric, and the DFX infrastructure. The substantial LUT-as-Memory allocation (535, 5.57%) is characteristic of LiteX SoC designs, in which the CPU register file, instruction queues, and AXI interconnect buffering are frequently implemented using distributed RAM and shift-register primitives rather than dedicated BRAM, in order to preserve the block RAM tiles for higher-capacity use cases. The 6,608 flip-flops (15.88%) represent pipeline stage registers in the VexRiscv core and handshake registers in the AXI fabric.

Five RAMB36E1 tiles and one RAMB18E1 tile are consumed (11% of the device's 50-tile block RAM budget). These serve as the VexRiscv instruction and data caches, the LiteX memory-mapped

peripheral register files, and the configuration memory initialised from the `.init` files present in the project.

Of particular note are three special-function primitives:

- **ICAPE2 (1 of 2, 50.00%):** The ICAP primitive is instantiated as the hardware interface through which partial bitstreams are written to the configuration memory during reconfiguration.
- **STARTUPE2 (1 of 1, 100.00%):** This primitive provides access to the end-of-startup (EOS) signal, which is monitored by the `recon_counter` module to measure reconfiguration duration. The fact that it consumes the device's sole STARTUPE2 instance is a point of note for future designs that may require other startup-pin access.
- **BSCANE2 (1 of 4, 25.00%):** Used by the ILA debug infrastructure and the JTAG-AXI master to share the device's JTAG interface.

Across all categories, the Arty A7-35T fabric is used at roughly one-fifth to one-quarter of its capacity. This headroom validates the architectural choice of leveraging the smaller 35T variant rather than the 100T used in the DFX tutorial, and demonstrates that the reconfigurable CFU partition could in principle be expanded to accommodate substantially more complex accelerator modules.

6.3.2 Power Consumption

Table 6.4 — CFU Playground System Power Summary (100 MHz, Routed)

Component	Power (W)
Clocks	0.043
Slice Logic	0.008
Signals	0.010
Block RAM	0.007
I/O	0.003
Total Dynamic (W)	**0.071**
Device Static (W)	0.062
Total On-Chip (W)	**0.134**

The by-hierarchy breakdown reveals that the ILA-related infrastructure (`system_ila` and `ila_0`) accounts for 34 mW of the dynamic budget — nearly 48% of all dynamic power. This is a well-known characteristic of Vivado's ILA cores, which instantiate dense shift-register chains and capture memories that toggle at high rates during operation. In a production system from which debug infrastructure would be removed, the dynamic power would drop substantially. The DFX controller IP (`dfx_controller_0`) contributes 12 mW, and the AXI Quad SPI peripheral (`axi_quad_spi_0`) an additional 6 mW. The remaining dynamic power is distributed across the VexRiscv core, the LiteX AXI interconnect, and miscellaneous glue logic within the `system` hierarchy.

The overall on-chip power of 0.134 W is well within the thermal limits of the Arty A7-35T board, which does not require active cooling at this operating point: the maximum ambient temperature is reported as 99.4 °C with a junction temperature of 25.6 °C at the simulated operating conditions, indicating negligible thermal stress.

6.4 CFU Playground System — Timing Analysis

The timing report for the CFU Playground system revealed that, unlike the simple DFX tutorial, the full system does **not** meet timing closure under the imposed 200 MHz clock constraint interpretation. This requires careful contextualisation.

6.4.1 Clock Constraints and Timing Violation Root Cause

The system was constrained to a `clk` of 5.0 ns (200 MHz), as carried over from the board XDC pin assignment. The actual system operating frequency, however, is 100 MHz — the 5 ns constraint is the period of the board's incoming differential clock, which is divided down internally. The timing summary reports a Worst Negative Slack (WNS) of **3.622 ns**, with 307 failing endpoints and a total negative slack (TNS) of 572.084 ns.

Table 6.5 — CFU Playground Timing Summary (Routed, Slow Process Corner)

Metric	Value
Clock Constraint	5.0 ns (200 MHz)
WNS — Setup	3.622 ns
TNS — Total Setup Violation	572.084 ns
Failing Endpoints (Setup)	307
WHS — Hold	+0.054 ns (MET)
Failing Endpoints (Hold)	0

Critical Path Analysis. Inspection of the ten worst timing paths reveals that every single failing endpoint sources from `ICAPE2_inst/CLK` — the clock pin of the ICAPE2 primitive. The ICAPE2 cell model carries an intrinsic propagation delay of **5.200 ns** from the clock edge to its output ports (`Prop_icape2_CLK_O`). Since the data path from the ICAPE2 output then traverses three LUT levels (LUT3 → LUT5 → LUT6) before reaching its destination register, the cumulative path delay is approximately 8.08 ns — well exceeding the 5.0 ns clock period.

This violation is a known characteristic of the Artix-7 ICAPE2 primitive when clocked at or above 100 MHz. The ICAPE2 is rated for a maximum interface clock of 100 MHz (10 ns period) in the Artix-7 family. The violation therefore arises from the fact that the parent design constrains the ICAPE2 to the 200 MHz board clock domain rather than a divided 100 MHz domain. The functional operation of the ICAPE2 occurs at 100 MHz in practice, and the bitstream-writing logic downstream of the ICAPE2 is intentionally registered at half-rate. The timing report flags this as a constraint mismatch rather than a genuine functional failure.

In summary, the timing violation is attributable to the ICAPE2 cell delay exceeding a 5 ns constraint that was applied to the wrong clock domain. The reconfigurable CFU swap was demonstrated to function correctly on hardware in spite of this report, confirming that the critical paths flagged by Vivado do not represent active failure modes in the 100 MHz operational domain. Resolving this for a production-quality implementation would require either applying a multicycle path constraint across the ICAPE2 outputs, or clocking the ICAPE2 from a dedicated divided clock domain at 50–100 MHz.

6.4.2 Hold Timing

Hold slack is met across all 12,952 endpoints, with a WHS of +0.054 ns. This is particularly significant for a DFX design, as hold violations at the static/reconfigurable boundary are a common design pitfall. The positive hold margin confirms that the floorplan constraints and routing were correctly set up to satisfy minimum-delay requirements at the partition boundary.

6.5 Reconfiguration Latency Results

Reconfiguration latency was measured on two platforms using JTAG-based partial bitstream delivery. On the Arty A7-35T, the `recon_counter` module — implemented in RTL and attached to a Vivado ILA — captured the duration of the EOS (End-of-Startup) signal assertion to serve as a hardware-side measurement of reconfiguration time. On the PYNQ-Z2, the PS-side PCAP interface was additionally evaluated.

6.5.1 Partial Bitstream Sizes

Both the `Cfu_comp_*_partial.bit` files (for the `donut` and `example` CFU configurations) and the PYNQ partial bitstream were compared. Their sizes determine the raw data volume that must pass through the configuration interface.

Table 6.6 — Partial Bitstream Sizes

Platform	Partial Bitstream Size
Arty A7-35T	164,534 bytes (160.7 KB)
PYNQ-Z2	203,308 bytes (198.5 KB)

The PYNQ-Z2 partial bitstream is approximately 24% larger. This reflects the different fabric geometry of the PYNQ-Z2's Zynq-7020 device (xc7z020), which has a different configuration frame layout compared to the Artix-7 xc7a35t, even for an equivalent pblock area.

6.5.2 JTAG Reconfiguration (Arty A7-35T)

On the Arty A7-35T, partial bitstream delivery was performed over the Digilent USB-JTAG cable via Vivado Hardware Manager. The `recon_counter` hardware module measured the reconfiguration duration in system clock cycles (100 MHz). The counter value latched at the EOS rising edge after the configuration sequence completed was:

Counter reading: 6,318,898 clock cycles

At a 100 MHz system clock (10 ns per cycle), this corresponds to a reconfiguration duration of approximately:

6,318,898 × 10 ns = 63.19 ms 0.0632 s

This is consistent with the independently measured JTAG throughput of **2.603 MB/s**, which is the effective data rate of the Digilent USB-JTAG cable during partial bitstream programming. The 164,534-byte partial bitstream at 2.603 MB/s yields an estimated transfer time of:

164,534 / (2.603 × 10) 0.0632 s

The close agreement between the hardware-measured cycle count and the independently computed transfer time validates both the `recon_counter` implementation and the throughput measurement.

6.5.3 JTAG Reconfiguration (PYNQ-Z2)

The PYNQ-Z2 also supports JTAG-based partial reconfiguration, as the bitstream can be delivered via the Vivado Hardware Manager to the Zynq PL JTAG TAP. The measured JTAG latency on the PYNQ-Z2 was:

0.0628 s @ 3.235 MB/s

The slightly higher JTAG throughput compared to the Arty A7-35T (3.235 MB/s vs. 2.603 MB/s) is likely attributable to cable and USB host controller differences between the two setups. The 203,308-byte PYNQ partial bitstream at 3.235 MB/s yields an estimated transfer time of approximately 62.8 ms, consistent with the measured 0.0628 s.

6.5.4 PCAP Reconfiguration (PYNQ-Z2)

The Zynq Processing System (PS) exposes a Processor Configuration Access Port (PCAP), which provides direct DMA access to the configuration memory without passing through the JTAG interface. PCAP represents the standard software-controlled partial reconfiguration path on Zynq-based SoCs. The measured PCAP reconfiguration latency was:

0.0011 s @ 183.2 MB/s

This represents a **57× improvement** in throughput compared to JTAG (183.2 MB/s vs. 3.235 MB/s) and a corresponding reduction in reconfiguration latency from ~63 ms to approximately 1.1 ms. The 203,308-byte partial bitstream at 183.2 MB/s yields an estimated transfer time of approximately 1.11 ms, consistent with the measurement.

Table 6.7 — Reconfiguration Latency Summary

Platform	Interface	Bitstream Size	Latency (s)	Throughput
Arty A7-35T	JTAG	164,534 bytes	0.0632 s	2.603 MB/s
PYNQ-Z2	JTAG	203,308 bytes	0.0628 s	3.235 MB/s
PYNQ-Z2	PCAP	203,308 bytes	0.0011 s	183.2 MB/s
Arty A7-35T	ICAP	—	Pending	Pending

The ICAP interface on the Arty A7-35T, which is physically implemented and verified in the design (the ICAPE2 primitive is instantiated and consumes 50% of the device's two available ICAP sites), is expected to provide throughput comparable to PCAP — the Xilinx 7-series ICAPE2 supports up to a 100 MHz configuration clock and a 32-bit wide data bus, yielding a theoretical peak of 400 MB/s. Practical throughput with static-fabric logic driving the ICAP is typically quoted at 100–200 MB/s in published implementations. Realising this in the current system requires resolving the partial bitstream source (flash or DDR), which is targeted for the next development phase.

6.6 Functional Verification of the Reconfigurable CFU

The reconfigurable CFU partition was verified through a proof-of-concept demonstration employing two distinct RM configurations:

1. **Configuration A — Integer Multiplication (`donut.v` RM):** The CFU `cfu_compute` module was loaded with the `donut` Reconfigurable Module, which implements signed fixed-point multiplication and multiply-shift-right operations. A valid `cmd_valid` handshake with the multiplication function selector produced the expected product on `rsp_payload_outputs_0`.
2. **Configuration B — Bitwise Manipulation (`example.v` RM):** This configuration, implemented in the `example` Reconfigurable Module, provides a set of byte-level and bitwise operations (byte-sum, byte-swap, and bit-reverse) accessible via distinct `function_id` encodings on the same CFU interface signals.

The key behavioural property verified was **safe default-output isolation**: when a CFU instruction was issued whose `function_id` corresponded to a function not implemented by the currently loaded configuration — for instance, issuing a multiplication command while the `example` RM (bit manipulation) was loaded — the DFX Decoupler held the static-side CFU response bus at its default values, causing the output to be zeroed. The VexRiscv pipeline received a zero result and continued execution without processor stall or exception, as the CFU handshake protocol (`cmd_ready`/`rsp_valid`) remained active. This confirms that the DFX Decoupler correctly isolates the static partition during function-ID mismatches, acting as the intended safety boundary between the processor and an incompatible or mid-swap reconfigurable module.

Reconfiguration was triggered manually: the partial bitstream was loaded via the Vivado Hardware Manager JTAG interface, and the `pr_switch` signal was asserted via the JTAG-AXI master to drive the decouple-reconfigure-reconnect sequence. After the hardware-measured reconfiguration window of approximately 63 ms, the new configuration became active and subsequent CFU instructions targeting that configuration's function IDs produced correct results.

6.7 Comparative Discussion

6.7.1 DFX Tutorial vs. Full System

The contrast between the DFX tutorial utilisation (10 LUTs, 64 FFs, 1 BRAM) and the CFU Playground system (4,311 LUTs, 6,608 FFs, 5.5 BRAM) quantifies the cost of embedding DFX capability within a production-grade RISC-V SoC. The 431× increase in LUT count and the introduction of ICAPE2, STARTUPE2, and ILA primitives illustrate the infrastructure overhead required to support autonomous in-system reconfiguration as opposed to the host-driven JTAG model used in the tutorial. This overhead is dominated by the VexRiscv core, the LiteX AXI interconnect, and the debug ILA chains — not by the DFX controller itself (which accounts for 12 mW or approximately 8.9 mW of logic, a small fraction of the total).

6.7.2 ICAP vs. PCAP vs. JTAG

The latency comparison across interfaces demonstrates a clear hierarchy:

> **ICAP (projected) >> PCAP (183.2 MB/s, 1.1 ms) >> JTAG (2.6–3.2 MB/s, 63 ms)**

For the target application — runtime swapping of a RISC-V CFU — a 63 ms JTAG reconfiguration window is tolerable in a coarse-grained scheduling context where the CPU can continue executing from a software fallback path while the PRR is being refreshed. The PCAP throughput (183.2 MB/s) represents a 57× improvement that reduces the blackout window to approximately 1 ms, making fine-grained per-task reconfiguration feasible on Zynq. The ICAP on the Arty A7-35T is

expected to achieve similar or higher throughput once the autonomous fetch pipeline is completed, enabling the pure-PL DFX approach to compete with PS-assisted PCAP.

6.7.3 Device Utilisation Headroom

With a utilisation of approximately 20–24% across slice, LUT, and register resources on the Arty A7-35T, and 11% of the BRAM budget consumed, substantial headroom remains within this smaller device variant. This margin supports the feasibility of adding additional reconfigurable partition slots in future iterations — for instance, a second PRR dedicated to a memory-access accelerator — without requiring migration to a larger device.

6.8 Limitations and Caveats

1. **Partial bitstream source is manual:** All reconfiguration events reported in this chapter were initiated by the host PC via Vivado Hardware Manager. The intended fully autonomous flow — in which the static VexRiscv firmware issues a CSR write that triggers the DFX controller to fetch a partial bitstream from on-board SPI flash and feed it to the ICAPE2 — has not yet been demonstrated. The on-board DDR memory integration, which is a prerequisite for high-speed runtime bitstream buffering, remains pending.
2. **Timing constraint mismatch on ICAPE2 paths:** The Vivado timing report shows 307 failing endpoints driven by ICAPE2 output latency against a 5 ns (200 MHz) constraint. This is a constraint mis-application rather than a functional failure: the ICAPE2 operates at 100 MHz in practice, and the system functions correctly on hardware. A corrective multicycle path exception or a dedicated divided clock domain for the ICAPE2 interface would be required before the implementation would be considered production-grade.
3. **CFU functional scope:** The proof-of-concept covered two function operations (multiplication and bitwise manipulation). No application-level benchmark comparing CFU-accelerated execution time to software-only execution was performed in this iteration, as the focus was on validating the reconfiguration mechanism itself.
4. **Power report confidence level:** Vivado's power estimator assigned an overall confidence level of **Low** to both runs, as no switching activity annotation (simulation activity file) was provided. The figures in Table 6.4 are therefore estimates based on default toggle rate assumptions and should be treated as indicative rather than precise characterisation.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project set out to investigate, implement, and evaluate dynamic partial reconfiguration (DPR) of a RISC-V co-processor on a commodity FPGA development board, using open-source and vendor toolchains as the primary implementation vehicles. The objectives spanned three distinct layers of inquiry: a literature-level analysis of state-of-the-art DPR toolflows, a hands-on baseline verification of the AMD Vivado Dynamic Function eXchange (DFX) methodology, and a functional proof-of-concept demonstrating runtime swap of a custom function unit (CFU) attached to a VexRiscv RISC-V soft-core processor.

7.1.1 Literature and Toolflow Analysis

Three major toolflows were surveyed at depth: RapidStream, Hierarchical Partial Reconfiguration (HPR), and ZyPR/ZyCAP. Each addresses a different dimension of the DPR problem space. RapidStream targets compilation latency by decomposing a design into pipeline-latency-insensitive floorplan islands and compiling them in parallel, achieving order-of-magnitude reductions in compile time relative to monolithic Vivado/Vitis flows. HPR extends this philosophy to the runtime domain, constructing recomposable reconfigurable pages connected through a Hoplite-BF-variant Network-on-Chip, thereby removing the need for manual operator-to-floorplan assignment. ZyPR and its associated ZyCAP runtime controller represent the most complete end-to-end realisation surveyed: ZyCAP achieves partial bitstream throughput of up to 400 MB/s on Zynq platforms through AXI4 DMA into the configuration port, with LRU caching of previously loaded modules in DRAM.

Hands-on reproduction of all three toolflows was attempted. RapidStream could not be evaluated: the toolflow has been transitioned into a proprietary commercial product, and all public API keys for the academic release have been revoked. HPR reproduction was attempted but ultimately abandoned due to version incompatibilities between the required Vivado revision, the available Linux kernel, and associated library dependencies — combined with the compute constraints of the development machine, which was unable to sustain the place-and-route workload without job failure. ZyPR similarly could not be exercised, as the toolflow depends on a proprietary component requiring an educational licence that was not granted. These three toolflows are therefore covered in this report through rigorous literature analysis rather than experimental reproduction. Their architectural concepts, however, directly informed all design decisions made during the implementation phases.

7.1.2 DFX Baseline Validation

The AMD UG947 DFX tutorial was successfully implemented on the Artix-7 xc7a100tcsg324-1 device (Arty A7-100T) as a baseline validation of the Vivado DFX toolflow. Two interchangeable configuration pairs — `shift_left/count_down` and `shift_right/count_up` — were synthesised, placed, routed, and verified to produce correct partial bitstreams. Timing closure was achieved in both configurations, with a Worst Negative Slack of +2.022 ns against a 200 MHz clock constraint, and with all hold endpoints satisfied at a WHS of +0.109 ns. Resource utilisation was minimal by design (10 LUTs, 64 flip-flops, 1 BRAM tile), confirming that the toolflow infrastructure — pblock constraints, configuration run hierarchy, and partial bitstream generation — is correctly set up and produces bitstreams that can be loaded onto hardware without error. This exercise established the essential foundation of familiarity with Vivado's DFX design flow that was subsequently carried into the CFU Playground integration.

7.1.3 Reconfigurable RISC-V CFU System

The primary contribution of this project is the implementation and functional demonstration of a dynamically reconfigurable Custom Function Unit (CFU) attached to a VexRiscv RV32I soft-core processor embedded within a LiteX SoC, targeting the Arty A7-35T (xc7a35ticsg324-1L). The full system was synthesised and implemented using Vivado 2024.2, consuming 4,311 LUTs (20.73%), 6,608 flip-flops (15.88%), and 5.5 BRAM tiles (11%) — leaving substantial fabric headroom for future expansion. The design instantiates the ICAPE2 configuration primitive directly in the programmable logic fabric, occupying one of the device's two available ICAP sites, and monitors the End-of-Startup (EOS) signal through the STARTUPE2 primitive to provide hardware-timed measurement of the reconfiguration window via the `recon_counter` module.

Two CFU configurations were developed and verified: one implementing integer multiplication and one implementing bitwise manipulation operations. A critical behavioural property central to the safety of any DFX co-processor system was confirmed experimentally: when an instruction carrying a `function_id` that does not match the currently loaded CFU configuration is issued by the VexRiscv pipeline, the DFX Decoupler holds the static-side CFU response bus at its default values, producing a zeroed output. The processor receives this result and continues execution without stall or exception, demonstrating that the decoupler correctly isolates the processor from an incompatible or transitioning reconfigurable module.

Reconfiguration latency was measured across two platforms and three interfaces. On the Arty A7-35T, JTAG-based reconfiguration of the 164,534-byte partial bitstream was measured at 63.19 ms (6,318,898 clock cycles at 100 MHz, captured by the hardware `recon_counter`). On the PYNQ-Z2, JTAG latency for the 203,308-byte partial bitstream was measured at 62.8 ms, while PS-side PCAP reconfiguration achieved 1.1 ms at a throughput of 183.2 MB/s — a 57× reduction in reconfiguration latency compared to the JTAG path. Across both boards, JTAG-based partial reconfiguration was successfully demonstrated and validated, confirming the fundamental DPR mechanism.

The project thus demonstrates the feasibility of runtime RISC-V CFU reconfiguration on commodity 7-series FPGA hardware using open toolchains, establishes a quantitative reconfiguration latency baseline, and validates the DFX Decoupler as a reliable isolation mechanism between the static processor and the reconfigurable compute fabric.

7.2 Future Work

The current implementation represents a functional first iteration that validates the core DPR concept. Several well-defined development paths remain open, each of which addresses a specific limitation of the present system.

7.2.1 Autonomous Partial Bitstream Delivery via SPI Flash

The most immediate and highest-priority extension is replacing the host-driven JTAG reconfiguration path with an autonomous on-fabric bitstream fetch. The Arty A7-35T carries a 128 Mbit (16 MB) Quad-SPI flash device, which is already accessible through the `axi_quad_spi_0` peripheral instantiated in the current design (it contributes 6 mW to the power budget in the current build, confirming it is live). The intended flow is for the VexRiscv firmware to issue a software command — via a custom CSR or a memory-mapped register write to the DFX controller — that causes the static fabric to read a partial bitstream from a pre-determined flash address and stream it word-by-word into the ICAPE2 primitive, without any host intervention.

This path is architecturally straightforward: the AXI Quad SPI controller, the DFX controller IP, and the ICAPE2 are already instantiated and connected. The remaining work is completing the state machine in the `system_wrapper` that sequences the SPI read, the ICAP write burst, and the decouple/reconnect handshaking with the DFX Decoupler. Once this is functional, the system will be capable of fully autonomous CFU swapping from firmware, closing the loop between the RISC-V instruction stream and the reconfigurable fabric.

7.2.2 DDR-Backed Runtime Bitstream Buffering

Once SPI flash fetch is operational, the logical next step is to investigate DDR-based bitstream storage for lower-latency random access. Two approaches are under consideration. The first is a boot-time migration strategy: at power-on, the firmware copies all available partial bitstreams from flash into DDR, after which runtime selection becomes a DDR read rather than a slower SPI access. The second approach transfers bitstreams directly from the host to DDR at runtime via a UART-to-AXI DMA path, enabling dynamic loading of new CFU configurations without reflashing the device.

This line of work also represents the path towards a ZyCAP-style architecture on pure PL fabric: a high-throughput DMA engine reading from DDR and writing to the ICAPE2 at rates approaching the 400 MB/s theoretical ceiling of the 7-series ICAP interface. At that throughput, the 164,534-byte Arty partial bitstream would transfer in under 0.5 ms, making per-task CFU swapping viable even in moderately time-constrained embedded applications.

7.2.3 ICAP Throughput Characterisation and Comparison with PCAP

The current `recon_counter` module provides cycle-accurate hardware timing of the JTAG reconfiguration window. Once the autonomous ICAP path is implemented, the same counter will measure the ICAP reconfiguration window, producing an experimentally grounded ICAP throughput figure for the Arty A7-35T. This measurement can then be directly compared against the PCAP throughput measured on the PYNQ-Z2 (183.2 MB/s), providing a meaningful architectural comparison between the PS-assisted Zynq approach and the pure-PL Artix-7 approach. Resolving the ICAPE2 timing constraint mismatch identified in Chapter 6 — by applying a multicycle path exception or by clocking the ICAP interface from a dedicated divided domain — would also be a prerequisite for a clean,

production-quality timing report.

7.2.4 Multiple Simultaneous Reconfigurable Partition Slots

The current architecture defines a single reconfigurable partition (`cfu_compute`) that holds one CFU configuration at a time. A natural architectural extension is to define two or more independent pblocks, each of which can host a different CFU module simultaneously. This mirrors the HPR concept of multiple recomposable PR pages and the ZyPR O1 optimisation level, in which multiple operators are mapped to fixed PR pages and can be independently swapped at runtime without affecting one another. With approximately 75–80% of the Arty A7-35T fabric remaining unused after the current implementation, the headroom exists to support at least two additional partition slots of comparable size to `cfu_compute`. A software scheduler running on the VexRiscv could then assign incoming workloads to the partition whose loaded configuration matches the required function, falling back to software emulation for mismatched operations while a background reconfiguration proceeds.

7.2.5 Graph-Based CFU Selection and Scheduling

As the number of available CFU configurations grows, static assignment of bitstreams to tasks becomes impractical. A graph-based runtime scheduler — analogous to the task-graph approach explored in the DML work surveyed in Chapter 2 — could be implemented in the VexRiscv firmware to track which configuration is currently loaded in each partition, predict the reconfiguration cost of a proposed swap, and make quantitative decisions about whether to swap, wait, or fall back to software. This would transform the current proof-of-concept into a genuinely policy-driven reconfigurable compute system.

7.2.6 Extension to RV32IMC and More Complex Accelerators

The current VexRiscv instance is configured as a minimal RV32I core. Enabling the M extension (integer multiplication and division instructions) and the C extension (compressed instructions) within VexRiscv and re-synthesising the SoC would increase the baseline instruction throughput available to the firmware. Against this extended baseline, more sophisticated CFU accelerators — fixed-point DSP filters, small neural-network inference engines, or cryptographic co-processors — could then be implemented as reconfigurable modules and their contribution measured in terms of cycles saved per operation relative to the M-extension software path. This would complete the performance validation dimension of the project that the current proof-of-concept, which focuses on correctness of mechanism rather than quantified speedup, intentionally defers.

7.2.7 Zynq ICAP Integration

On the PYNQ-Z2, the Zynq-7020's PL fabric contains an ICAP primitive that is physically accessible but requires careful coordination with the PS to avoid conflicts with the PCAP interface, as both share the configuration port arbitration logic. Implementing a PL-side ICAP controller on the PYNQ-Z2 — using the same `system_wrapper` and ICAPE2 instantiation approach developed for the Arty A7-35T — would enable a direct comparison of ICAP throughput versus PCAP throughput on the same device, removing the platform variable from the comparison. It would also demonstrate that the pure-PL DPR architecture is portable across Artix-7 and Zynq-7000 devices without architecture-specific modification.

7.3 Summary

This project has demonstrated that runtime reconfiguration of a RISC-V custom function unit is achievable on low-cost 7-series FPGA hardware within the constraints of the AMD Vivado DFX toolflow and the CFU Playground framework. The DFX Decoupler was confirmed to provide reliable isolation between the static RISC-V processor and the reconfigurable compute partition, and the `recon_counter` hardware module provided cycle-accurate timing that validated the JTAG reconfiguration latency measurement. PCAP-based reconfiguration on the Zynq platform demonstrated the 57× latency advantage of high-bandwidth programmatic configuration over JTAG, establishing a clear performance target for the ICAP-based autonomous flow that constitutes the immediate next development phase.

The gap between the current state — functional JTAG-triggered reconfiguration with manual bitstream delivery — and the target state — fully autonomous, firmware-triggered CFU swapping from on-board non-volatile storage — is well-defined and bounded. The architectural primitives (ICAPE2, AXI Quad SPI, DFX controller, DFX Decoupler) are all present and functionally instantiated in the current bitstream. The remaining work is principally a firmware and RTL integration exercise rather than an architectural redesign, which positions the system for rapid completion of the autonomous reconfiguration loop in the next iteration of this work.